

TECHNICAL DOCUMENTATION

Telegram API Technical Documentation

Telegram OSINT Collection Service — architecture, endpoint reference, Telegram integration, security and operational analysis

SERVICE

Telegram OSINT Collection Service (`telegram_service`)

CODEBASE VERSION

0.1.0

DOCUMENT VERSION

1.0

DATE

6 September 2026

PREPARED FOR

Team Leader / Technical Lead

CLASSIFICATION

Internal — Confidential

SCOPE

Complete static analysis of the source tree, the runtime SQLite database, and the test suite

ENDPOINTS DOCUMENTED

35 (all published endpoints)

Source of truth. Every statement in this document is derived from the actual implementation. Where existing project documentation disagreed with the code, the code was taken as authoritative and the discrepancy is recorded in section 18. Functionality that is not implemented is explicitly marked as such rather than assumed. No real credentials, secrets, session data or collected message content appear anywhere in this document.

Table of Contents

1. Executive Summary	4	5.25 POST /api/telegram/search/results/save	53
2. System Architecture	7	5.26 GET /api/telegram/channels/search	56
2.1 Layer model	7	5.27 POST /api/telegram/channels/{telegram_id}/register	57
2.2 Component reference	9	5.28 POST /api/telegram/discovery/extract-links	58
2.3 Runtime shape	10	5.29 POST /api/telegram/bots/{source_id}/start	60
3. Telegram API Integration	11	5.30 POST /api/telegram/invites/join	62
3.1 Two APIs, clearly separated	11	5.31 POST /api/sources/{source_id}/backfill	65
3.2 Telegram API type and client	11	5.32 GET /api/sources/{source_id}/backfill	68
3.3 Authentication mechanism	12	5.33 GET /api/sources/{source_id}/backfill/{job_id}	68
3.4 Session management and the invalid-session latch	12	5.34 GET /api/notifications	69
3.5 Connection lifecycle	12	5.35 POST /api/notifications/{notification_id}/read	71
3.6 How responses are processed	13	6. End-to-End Workflows	72
3.7 How Telegram errors are handled	13	6.1 The access state machine (the backbone of every workflow)	72
4. Endpoint Inventory	14	6.2 Telegram Account Authentication Flow — Implemented	73
5. Detailed Endpoint Documentation	18	6.3 Source Discovery and Registration Flow — Implemented	74
5.1 GET /health	18	6.4 Private Source Access Flow (join request) — Implemented, with a Telegram-imposed blind spot	75
5.2 GET /ready	19	6.5 Message Monitoring / Collection Flow — Implemented	76
5.3 GET /api/telegram/status	19	6.6 Live Search Flow — Implemented	77
5.4 POST /api/telegram/auth/send-code	21	6.7 Historical Backfill Flow — Partially Implemented	77
5.5 POST /api/telegram/auth/verify-code	22	6.8 Bot Interaction Flow — Implemented	78
5.6 POST /api/telegram/auth/verify-password	23	6.9 Notification Flow — Partially Implemented	78
5.7 POST /api/sources	25	6.10 Not Implemented workflows	79
5.8 GET /api/sources	28	7. Telegram Accessibility and Privacy Behavior	80
5.9 GET /api/sources/{source_id}	29	7.1 Access by source type	80
5.10 PATCH /api/sources/{source_id}	29	7.2 Capability matrix	82
5.11 DELETE /api/sources/{source_id}	30	7.3 What "public" means in this service	82
5.12 POST /api/sources/{source_id}/request-access	31	8. Telegram API Limitations	84
5.13 GET /api/sources/{source_id}/access-status	33	8.1 Telegram Platform Limitations	84
5.14 POST /api/sources/{source_id}/check-access	34	8.2 Application Limitations	85
5.15 POST /api/sources/{source_id}/monitoring/start	36	9. Security	87
5.16 POST /api/sources/{source_id}/monitoring/stop	37	9.1 Security architecture as implemented	87
5.17 GET /api/sources/{source_id}/monitoring-status	38	9.2 Findings	87
5.18 GET /api/sources/{source_id}/messages	38	10. Account Safety and Operational Considerations	90
5.19 GET /api/messages/{message_id}	40	10.1 Protections that ARE implemented	90
5.20 GET /api/messages	41		
5.21 GET /api/telegram/search/global	43		
5.22 GET /api/sources/{source_id}/search	47		
5.23 GET /api/telegram/channels/{channel_id}/search	50		
5.24 POST /api/telegram/search/sources	51		

10.2 Protections that are NOT implemented (recommended)	91	15.1 Postman setup	105
10.3 Operational risk by activity	92	15.2 Request reference	105
10.4 Evidence from the running deployment	92	15.3 Handling errors in a client	108
11. Database Interaction	93	16. Error Handling	109
11.1 Schema overview — 11 tables	93	16.1 The two error envelopes	109
11.2 Entity relationships	94	16.2 Centralized error reference	110
11.3 Write paths	94	16.3 Errors that never surface as HTTP errors	111
11.4 Data integrity mechanisms	95	17. Current Implementation Status	113
11.5 Migrations	95	18. Known Issues and Recommendations	117
12. Configuration	96	18.1 Current Issues (defects in behaviour today)	117
12.1 Environment variables	97	18.2 Documentation vs implementation discrepancies	118
12.2 Example .env (placeholders only)	98	18.3 Technical debt	119
13. How to Run the API	99	18.4 Recommended improvements (future work — none of this exists today)	120
13.1 Prerequisites	99	19. API Sequence Diagrams	121
13.2 Installation	99	19.1 Source registration with access verification	121
13.3 Configuration	99	19.2 Private source: join request and 12-hour reconciliation	122
13.4 Starting the service	100	19.3 Scheduled collection cycle	123
13.5 Verifying it is up	100	19.4 Live global search with quota fallback	123
13.6 API documentation	100	19.5 Bot /start and link harvesting	124
13.7 Running the tests	100	19.6 Backfill job (asynchronous)	124
14. How to Use the API	101	20. Final Technical Summary	125
15. Postman / cURL Examples	105		

1. Executive Summary

Service name: Telegram OSINT Collection Service (`telegram_service`) **Codebase version:** `0.1.0` (FastAPI `app.title = "Telegram OSINT Collection Service"`) **Analysis basis:** static read of the complete source tree, the SQLite database produced by the running service, and the test suite. Where a document and the code disagreed, the **code was taken as the source of truth** and the discrepancy is recorded in section 18.

What this service is

A **backend HTTP service that turns one authenticated Telegram user account into a controlled, auditable data-collection API**. It authenticates a single Telegram account over MTProto (Telethon), registers Telegram channels/groups/bots as "sources", determines each source's *real* access status by asking Telegram rather than assuming it, runs a legitimate join-request workflow for private sources, searches Telegram live, and collects messages incrementally into a local SQLite database once — and only once — access has been positively verified.

It is explicitly the **data-collection and access-management layer** of a larger OSINT platform. NLP, OCR, entity extraction, correlation, risk scoring and case management are **Not Implemented** here and are expected to consume this service's normalized tables downstream.

Business / technical objective

Objective	How the implementation meets it
Collect Telegram content lawfully, without bypassing Telegram's access controls	Every read is performed as a normal authenticated user account. Private content is only reached after Telegram itself approves a join request.
Never guess an access status	<code>check_access</code> performs a real Telegram read probe (or <code>CheckChatInviteRequest</code>) and drives an explicit 13-state machine.
Be restart-safe and non-duplicating	Per-message commits, a per-source checkpoint, and a DB-level <code>UNIQUE(source_id, telegram_message_id)</code> constraint.
Be defensible / auditable	Every significant operation writes an <code>audit_logs</code> row; every collected message's raw Telegram payload is preserved to disk and SHA-256 hashed.
Protect the Telegram account	Bounded page sizes, a global search-concurrency cap of 3, in-flight request de-duplication, <code>max_instances=1</code> on all background jobs, no retry-on-FloodWait loops, exactly one join request per source.

Telegram functionality currently supported

Capability	Status
Non-interactive account login (code + 2FA) and session reuse	Implemented
Source registration with identifier normalization and double de-duplication	Implemented
Live access probing and a validated access state machine	Implemented
Join public channel / import invite link / submit approval-gated join request	Implemented
12-hour join-request reconciliation with approval and rejection detection	Implemented
Incremental forward monitoring with checkpointing	Implemented
Historical backward backfill as a background job	Implemented
Live Telegram-wide message search (<code>channels.searchPosts</code> with <code>messages.searchGlobal</code> fallback)	Implemented, quota-limited by Telegram
Live per-channel message search (<code>messages.search</code>)	Implemented
Channel/group discovery by keyword (<code>contacts.search</code>)	Implemented
Telegram-link extraction and classification from free text	Implemented
Bot <code>/start</code> interaction and link harvesting from the reply	Implemented
Media metadata capture; optional policy-gated media download	Implemented (download off by default)
Stored-message retrieval and flat-filter search over collected data	Implemented
Persisted notifications with pluggable outbound dispatch	Implemented (no dispatcher registered by default)
Sending arbitrary messages, posting, reacting, editing, deleting	Not Implemented
Any bypass of Telegram permissions, invite guessing, or membership spoofing	Not Implemented — explicitly out of scope

High-level request flow



External Telegram APIs and libraries used

Item	Value
Protocol	Telegram MTPROTO (not the HTTP Bot API)
Client library	Telethon <code>>=1.36.0</code>
Account type	One Telegram user account (api_id / api_hash from <code>my.telegram.org/apps</code>)
Telegram RPCs used	<code>contacts.resolveUsername</code> / <code>channels.getChannels</code> (via <code>get_entity</code>), <code>messages.getHistory</code> / <code>messages.search</code> (via <code>iter_messages</code>), <code>messages.searchGlobal</code> , <code>channels.searchPosts</code> , <code>channels.checkSearchPostsFlood</code> , <code>contacts.search</code> , <code>messages.checkChatInvite</code> , <code>messages.importChatInvite</code> , <code>channels.joinChannel</code> , <code>messages.sendMessage</code> (bot /start only), <code>auth.sendCode</code> / <code>auth.signIn</code>
Web framework	FastAPI + Uvicorn
Persistence	SQLAlchemy 2.x async ORM over SQLite (<code>aiosqlite</code>)
Scheduling	APScheduler <code>AsyncIOScheduler</code> , embedded in the API process

Major limitations (detail in sections 7, 8 and 18)

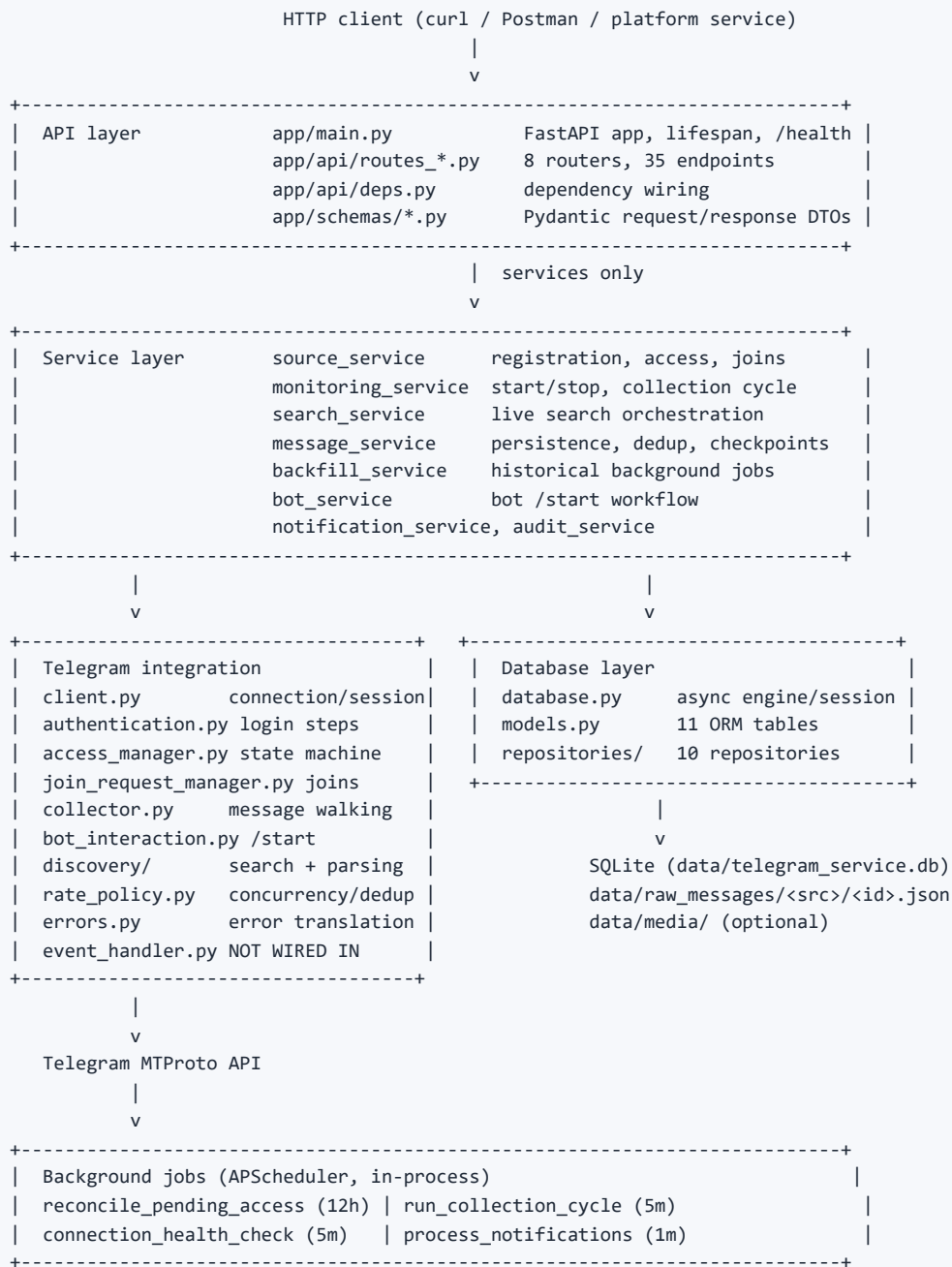
1. **The service has no authentication of its own.** Every one of its 35 endpoints is unauthenticated, and the default bind address is `0.0.0.0` . See finding **SEC-01** (High).
2. **"Global search" is not a search of all of Telegram.** `channels.searchPosts` is limited by a small free daily quota (then Telegram Stars, which this service will never spend); the fallback `messages.searchGlobal` is scoped to what Telegram's index associates with this account.
3. **Access is exactly this one account's access** — nothing more. Private sources stay unreadable until Telegram approves a join request.
4. **A silently declined join request is indistinguishable from a pending one** for username-based requests; Telegram exposes no decline signal to the requester.
5. **The event-driven access-change fast path described in the README is not wired into the running application** (`register_access_change_handlers` is never called). Only the 12-hour reconciliation job is live. See **DOC-01**.
6. **Backfill jobs do not survive a process restart** — they run as in-process `asyncio` tasks with no resumption on startup. See **ISS-02**.
7. **No Alembic migrations**; schema evolution relies on `create_all` plus a hand-maintained additive-column patch list for SQLite only.

2. System Architecture

2.1 Layer model

The codebase enforces a strict, one-directional dependency rule that was verified by inspection:

- **Routes never import Telethon and never build SQL.** They depend only on a service, injected through `app/api/deps.py`.
- **Services** own all business rules and are the only layer that combines Telegram operations with persistence.
- `app/telegram/*` **is the only package that imports Telethon.** `app/telegram/client.py` is the only module that constructs a `TelegramClient`.
- `app/database/repositories/*` **is the only code that builds queries.** Swapping `DATABASE_URL` from SQLite to PostgreSQL therefore requires no change above the repository layer (with the caveat in section 18 about the SQLite-specific `ON CONFLICT` insert).



2.2 Component reference

Component	Files	Responsibility
API layer	<code>app/main.py</code> , <code>app/api/routes_*.py</code>	Publishes 35 endpoints across 8 routers. Registers one global exception handler for <code>TelegramSearchError</code> . Owns the lifespan: DB init, first Telegram connect, scheduler start/stop, client disconnect.
Router/controller	8 modules: <code>routes_auth</code> , <code>routes_sources</code> , <code>routes_monitoring</code> , <code>routes_messages</code> , <code>routes_notifications</code> , <code>routes_search</code> , <code>routes_bots</code> , <code>routes_backfill</code>	Translate service exceptions into HTTP status codes (<code>404/409/422</code>), shape responses, declare OpenAPI metadata. No business logic.
Dependency wiring	<code>app/api/deps.py</code>	Builds each service with a request-scoped <code>AsyncSession</code> , the process-wide client manager, and <code>Settings</code> . Routes receive services, never sessions or Telethon objects.
Service / business logic	<code>app/services/*.py</code>	Access rules (<code>SEARCHABLE_STATUSES</code> , <code>BACKFILLABLE_STATUSES</code>), state-machine enforcement, de-duplication, audit writes, error translation, structured logging.
Telegram integration	<code>app/telegram/*</code>	<code>TelegramClientManager</code> (singleton connection + invalid-session latch), non-interactive login, access probing, join submission/polling, message normalization, search RPCs, URL classification, request concurrency policy, error mapping.
Database layer	<code>app/database/database.py</code> , <code>models.py</code> , <code>repositories/</code>	Async engine/sessionmaker, <code>create_all</code> bootstrap plus a SQLite-only additive-column patcher, 11 ORM models, 10 repositories.
Models / entities	<code>app/database/models.py</code>	11 tables and 7 enums (<code>SourceType</code> , <code>SourceAccessStatus</code> , <code>AccessRequestStatus</code> , <code>MediaType</code> , <code>ProcessingStatus</code> , <code>CollectionMethod</code> , <code>BackfillJobStatus</code>).
Schemas / DTOs	<code>app/schemas/*.py</code>	Pydantic v2 models. Search DTOs are hand-mapped from Telegram-layer dataclasses in <code>routes_search.py</code> so ORM models are never serialized directly.
Configuration	<code>app/config.py</code> , <code>.env</code>	<code>pydantic-settings</code> <code>Settings</code> , cached with <code>lru_cache</code> . No secret has a hard-coded default.
Authentication / session	<code>app/telegram/client.py</code> , <code>authentication.py</code>	One Telethon session file per service instance. A latched <code>session_invalid</code> flag short-circuits every Telegram-requiring operation until re-authentication succeeds. There is no application-level (caller) authentication.
Background jobs	<code>app/scheduler/jobs.py</code>	Four APScheduler interval jobs, all <code>max_instances=1</code> , <code>coalesce=True</code> , each opening its own DB session.
Logging / auditing	<code>app/observability.py</code> , <code>app/services/audit_service.py</code>	<code>log_structured</code> emits single-line JSON events. <code>AuditService.log</code> writes an <code>audit_logs</code> row for ~30 declared event types.
Error handling	<code>app/telegram/errors.py</code>	<code>translate_telegram_error</code> maps Telethon exceptions to a stable <code>{code, message, retryable, retry_after_seconds}</code> payload with an appropriate HTTP status.

Component	Files	Responsibility
Notifications	<code>app/services/notification_service.py</code> , <code>app/notifications/manager.py</code>	Notifications are always persisted first; outbound dispatch is a pluggable registry with no provider registered by default .

2.3 Runtime shape

- **One process.** Uvicorn serves the API, the APScheduler jobs run on the same event loop, and a single Telethon MTProto connection is shared process-wide (`get_client_manager()` singleton).
- **One Telegram account.** There is no multi-account pool, rotation, or proxy support anywhere in the codebase.
- **Backfill runs as** `asyncio.create_task` inside the same process — there is no external worker or queue.

Architectural consequence: because the Telethon client, the API and the scheduler share one process and one account, API load and background collection compete for the same Telegram rate budget. The only in-code guard is a semaphore of 3 concurrent *search* RPCs (`rate_policy.SEARCH_CONCURRENCY_LIMIT`); collection, access checks and joins are not covered by it.

3. Telegram API Integration

3.1 Two APIs, clearly separated

Throughout this document:

- **Application API** — the 35 REST endpoints this service publishes (`/api/...`). Documented in sections 4 and 5.
- **Telegram API** — Telegram's MTPROTO methods that the service calls on the caller's behalf, through Telethon.

The two are connected in exactly one place: a **service** method calls a function in `app/telegram/*`, which issues the MTPROTO request using the shared authenticated client. No route ever reaches Telegram directly.

Application API	Service	Telegram layer	Telegram API (MTPROTO)
POST <code>/api/sources</code>	SourceService .register_source	access_manager.check_access	contacts.resolveUsername + messages.getHistory(1)
POST <code>/api/sources/{id}/request-access</code>	SourceService .request_access	join_request_manager .submit_join_request	channels.joinChannel OR messages.importChatInvite
POST <code>/api/telegram/invites/join</code>	SourceService .join_by_invite	join_request_manager	messages.checkChatInvite + messages.importChatInvite
GET <code>/api/telegram/search/global</code>	SearchService .global_search	discovery.global_search	channels.checkSearchPostsFlood -> channels.searchPosts -> messages.searchGlobal
GET <code>/api/sources/{id}/search</code>	SearchService .channel_search_by_source	discovery.channel_search	messages.search
GET <code>/api/telegram/channels/search</code>	SearchService .discover_channels	discovery.channel_discovery	contacts.search
POST <code>/api/telegram/bots/{id}/start</code>	BotService .start_bot	bot_interaction.start_bot	messages.sendMessage + messages.getHistory
POST <code>/api/sources/{id}/backfill</code>	BackfillService .start_backfill	collector .collect_historical_messages	messages.getHistory (backward walk)
(scheduled)	MonitoringService .run_collection_for_source	collector .collect_new_messages	messages.getHistory (forward, min_id)
GET <code>/api/messages</code>	MessageService	-- none --	NONE (local SQLite only)

3.2 Telegram API type and client

Aspect	Implementation
API type	MTPROTO user API . The HTTP Bot API is not used anywhere.
Library	Telethon (<code>telethon>=1.36.0</code>)
Client construction	Only in <code>TelegramClientManager._build_client()</code> ; raises <code>TelegramNotConfiguredError</code> if <code>TELEGRAM_API_ID</code> / <code>TELEGRAM_API_HASH</code> are absent.
Instance scope	Process-wide singleton via <code>get_client_manager()</code> ; guarded by an <code>asyncio.Lock</code> so concurrent first-time connects cannot race.
Session storage	A Telethon SQLite session file at <code>TELEGRAM_SESSION_PATH</code> (default <code>data/telegram_service.session</code>). Parent directory is created automatically.

3.3 Authentication mechanism

The service authenticates **as a human Telegram user account**, not as a bot. `api_id` / `api_hash` identify the *application*; the phone-number login identifies the *account* whose permissions all collection then inherits.

Login is split into explicit HTTP steps because a long-running server cannot block on `input()` the way a CLI can:

```
POST /api/telegram/auth/send-code      -> auth.sendCode      (stores phone_code_hash in memory only)
POST /api/telegram/auth/verify-code   -> auth.signIn        (may report requires_password=true)
POST /api/telegram/auth/verify-password -> auth.checkPassword (only when 2FA is enabled)
GET  /api/telegram/status              -> confirms + returns masked identity
```

`pending_phone` and `pending_phone_code_hash` live **only in memory** on the client manager and are cleared on success. They are never persisted or logged. After a successful sign-in the Telethon session file is authorized and every restart reconnects with no further login.

3.4 Session management and the invalid-session latch

This is one of the more deliberate parts of the design and matters for account safety.

- `verify_authorized()` is a mandatory pre-flight before **every** Telegram-requiring operation (entity resolution, access checks, search, monitoring, backfill, bot start). It connects if needed and confirms `is_user_authorized()`.
- When any code path observes an error in `SESSION_INVALID_ERRORS` (`AuthKeyUnregisteredError` , `AuthKeyInvalidError` , `AuthKeyPermEmptyError` , `SessionExpiredError` , `SessionRevokedError` , `UserDeactivatedError` , `UserDeactivatedBanError` , `ActiveUserRequiredError`), the manager **latches** `session_invalid = True`.
- While latched, every Telegram-requiring operation short-circuits **without issuing another RPC**, returning `TELEGRAM_SESSION_INVALID` / HTTP 401. This is what stops the service from hammering Telegram with a key Telegram has already rejected.
- Only a successful `verify-code` or `verify-password` clears the latch.
- `SessionPasswordNeededError` is deliberately **excluded** from that set — it is a normal step of sign-in, not an invalidated session.

3.5 Connection lifecycle

Phase	Behaviour
Startup (<code>lifespan</code>)	<code>init_db()</code> -> connect if configured (a failure is logged as a warning, never fatal) -> <code>setup_scheduler().start()</code> .
Steady state	<code>connection_health_check</code> runs every <code>CONNECTION_HEALTH_INTERVAL_MINUTES</code> (default 5) and calls <code>ensure_connected()</code> — one attempt per run, never an internal retry loop. It also upserts the single <code>telegram_accounts</code> row.
Shutdown	<code>scheduler.shutdown(wait=False)</code> then <code>client_manager.disconnect()</code> .

3.6 How responses are processed

1. **Messages** are converted by `collector.normalize_message()` into a `NormalizedMessage` dataclass: id, sender id/username/display name, dates, text, views/forwards/reply count, `grouped_id`, media classification, permalink (`https://t.me/<username>/<id>`, `None` without a username), plus the full `message.to_dict()` payload and its SHA-256 hash. **Fields Telegram does not expose are left `None`, never inferred.**
2. **Search results** are converted into `TelegramSearchResultItem` dataclasses, then hand-mapped to Pydantic DTOs in `routes_search.py`. Each result carries `visibility` (`public` / `account_scoped` / `member`) and `collection_method`, so a consumer can tell *how* the service saw it.
3. **Entities** resolved during an access check are cached in `telegram_entities` — explicitly a resolution cache, **never treated as proof of access.**
4. **Raw evidence** for every persisted message is written to `data/raw_messages/<source_id>/<telegram_message_id>.json` before the row is inserted.

3.7 How Telegram errors are handled

Two distinct strategies, chosen by whether the caller has a source record to fall back on:

(a) Domain-state mapping — `access_manager`, `join_request_manager` and `collector` catch Telethon errors and convert them into `SourceAccessStatus` values or an `error` string on an outcome dataclass. Nothing raises through to the route; the source's `status_reason` and the checkpoint's `error_count` record what happened.

(b) Stable API error translation — the search/discovery/backfill paths have no source state machine, so `errors.translate_telegram_error()` maps the exception to a `TelegramSearchError` carrying `code`, `message`, `http_status`, `retryable`, `retry_after_seconds`. `app/main.py` registers a global handler that renders it as JSON. The full mapping is in section 16.

Credential hygiene in errors: every error message is built from the exception *class name* plus Telegram's own public error text. `api_hash`, session contents, verification codes and 2FA passwords never appear in an error, a log line, or an API response — this was verified across `client.py`, `authentication.py`, `errors.py` and `observability.py`.

4. Endpoint Inventory

35 endpoints across 10 functional groups. This table was derived from the router source files and cross-checked against `tests/test_endpoint_coverage.py`, which fails the build if the set of endpoints the app publishes differs from the set the test exercises.

Authentication column: "None" is literal. There is no API key, bearer token, session cookie, CORS policy or IP allow-list anywhere in the codebase — a grep for `api_key`, `HTTPBearer`, `OAuth2`, `jwt`, `middleware` and `CORS` across `app/` returns nothing. The "Telegram session" entries below mean the endpoint *requires the service's own Telegram account to be authorized*, not that the caller is authenticated. See finding SEC-01.

4.1 Health / Status

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/health</code>	Liveness probe	None	—	<code>{"status": "ok"}</code>	Implemented
GET	<code>/ready</code>	Readiness; reports whether Telegram credentials are configured	None	—	<code>{"status", "telegram_configured"}</code>	Implemented

4.2 Telegram Session / Authentication

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/api/telegram/status</code>	Connection + authorization state, masked account identity	None (caller)	—	<code>TelegramStatusResponse</code>	Implemented
POST	<code>/api/telegram/auth/send-code</code>	Step 1 of login: request an SMS/app code	None (caller)	<code>SendCodeRequest</code>	<code>AuthStepResponse</code>	Implemented
POST	<code>/api/telegram/auth/verify-code</code>	Step 2: sign in with the code	None (caller)	<code>VerifyCodeRequest</code>	<code>AuthStepResponse</code>	Implemented
POST	<code>/api/telegram/auth/verify-password</code>	Step 3: 2FA password (only if required)	None (caller)	<code>VerifyPasswordRequest</code>	<code>AuthStepResponse</code>	Implemented

4.3 Source Management

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
POST	<code>/api/sources</code>	Register a source; resolves and probes access immediately	None + Telegram session	<code>SourceCreate</code>	<code>SourceOut</code> (201)	Implemented
GET	<code>/api/sources</code>	List registered sources	None	—	<code>SourceOut[]</code>	Implemented
GET	<code>/api/sources/{source_id}</code>	Fetch one source	None	—	<code>SourceOut</code>	Implemented
PATCH	<code>/api/sources/{source_id}</code>	Update <code>monitoring_enabled</code> / <code>title</code>	None	<code>SourceUpdate</code>	<code>SourceOut</code>	Implemented
DELETE	<code>/api/sources/{source_id}</code>	Delete a source and cascade its data	None	—	204 no content	Implemented
POST	<code>/api/sources/{source_id}/request-access</code>	Submit exactly one join request	None + Telegram session	—	access-request object	Implemented
GET	<code>/api/sources/{source_id}/access-status</code>	Operational status + last live probe result	None	—	<code>AccessStatusOut</code>	Implemented
POST	<code>/api/sources/{source_id}/check-access</code>	Re-probe Telegram for current access	None + Telegram session	—	<code>SourceOut</code>	Implemented

4.4 Monitoring

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
POST	<code>/api/sources/{source_id}/monitoring/start</code>	Enter MONITORING and run an immediate collection	None + Telegram session	—	<code>MonitoringStatusOut</code>	Implemented
POST	<code>/api/sources/{source_id}/monitoring/stop</code>	Disable monitoring	None	—	<code>MonitoringStatusOut</code>	Implemented
GET	<code>/api/sources/{source_id}/monitoring-status</code>	Monitoring state and last collection markers	None	—	<code>MonitoringStatusOut</code>	Implemented

4.5 Message Retrieval (stored / local only — never calls Telegram)

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/api/sources/{source_id}/messages</code>	Stored messages for one source, filterable	None	—	<code>MessageOut []</code>	Implemented
GET	<code>/api/messages/{message_id}</code>	One stored message by internal id	None	—	<code>MessageOut</code>	Implemented
GET	<code>/api/messages</code>	Flat-filter search across all stored messages	None	—	<code>MessageOut []</code>	Implemented

4.6 Live Telegram Search (every call hits Telegram)

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/api/telegram/search/global</code>	Telegram-wide public post search	None + Telegram session	—	<code>TelegramGlobalSearchResponse</code>	Implemented (quota-limited)
POST	<code>/api/telegram/search/sources</code>	Live search across a chosen set of registered sources	None + Telegram session	<code>MultiSourceSearchRequest</code>	<code>MultiSourceSearchResponse</code>	Implemented (no cursor returned)
GET	<code>/api/sources/{source_id}/search</code>	Live search inside one registered source	None + Telegram session	—	<code>TelegramChannelSearchResponse</code>	Implemented
GET	<code>/api/telegram/channels/{channel_id}/search</code>	Live search inside an arbitrary channel (fresh probe each call)	None + Telegram session	—	<code>TelegramChannelSearchResponse</code>	Implemented
POST	<code>/api/telegram/search/results/save</code>	Promote one search result into a stored message	None + Telegram session	<code>SaveSearchResultRequest</code>	<code>SaveSearchResultResponse (201)</code>	Implemented

4.7 Telegram Source Discovery

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/api/telegram/channels/search</code>	Find candidate channels/groups by keyword	None + Telegram session	—	<code>TelegramChannelDiscoveryResponse</code>	Implemented (no pagination)
POST	<code>/api/telegram/channels/{telegram_id}/register</code>	Register a discovered channel as a source	None + Telegram session	— (query param only)	<code>SourceOut (201)</code>	Implemented
POST	<code>/api/telegram/discovery/extract-links</code>	Classify Telegram links found in free text (read-only)	None + Telegram session	<code>ExtractLinksRequest</code>	<code>ExtractLinksResponse</code>	Implemented

4.8 Invite / Join and Bot Operations

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
POST	<code>/api/telegram/bots/{source_id}/start</code>	Send one <code>/start</code> to a registered bot, read its reply, harvest Telegram links	None + Telegram session	—	<code>StartBotResponse</code>	Implemented
POST	<code>/api/telegram/invites/join</code>	Join or request to join via <code>t.me/+hash</code>	None + Telegram session	<code>JoinByInviteRequest</code>	<code>JoinByInviteResponse</code>	Implemented

4.9 Historical Backfill

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
POST	<code>/api/sources/{source_id}/backfill</code>	Queue a backward historical collection job	None + Telegram session	<code>BackfillRequest</code> (optional)	<code>BackfillJobResponse</code> (202)	Implemented
GET	<code>/api/sources/{source_id}/backfill</code>	List backfill jobs for a source (latest 20)	None	—	<code>BackfillJobResponse[]</code>	Implemented
GET	<code>/api/sources/{source_id}/backfill/{job_id}</code>	Poll one job's progress	None	—	<code>BackfillJobResponse</code>	Implemented

4.10 Notifications

Method	Endpoint	Purpose	Authentication	Request Body	Response	Status
GET	<code>/api/notifications</code>	List persisted notifications	None	—	<code>NotificationOutput[]</code>	Implemented
POST	<code>/api/notifications/{notification_id}/read</code>	Mark one notification read	None	—	<code>NotificationOutput</code>	Implemented

Categories deliberately absent. There is no endpoint group for sending messages, posting content, managing multiple Telegram accounts, exporting data, administering users, or configuring the service at runtime. Those are **Not Implemented**.

5. Detailed Endpoint Documentation

Conventions used in this section

- Base URL in all examples: `http://localhost:8000` (from `API_HOST=0.0.0.0`, `API_PORT=8000`).
- "Authentication" describes what the *implementation* requires. **No endpoint authenticates the caller.** Where an endpoint needs the service's Telegram account to be authorized, that is stated explicitly.
- Two error envelope shapes exist and are used consistently — `{"detail": ...}` for `HTTPException` /validation, and `{"error": {...}}` for `TelegramSearchError`. See section 16.
- Every request/response field below was read from the Pydantic schema or the route signature. No field has been invented.

5.1 GET /health

Purpose. Liveness probe. Returns a constant; touches nothing.

When to use. Container/orchestrator liveness checks and smoke tests.

Authentication. None. No Telegram session required.

Request

```
GET /health HTTP/1.1
Host: localhost:8000
```

Request Parameters. None.

Processing Flow

```
Client Request -> FastAPI route -> constant response
```

Response (200)

```
{ "status": "ok" }
```

Field	Type	Description
<code>status</code>	string	Always the literal <code>"ok"</code> .

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Always, while the process is serving.

Example

```
curl http://localhost:8000/health
```

5.2 GET /ready

Purpose. Readiness probe. Reports whether Telegram API credentials are configured — it does **not** report whether the session is authorized or connected.

When to use. Deployment gate; a quick way to confirm `.env` was loaded.

Authentication. None.

Request

```
GET /ready HTTP/1.1
Host: localhost:8000
```

Processing Flow

```
Client Request -> route -> get_client_manager().is_configured (reads Settings only, no RPC)
```

Response (200)

```
{ "status": "ready", "telegram_configured": true }
```

Field	Type	Description
<code>status</code>	string	"ready" when both <code>TELEGRAM_API_ID</code> and <code>TELEGRAM_API_HASH</code> are set, otherwise "not_configured".
<code>telegram_configured</code>	boolean	The same condition as a boolean.

`/ready` returning "ready" does **not** mean Telegram is reachable or the account is signed in. Use `GET /api/telegram/status` for that.

Example

```
curl http://localhost:8000/ready
```

5.3 GET /api/telegram/status

Purpose. The authoritative view of the service's Telegram connection: connected, authenticated, which account, and whether the session has been latched invalid.

When to use. Before driving any Telegram-dependent workflow; after a suspected session revocation; as an operations dashboard tile.

Authentication. None for the caller. Internally attempts a connect, and calls `get_me()` when authenticated.

Request

```
GET /api/telegram/status HTTP/1.1
Host: localhost:8000
```

Processing Flow

```
Client Request
  |
  v
route (routes_auth) -> authentication.get_status(client_manager)
  |
  +-- not configured -> {"error": "not configured"}
  |
  v
client_manager.connect() ----- failure -> {connected:false, error, session_invalid}
  |
  v
is_authenticated() -> client.get_me() -> mask phone -> response
```

Response (200)

```
{
  "connected": true,
  "authenticated": true,
  "account": { "id": "123456789", "username": "collector_account", "phone": "+1*****89" },
  "error": null,
  "session_invalid": false
}
```

Field	Type	Description
<code>connected</code>	boolean	The Telethon client currently holds a connection.
<code>authenticated</code>	boolean	<code>is_user_authorized()</code> returned true and the session is not latched invalid.
<code>account</code>	object or null	Present only when authenticated.
<code>account.id</code>	string or null	Telegram user id of the service's account.
<code>account.username</code>	string or null	Account username, if it has one.
<code>account.phone</code>	string or null	Masked — first two and last two digits only (<code>_mask_phone</code>).
<code>error</code>	string or null	Connection error, or the latched invalid-session reason.
<code>session_invalid</code>	boolean	True once a session-invalidating Telegram error has been observed. Only re-authentication clears it.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Always — failure states are reported in the body, not as an HTTP error.

This endpoint never returns `api_hash`, session data, or an unmasked phone number.

Example

```
curl http://localhost:8000/api/telegram/status
```

```
import httpx
s = httpx.get("http://localhost:8000/api/telegram/status").json()
if not s["authenticated"]:
    raise SystemExit(f"Telegram not ready: {s['error']}")
```

5.4 POST /api/telegram/auth/send-code

Purpose. Step 1 of the non-interactive login: asks Telegram to send a verification code to the account's phone.

When to use. First-time setup, or after `session_invalid` becomes true.

Authentication. None for the caller. Requires `TELEGRAM_API_ID` / `TELEGRAM_API_HASH` to be configured.

Request

```
POST /api/telegram/auth/send-code HTTP/1.1
Host: localhost:8000
Content-Type: application/json
```

Request Body

```
{ "phone": "+15551234567" }
```

Parameter	Type	Required	Description	Example
phone	string	Yes	Phone number of the Telegram account, in international format.	"+15551234567"

Processing Flow

```
Client Request -> route -> authentication.send_code()
|
v
client_manager.connect()
|
v
Telegram: auth.sendCode
|
+-- ApiIdInvalidError / PhoneNumberInvalidError / FloodWaitError / RPCError
|    -> {"ok": false, "error": "..."} (still HTTP 200)
v
store pending_phone + pending_phone_code_hash IN MEMORY -> {"ok": true}
```

Response (200)

```
{ "ok": true, "requires_password": false, "error": null }
```

Field	Type	Description
<code>ok</code>	boolean	The code request succeeded.
<code>requires_password</code>	boolean	Always <code>false</code> at this step.
<code>error</code>	string or null	Human-readable failure reason, e.g. <code>"invalid phone number"</code> , <code>"rate-limited by Telegram, wait 300s"</code> .

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Both success and handled failure — check <code>ok</code> .
422	Validation error	<code>phone</code> missing or not a string.

Error Responses

```
{ "ok": false, "requires_password": false, "error": "rate-limited by Telegram, wait 86400s" }
```

Example

```
curl -X POST http://localhost:8000/api/telegram/auth/send-code \  
-H "Content-Type: application/json" \  
-d '{"phone": "+15551234567"}'
```

This endpoint triggers a real Telegram login SMS and is **unauthenticated**. Anyone who can reach the port can initiate login attempts and cause Telegram to rate-limit the account. Do not expose this service to an untrusted network (SEC-01).

5.5 POST /api/telegram/auth/verify-code

Purpose. Step 2: completes sign-in with the code Telegram sent, or reports that a 2FA password is still needed.

When to use. Immediately after a successful `send-code` , in the same process lifetime (the `phone_code_hash` is in-memory only).

Authentication. None for the caller.

Request Body

```
{ "phone": "+15551234567", "code": "12345" }
```

Parameter	Type	Required	Description	Example
<code>phone</code>	string	Yes	Must exactly match the phone used in <code>send-code</code> .	<code>"15551234567"</code>
<code>code</code>	string	Yes	The verification code, as a string.	<code>"12345"</code>

Processing Flow

```
Client Request -> route -> authentication.verify_code()
|
v
guard: pending_phone_code_hash exists AND pending_phone == phone
| (else -> ok:false "no pending code request for this phone")
v
Telegram: auth.signIn(phone, code, phone_code_hash)
|
+-- SessionPasswordNeededError -> {ok:false, requires_password:true}
+-- PhoneCodeInvalidError / PhoneCodeExpiredError -> {ok:false, error:"<ClassName>"}
v
clear pending hash -> clear_session_invalid() -> {ok:true}
```

Response (200)

```
{ "ok": true, "requires_password": false, "error": null }
```

Field	Type	Description
ok	boolean	Sign-in completed; the session file is now authorized.
requires_password	boolean	true means 2FA is enabled — call verify-password next.
error	string or null	"2FA password required" , "PhoneCodeInvalidError" , "PhoneCodeExpiredError" , or a raw RPC description.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Success and all handled failures.
422	Validation error	Missing phone or code .

Example

```
curl -X POST http://localhost:8000/api/telegram/auth/verify-code \
-H "Content-Type: application/json" \
-d '{"phone": "+15551234567", "code": "12345"}'
```

5.6 POST /api/telegram/auth/verify-password

Purpose. Step 3: supplies the two-factor (cloud password) to finish sign-in.

When to use. Only when verify-code returned requires_password: true .

Authentication. None for the caller.

Request Body

```
{ "password": "<SECRET>" }
```

Parameter	Type	Required	Description	Example
password	string	Yes	The account's 2FA cloud password. Never logged or stored.	"<SECRET>"

Processing Flow

```
Client Request -> route -> authentication.verify_password()
|
v
Telegram: auth.checkPassword (via client.sign_in(password=...))
|
+-- RPCError -> {ok:false, error:"<ClassName>: <text>"}
v
clear pending phone + hash -> clear_session_invalid() -> {ok:true}
```

Response (200)

```
{ "ok": true, "requires_password": false, "error": null }
```

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Success and handled failure.
422	Validation error	password missing.

The 2FA password crosses the wire in a plain JSON body. The service terminates **HTTP, not HTTPS** — deploy it behind a TLS-terminating reverse proxy on a trusted network (SEC-03).

Example

```
curl -X POST http://localhost:8000/api/telegram/auth/verify-password \
-H "Content-Type: application/json" \
-d '{"password": "<SECRET>"}
```

5.7 POST /api/sources

Purpose. Registers a Telegram channel, group, bot or user as a tracked **source**, then immediately resolves it against Telegram and records its real access status.

When to use. The entry point for everything else. Any workflow that needs a `source_id` starts here.

Authentication. None for the caller. Telegram must be configured; if it is unreachable the source is still created and left at `DISCOVERED`.

Request

```
POST /api/sources HTTP/1.1
Host: localhost:8000
Content-Type: application/json
```

Request Body

```
{ "identifier": "@examplechannel", "monitoring_enabled": true }
```

Parameter	Type	Required	Description	Example
<code>identifier</code>	string	Yes	<code>@username</code> , bare username, <code>https://t.me/...</code> URL, invite link (<code>t.me/+hash</code> , <code>t.me/joinchat/hash</code>), or a numeric Telegram id.	<code>"@examplechannel"</code>
<code>source_type</code>	string	No	Accepted and ignored. The route never reads it; the real type comes from resolution.	<code>"channel"</code>
<code>monitoring_enabled</code>	boolean	No (default <code>true</code>)	Sets the flag on the row. It does not start monitoring.	<code>true</code>

Common misunderstanding: `monitoring_enabled: true` only sets a column. Collection requires `access_status == "MONITORING"`, which is only reachable through `POST /api/sources/{id}/monitoring/start`. A source registered with `monitoring_enabled: true` that is left at `PUBLIC_ACCESSIBLE` is **never collected** — the scheduler's `list_monitoring_enabled()` requires both conditions. This state is present in the live database today (source 7).

Processing Flow

```

Client Request
  |
  v
Route validation (Pydantic SourceCreate)
  |
  v
SourceService.register_source()
  |
  v
normalize_identifier()          string-level only, no network
  |
  v
Dedup pass 1: sources.get_by_identifier(normalized) -> 409 if hit
  |
  v
INSERT telegram_sources (access_status = DISCOVERED) + audit SOURCE_REGISTERED + COMMIT
  |
  v
check_access(source.id)
  |   verify_authorized() -> Telegram: contacts.resolveUsername / getChannels
  |                       -> Telegram: messages.getHistory(limit=1) [not for User entities]
  |                       -> or messages.checkChatInvite for invite hashes
  v
record last_probe_* unconditionally; apply state machine to access_status
  |
  v
Dedup pass 2: same telegram_entity_id already tracked?
  |   yes -> delete this row, detach audit, COMMIT, raise 409
  v
API Response 201 (SourceOut)

```

Response (201)

```

{
  "id": 1,
  "identifier": "@examplechannel",
  "telegram_entity_id": "1234567890",
  "username": "examplechannel",
  "title": "Example Channel",
  "source_type": "channel",
  "access_status": "PUBLIC_ACCESSIBLE",
  "status_reason": null,
  "monitoring_enabled": true,
  "monitoring_started_at": null,
  "last_status_check_at": "2026-09-06T09:15:02.113Z",
  "next_status_check_at": null,
  "last_message_id": null,
  "last_collected_at": null,
  "last_probe_status": "PUBLIC_ACCESSIBLE",
  "last_probe_reason": null,
  "last_probe_at": "2026-09-06T09:15:02.113Z",
  "created_at": "2026-09-06T09:15:01.880Z",
  "updated_at": "2026-09-06T09:15:02.115Z"
}

```

Response Fields

Field	Type	Description
<code>id</code>	integer	Internal source id used by every other endpoint.
<code>identifier</code>	string	The normalized identifier (<code>@name</code> lower-cased, <code>https://t.me/+hash</code> , or the numeric id).
<code>telegram_entity_id</code>	string or null	Telegram's own id, filled in once resolution succeeds.
<code>username</code> / <code>title</code>	string or null	From the resolved entity; <code>null</code> if not exposed.
<code>source_type</code>	string	<code>channel</code> , <code>supergroup</code> , <code>group</code> , <code>user</code> , <code>bot</code> , <code>unknown</code> .
<code>access_status</code>	string	One of the 13 state-machine states (section 6.1).
<code>status_reason</code>	string or null	Why the status is what it is, e.g. <code>"membership required to read this source"</code> .
<code>monitoring_enabled</code>	boolean	Flag only — see the callout above.
<code>monitoring_started_at</code>	datetime or null	Set when the source first enters MONITORING.
<code>last_status_check_at</code> / <code>next_status_check_at</code>	datetime or null	Operational status-check markers. <code>next_status_check_at</code> is never written by any code path.
<code>last_message_id</code>	integer or null	Highest Telegram message id collected so far.
<code>last_collected_at</code>	datetime or null	End of the last collection cycle.
<code>last_probe_status</code> / <code>last_probe_reason</code> / <code>last_probe_at</code>	string / string / datetime, nullable	The most recent live probe result , recorded unconditionally even when the state machine refuses to let it overwrite the operational <code>access_status</code> .
<code>created_at</code> / <code>updated_at</code>	datetime	Row timestamps.

HTTP Status Codes

Status	Meaning	When It Occurs
201	Created	Source registered (whatever access status resulted).
409	Conflict	Identifier already registered, or the resolved entity is already tracked under another identifier.
422	Unprocessable	<code>identifier</code> is not a recognizable Telegram identifier, or body validation failed.

Error Responses

```
{ "detail": "source already registered as id=1" }
```

```
{ "detail": "source already registered as id=1 (identifier '@examplechannel', same Telegram entity 1234567890)" }
```

```
{ "detail": "'not a channel!' is not a recognizable Telegram identifier (@username, t.me link, or numeric ID)" }
```

Example Usage

```
curl -X POST http://localhost:8000/api/sources \
-H "Content-Type: application/json" \
-d '{"identifier": "@examplechannel", "monitoring_enabled": true}'
```

```
import httpx

r = httpx.post("http://localhost:8000/api/sources",
               json={"identifier": "@examplechannel", "monitoring_enabled": True})
if r.status_code == 409:
    print("already registered:", r.json()["detail"])
r.raise_for_status()
source = r.json()
print(source["id"], source["access_status"])
```

5.8 GET /api/sources

Purpose. Lists registered sources, ordered by id ascending.

When to use. Inventory views; finding a `source_id`.

Authentication. None. No Telegram call.

Request

```
GET /api/sources?limit=100&offset=0 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
<code>limit</code>	integer (query)	No (default 100)	Page size. Not bounded by the implementation.	<code>50</code>
<code>offset</code>	integer (query)	No (default 0)	Rows to skip.	<code>100</code>

Processing Flow

```
Client Request -> route -> SourceService.list_sources -> SourceRepository.list_all -> SELECT ... ORDER BY id
```

Response (200) — a JSON array of `SourceOut` objects (fields as in 5.7).

```
[ { "id": 1, "identifier": "@examplechannel", "access_status": "MONITORING", "...": "..." } ]
```

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Always, including an empty array.
422	Validation error	<code>limit</code> / <code>offset</code> not integers.

Example

```
curl "http://localhost:8000/api/sources?limit=50&offset=0"
```

5.9 GET /api/sources/{source_id}

Purpose. Fetches one source by internal id.

When to use. Polling a source's `access_status` after registration or a join request.

Authentication. None. No Telegram call — this reads the stored row only.

Request

```
GET /api/sources/1 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
<code>source_id</code>	integer (path)	Yes	Internal source id.	<code>1</code>

Processing Flow

```
Client Request -> route -> SourceService.get_source -> session.get(TelegramSource, id) -> 404 if None
```

Response (200) — `SourceOut`, identical shape to 5.7.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Source exists.
404	Not Found	No such source id.

Error Responses

```
{ "detail": "source not found" }
```

Example

```
curl http://localhost:8000/api/sources/1
```

5.10 PATCH /api/sources/{source_id}

Purpose. Updates a source's `monitoring_enabled` flag and/or `title`.

When to use. Renaming a source for readability. **Not** the correct way to start or stop monitoring.

Authentication. None. No Telegram call.

Request Body

```
{ "title": "Example Channel (priority)", "monitoring_enabled": false }
```

Parameter	Type	Required	Description	Example
<code>source_id</code>	integer (path)	Yes	Internal source id.	<code>1</code>
<code>monitoring_enabled</code>	boolean	No	Written straight to the column.	<code>false</code>
<code>title</code>	string	No	Free-text label; overwritten again on the next successful access check.	<code>"Example Channel"</code>

Confirmed defect ISS-01 — this endpoint does not persist anything. `SourceService.update_source` calls `SourceRepository.update`, which `flush()` es but never `commit()` s, and no caller commits afterwards. When FastAPI closes the request-scoped session from `get_db`, the open transaction is rolled back. The endpoint returns **200 with the modified object**, but re-reading the source afterwards shows the old values. Verified by reproducing the exact dependency/repository pattern against SQLAlchemy 2.0.52 + aiolsqlite. Do not rely on this endpoint until it is fixed (one-line fix: `commit in update_source`).

Second issue in the same method (ISS-04): `update_source` writes fields directly through the repository, bypassing `_apply_status` — the choke point that normally enforces "monitoring may not be enabled from a blocked state". Once the missing commit is fixed, setting `monitoring_enabled: true` here on, say, an `ACCESS_DENIED` source would persist an inconsistent row. Collection would still not run (the job requires `monitoring_enabled = true AND access_status = 'MONITORING'`), but the invariant would be violated. Use `monitoring/start` and `monitoring/stop` instead.

Processing Flow

```
Client Request -> Pydantic SourceUpdate (exclude_unset) -> SourceService.update_source
-> drops None values -> SourceRepository.update (flush only, NO COMMIT, no state machine)
-> response serialized from the in-memory object
-> get_db closes the session -> TRANSACTION ROLLED BACK -> change lost
```

Response (200) — `SourceOut`.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Updated.
404	Not Found	No such source.
422	Validation error	Wrong field types.

Example

```
curl -X PATCH http://localhost:8000/api/sources/1 \
-H "Content-Type: application/json" \
-d '{"title": "Example Channel (priority)"}'
```

5.11 DELETE /api/sources/{source_id}

Purpose. Permanently removes a source and, by ORM cascade, its messages, media, access requests and checkpoint.

When to use. Retiring a source. **Destructive and irreversible.**

Authentication. None. No Telegram call.

Request

```
DELETE /api/sources/1 HTTP/1.1
Host: localhost:8000
```

Processing Flow

```
Client Request -> route -> SourceService.delete_source
-> _require_source (404 if missing)
-> session.delete(source) ORM cascade: messages -> media, access_requests, checkpoint
-> COMMIT
```

Response (204) — empty body.

HTTP Status Codes

Status	Meaning	When It Occurs
204	No Content	Deleted.
404	Not Found	No such source.

Unauthenticated and destructive. Deleting source `1` in the current database would remove 4,400 collected messages. Raw evidence JSON under `data/raw_messages/<source_id>/` is **not** deleted — it is orphaned on disk. Audit rows survive with `source_id` left dangling (SQLite foreign keys are not enabled; only `audit.detach_source` during duplicate-rejection clears them).

Example

```
curl -X DELETE http://localhost:8000/api/sources/1 -i
```

5.12 POST /api/sources/{source_id}/request-access

Purpose. Submits **exactly one** legitimate Telegram join request for a private source, and records it for the 12-hour reconciliation job to follow up.

When to use. When `access_status` is `JOIN_REQUEST_REQUIRED` (or `ERROR`, to retry after a transient failure).

Authentication. None for the caller. **Requires an authorized Telegram session.**

Request

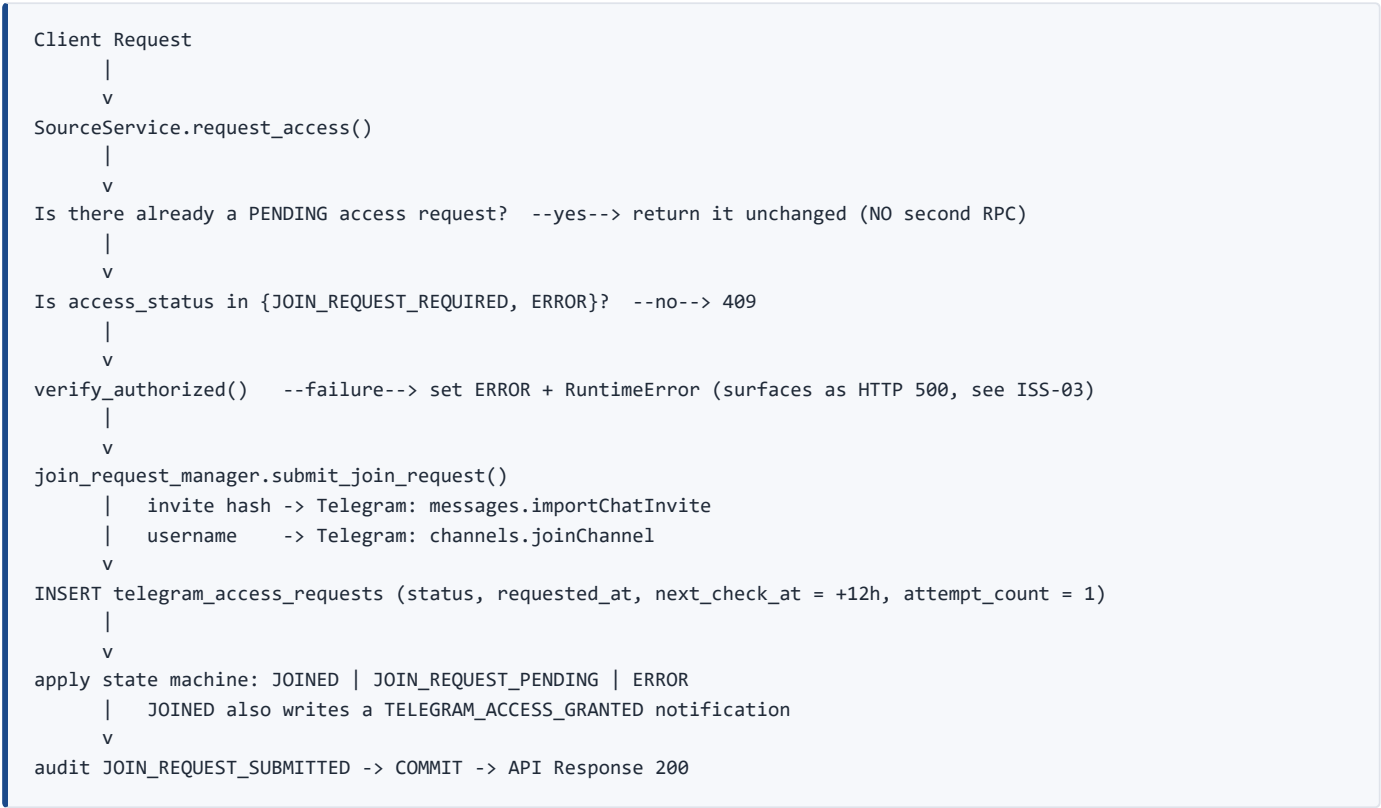
```
POST /api/sources/2/request-access HTTP/1.1
Host: localhost:8000
```

Request Parameters

Parameter	Type	Required	Description	Example
<code>source_id</code>	integer (path)	Yes	Internal source id.	<code>2</code>

Request Body. None.

Processing Flow



Response (200)

```
{
  "id": 1,
  "source_id": 2,
  "status": "PENDING",
  "requested_at": "2026-09-06T09:20:00.000Z",
  "next_check_at": "2026-09-06T21:20:00.000Z",
  "attempt_count": 1
}
```

Field	Type	Description
id	integer	Access-request row id.
source_id	integer	The source this request belongs to.
status	string	PENDING , APPROVED (joined immediately), or ERROR .
requested_at	datetime	When the request was submitted.
next_check_at	datetime or null	When reconciliation will re-check (submission time + 12h).
attempt_count	integer	Number of reconciliation attempts; 1 at submission.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Request submitted, or an existing PENDING request returned unchanged.
404	Not Found	No such source.
409	Conflict	Source is not in <code>JOIN_REQUEST_REQUIRED</code> or <code>ERROR</code> (e.g. a public channel that needs no join).
500	Internal Server Error	Telegram session unusable — <code>RuntimeError</code> is not mapped by the route (finding ISS-03).

Error Responses

```
{ "detail": "source 1 is in state PUBLIC_ACCESSIBLE; access can only be requested from JOIN_REQUEST_REQUIRED" }
```

Guarantee enforced in code: while a PENDING request exists, this endpoint is a no-op that returns the existing row — no duplicate join is ever sent to Telegram, however many times it is called. After a rejection, no alternative access mechanism is attempted.

Example

```
curl -X POST http://localhost:8000/api/sources/2/request-access
```

5.13 GET /api/sources/{source_id}/access-status

Purpose. Returns the source's **operational** status and, separately, the result of the most recent **live probe**. The two are deliberately not merged.

When to use. Diagnosing "why is this source not collecting?"; distinguishing "we are monitoring it" from "the last probe said X".

Authentication. None. No Telegram call — reads stored fields only.

Request

```
GET /api/sources/1/access-status HTTP/1.1
Host: localhost:8000
```

Processing Flow

```
Client Request -> route -> SourceService.get_access_status (row read) -> map to AccessStatusOut
```

Response (200)

```
{
  "source_id": 1,
  "access_status": "MONITORING",
  "status_reason": null,
  "last_status_check_at": "2026-09-06T09:15:02.113Z",
  "next_status_check_at": null,
  "last_probe_status": "PUBLIC_ACCESSIBLE",
  "last_probe_reason": null,
  "last_probe_at": "2026-09-06T09:15:02.113Z"
}
```

Field	Type	Description
<code>source_id</code>	integer	Source id.
<code>access_status</code>	string	The operational state (e.g. <code>MONITORING</code>).
<code>status_reason</code>	string or null	Explanation attached to the current state.
<code>last_status_check_at</code>	datetime or null	When <code>check_access</code> last ran.
<code>next_status_check_at</code>	datetime or null	Always <code>null</code> — no code path writes this column.
<code>last_probe_status</code>	string or null	What the last live probe actually found. May legitimately differ from <code>access_status</code> (a probe finding <code>PUBLIC_ACCESSIBLE</code> must not downgrade an operational <code>MONITORING</code>).
<code>last_probe_reason</code>	string or null	Probe explanation.
<code>last_probe_at</code>	datetime or null	Probe timestamp.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Source exists.
404	Not Found	No such source.

Example

```
curl http://localhost:8000/api/sources/1/access-status
```

5.14 POST /api/sources/{source_id}/check-access

Purpose. Forces a fresh live access probe against Telegram and updates the stored source record.

When to use. After a join request may have been approved; after an `ERROR` ; when access is suspected to have been revoked.

Authentication. None for the caller. Requires an authorized Telegram session (a failure is recorded as `ERROR` , not raised).

Request

```
POST /api/sources/1/check-access HTTP/1.1
Host: localhost:8000
```

Processing Flow



Response (200) — `SourceOut` (see 5.7).

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Probe ran; the outcome is in the body (including failures recorded as <code>ERROR</code>).
404	Not Found	No such source.

Access outcomes produced by the probe

Probe result	Meaning
<code>PUBLIC_ACCESSIBLE</code>	Public channel/group readable without membership.
<code>ACCESSIBLE</code>	Already a member (<code>left == False</code>), or a User entity, or <code>ChatInviteAlready</code> .
<code>JOIN_REQUEST_REQUIRED</code>	Private but has a username, or an invite link that needs joining/approval.
<code>ACCESS_DENIED</code>	Private with no invite, admin rights required, or an expired invite link.
<code>NOT_FOUND</code>	Username unoccupied/invalid, channel invalid, or invalid invite hash.
<code>ERROR</code>	FloodWait, other RPC failure, or an invalid Telegram session (<code>session_invalid</code> also latches).

Example

```
curl -X POST http://localhost:8000/api/sources/1/check-access
```

5.15 POST /api/sources/{source_id}/monitoring/start

Purpose. Moves a verified-accessible source into `MONITORING`, enables the flag, and **runs one collection cycle immediately** rather than waiting for the scheduler.

When to use. Once `access_status` is `PUBLIC_ACCESSIBLE`, `ACCESSIBLE`, `JOINED` or already `MONITORING`.

Authentication. None for the caller. **Requires an authorized Telegram session** — checked before anything is written.

Request

```
POST /api/sources/1/monitoring/start HTTP/1.1
Host: localhost:8000
```

Processing Flow

```
Client Request
  |
  v
MonitoringService.start_monitoring()
  |
  v
verify_authorized() --fail--> MonitoringStateError -> 409
  |
  v
access_status in {PUBLIC_ACCESSIBLE, ACCESSIBLE, JOINED, MONITORING}? --no--> 409
  |
  v
transition -> MONITORING; set monitoring_enabled = true, monitoring_started_at = now
  |
  v
audit MONITORING_STARTED -> COMMIT
  |
  v
run_collection_for_source() [immediate first cycle]
  |   verify_authorized -> get_entity
  |   checkpoint = get_or_create(source)
  |   Telegram: iter_messages(min_id = checkpoint, reverse=True, limit = COLLECTION_BATCH_LIMIT)
  |   normalize -> write raw evidence JSON -> INSERT ... ON CONFLICT DO NOTHING
  |   advance checkpoint, one COMMIT per message
  v
API Response 200 (MonitoringStatusOut)
```

Response (200)

```
{
  "source_id": 1,
  "monitoring_enabled": true,
  "access_status": "MONITORING",
  "monitoring_started_at": "2026-09-06T09:30:00.000Z",
  "last_collected_at": "2026-09-06T09:30:04.512Z",
  "last_message_id": 72697
}
```

Field	Type	Description
source_id	integer	Source id.
monitoring_enabled	boolean	Now <code>true</code> .
access_status	string	<code>MONITORING</code> on success.
monitoring_started_at	datetime or null	Set on the first entry into MONITORING.
last_collected_at	datetime or null	End of the collection cycle this call just ran.
last_message_id	integer or null	Checkpoint after that cycle.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Monitoring started and the initial cycle completed (even if the cycle itself collected nothing).
404	Not Found	No such source.
409	Conflict	Source not in a verified-accessible state, or the Telegram session is unusable.

Error Responses

```
{ "detail": "source 2 is in state JOIN_REQUEST_PENDING; monitoring requires verified access first" }
```

```
{ "detail": "cannot start monitoring: The Telegram authorization session is no longer valid. Re-authentication is requi
```

This call is **synchronous through the first collection cycle**. For a large channel that can pull up to `COLLECTION_BATCH_LIMIT` (default 200) messages, download nothing (media off by default), and write 200 evidence files before responding. Expect a multi-second response; set a generous client timeout.

Example

```
curl -X POST http://localhost:8000/api/sources/1/monitoring/start
```

5.16 POST /api/sources/{source_id}/monitoring/stop

Purpose. Disables monitoring for a source. The scheduled collection job will skip it from the next cycle.

When to use. Pausing collection without deleting anything.

Authentication. None. No Telegram call.

Processing Flow

```
Client Request -> MonitoringService.stop_monitoring
-> monitoring_enabled = false (access_status is deliberately left as MONITORING)
-> audit MONITORING_STOPPED -> COMMIT
```

Stopping leaves `access_status = "MONITORING"` and only clears the flag. That is intentional and reversible: `monitoring/start` then simply re-enables the flag without a state transition. The collection job requires *both* conditions, so nothing is collected while stopped.

Response (200) — `MonitoringStatusOut` with `monitoring_enabled: false`.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Stopped.
404	Not Found	No such source.

Example

```
curl -X POST http://localhost:8000/api/sources/1/monitoring/stop
```

5.17 GET /api/sources/{source_id}/monitoring-status

Purpose. Reports monitoring state and collection progress markers for one source.

When to use. Dashboards; confirming that collection is advancing (`last_message_id` increasing).

Authentication. None. No Telegram call.

Response (200) — `MonitoringStatusOut`, fields as in 5.15.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Source exists.
404	Not Found	No such source.

Example

```
curl http://localhost:8000/api/sources/1/monitoring-status
```

5.18 GET /api/sources/{source_id}/messages

Purpose. Reads **locally stored** messages for one source, with flat filters. **Never calls Telegram.**

When to use. Reviewing what has actually been collected. For a live query against Telegram, use `GET /api/sources/{source_id}/search` (5.22).

Authentication. None. Database only.

Request

```
GET /api/sources/1/messages?keyword=invoice&limit=50&offset=0 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
source_id	integer (path)	Yes	Source id.	1
limit	integer (query)	No (default 50)	Page size. Unbounded in code (finding ISS-05).	100
offset	integer (query)	No (default 0)	Rows to skip.	50
keyword	string (query)	No	Case-insensitive substring match on text (SQL <code>ILIKE %kw%</code>). No boolean/ <code>AND</code> / <code>OR</code> syntax.	invoice
sender_username	string (query)	No	Exact match on the stored sender username.	poster
media_type	string (query)	No	Exact match: photo, video, document, audio, voice, sticker, other.	photo
date_from	datetime (query)	No	message_date >= . ISO-8601.	2026-09-01T00:00:00Z
date_to	datetime (query)	No	message_date <= .	2026-09-06T00:00:00Z

Processing Flow

```
Client Request -> route -> MessageService.list_for_source -> MessageRepository.list_for_source
-> SELECT ... WHERE source_id = ? [+ filters] ORDER BY telegram_message_id DESC LIMIT/OFFSET
```

Response (200) — array of MessageOut .

```
[
  {
    "id": 12805,
    "source_id": 1,
    "telegram_message_id": 72697,
    "sender_id": "1234567890",
    "sender_username": "poster",
    "sender_display_name": "Example Poster",
    "message_date": "2026-09-06T08:59:11Z",
    "edit_date": null,
    "text": "Example message body",
    "views": 1420,
    "forwards": 3,
    "reply_count": null,
    "media_type": "photo",
    "source_url": "https://t.me/examplechannel/72697",
    "collected_at": "2026-09-06T09:00:02Z",
    "raw_data_hash": "9f2c...e11a",
    "processing_status": "raw"
  }
]
```

Response Fields

Field	Type	Description
<code>id</code>	integer	Internal message row id (used by <code>GET /api/messages/{message_id}</code>).
<code>source_id</code>	integer	Owning source.
<code>telegram_message_id</code>	integer	Telegram's own message id within the channel.
<code>sender_id</code> / <code>sender_username</code> / <code>sender_display_name</code>	string, nullable	<code>null</code> when Telegram does not expose the sender (common for broadcast channels).
<code>message_date</code> / <code>edit_date</code>	datetime, nullable	UTC.
<code>text</code>	string or null	Message body; <code>null</code> for media-only messages.
<code>views</code> / <code>forwards</code> / <code>reply_count</code>	integer, nullable	<code>null</code> when not exposed.
<code>media_type</code>	string or null	Classified type; <code>null</code> when no media.
<code>source_url</code>	string or null	Permalink; <code>null</code> when the source has no username.
<code>collected_at</code>	datetime	When this service stored it.
<code>raw_data_hash</code>	string or null	SHA-256 of the raw Telegram payload (evidence integrity).
<code>processing_status</code>	string	Always <code>"raw"</code> today — <code>normalized</code> / <code>enriched</code> exist in the enum but nothing sets them.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Including an empty array for an unknown <code>source_id</code> (no 404 here).
422	Validation error	Bad datetime or integer format.

Example

```
curl "http://localhost:8000/api/sources/1/messages?keyword=invoice&media_type=photo&limit=20"
```

5.19 GET /api/messages/{message_id}

Purpose. Fetches one stored message by its internal row id.

Authentication. None. Database only.

Request Parameters

Parameter	Type	Required	Description	Example
<code>message_id</code>	integer (path)	Yes	Internal row id — not the Telegram message id.	<code>12805</code>

Processing Flow

```
Client Request -> MessageService.get_message -> session.get(TelegramMessage, id) -> 404 if None
```

Response (200) — a single `MessageOut` (fields as in 5.18).

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Row exists.
404	Not Found	<code>{"detail": "message not found"} .</code>

Example

```
curl http://localhost:8000/api/messages/12805
```

5.20 GET /api/messages

Purpose. Flat-filter search across **all** stored messages, independent of any Telegram search RPC.

When to use. Analysis over already-collected data; the cheapest and safest query in the service (no Telegram traffic, no rate-limit exposure).

Authentication. None. Database only.

Request

```
GET /api/messages?keyword=invoice&source_id=1&limit=50 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
<code>keyword</code>	string (query)	No	Case-insensitive substring on <code>text</code> .	<code>invoice</code>
<code>source_id</code>	integer (query)	No	Restrict to one source.	<code>1</code>
<code>sender_username</code>	string (query)	No	Exact match.	<code>poster</code>
<code>media_type</code>	string (query)	No	Exact match.	<code>document</code>
<code>date_from</code> / <code>date_to</code>	datetime (query)	No	Inclusive bounds on <code>message_date</code> .	<code>2026-09-01T00:00:00Z</code>
<code>limit</code>	integer (query)	No (default 50)	Page size, unbounded in code.	<code>100</code>
<code>offset</code>	integer (query)	No (default 0)	Rows to skip.	<code>0</code>

Processing Flow

```
Client Request -> MessageService.search -> MessageRepository.search
-> SELECT ... [filters] ORDER BY message_date DESC LIMIT/OFFSET
```

Response (200) — array of `MessageOut` .

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Including an empty array.
422	Validation error	Bad parameter types.

Not implemented: boolean/nested query syntax (`AND` / `OR` / `NOT` , phrase grouping), full-text indexing, relevance ranking, and result counts/total pages. Filters are flat and combined with SQL `AND` . Ordering is always `message_date DESC` .

Example

```
curl "http://localhost:8000/api/messages?keyword=invoice&date_from=2026-09-01T00:00:00Z&limit=20"
```

```
import httpx
rows = httpx.get("http://localhost:8000/api/messages",
                 params={"keyword": "invoice", "source_id": 1, "limit": 100}).json()
print(len(rows), "stored matches")
```

5.21 GET /api/telegram/search/global

Purpose. A **live, Telegram-wide** search for public posts. Prefers Telegram's genuine cross-membership post search (`channels.searchPosts`) and falls back to `messages.searchGlobal` when the free quota is exhausted. **Never reads the local database.**

When to use. Discovering content outside the sources you already track. For already-collected data use `GET /api/messages` — it costs nothing and carries no rate-limit risk.

Authentication. None for the caller. **Requires an authorized Telegram session** (`_require_session()` — 401 otherwise).

Request

```
GET /api/telegram/search/global?q=example&limit=20&sort=date_desc HTTP/1.1
Host: localhost:8000
```

Request Parameters

Parameter	Type	Required	Description	Example
<code>q</code>	string (query)	Yes	Search text, 1–256 characters (enforced by both FastAPI and the discovery layer).	<code>example</code>
<code>limit</code>	integer (query)	No (default 20)	1–100, enforced by FastAPI and re-clamped by <code>clamp_limit</code> .	<code>50</code>
<code>cursor</code>	string (query)	No	Opaque base64 cursor from a previous response's <code>next_cursor</code> . Not stable across app versions.	<code>eyJvZmZzZXQ...</code>
<code>from_date</code>	datetime (query)	No	Lower date bound. Native (<code>min_date</code>) on the <code>searchGlobal</code> path; applied locally on the <code>searchPosts</code> path.	<code>2026-09-01T00:00:00Z</code>
<code>to_date</code>	datetime (query)	No	Upper date bound; same caveat.	<code>2026-09-06T00:00:00Z</code>
<code>channel_user_name</code>	string (query)	No	Local post-filter on the returned page only.	<code>examplechannel1</code>
<code>channel_id</code>	string (query)	No	Local post-filter on the returned page only.	<code>1234567890</code>
<code>media_type</code>	string (query)	No	<code>photo</code> , <code>video</code> , <code>document</code> , <code>voice</code> , <code>audio</code> . Native filter on <code>searchGlobal</code> ; local on <code>searchPosts</code> .	<code>photo</code>
<code>sort</code>	string (query)	No (default <code>date_desc</code>)	Only <code>date_desc</code> or <code>relevance</code> . <code>relevance</code> performs no re-sorting — Telegram's own order is preserved.	<code>date_desc</code>
<code>include_raw</code>	boolean (query)	No (default <code>false</code>)	Include the full raw Telegram payload per result. Large.	<code>false</code>

Processing Flow

```

Client Request
  |
  v
Route validation (q length, limit range, sort pattern)
  |
  v
SearchService.global_search -> _require_session() -> 401 NOT_AUTHENTICATED if unusable
  |
  v
rate_policy.run_bounded_search(key)      max 3 concurrent search RPCs;
  |                                     identical concurrent calls share one execution
  v
Telegram: channels.checkSearchPostsFlood(q)
  |
  +-- free quota remains --> Telegram: channels.searchPosts      visibility = "public"
  |
  +-- quota exhausted -----> WARNING added, NO Stars are ever spent
  +-- PremiumAccountRequiredError / RPCError --> WARNING added
      |
      v
      Telegram: messages.searchGlobal(broadcasts_only=True)  visibility = "account_scoped"
  |
  v
Normalize messages -> local filters (keyword re-verification ONLY on the searchGlobal path)
  |
  v
Sort (date_desc) -> build next_cursor from the last message + next_rate
  |
  v
audit SEARCH_GLOBAL + structured log -> API Response 200

```

Response (200)

```

{
  "results": [
    {
      "message_id": "72697",
      "channel": { "id": "1234567890", "title": "Example Channel", "username": "examplechannel", "type": "channel" },
      "text": "Example matching message",
      "date": "2026-09-06T08:59:11Z",
      "sender": { "id": "9876543210", "username": "poster", "display_name": "Example Poster" },
      "media": { "media_type": "photo", "filename": null, "mime_type": "image/jpeg", "file_size": 84213 },
      "url": "https://t.me/examplechannel/72697",
      "source_type": "telegram",
      "visibility": "public",
      "collection_method": "global_search",
      "raw": null
    }
  ],
  "pagination": { "limit": 20, "next_cursor": "eyJvZmZzZXRFaWQiOjcyNjk3fQ==", "has_more": true },
  "method_used": "channels.searchPosts",
  "quota": { "remaining": 4, "total_daily": 5, "stars_required": 0 },
  "warnings": []
}

```

Response Fields

Field	Type	Description
<code>results[].message_id</code>	string	Telegram message id, as a string.
<code>results[].channel</code>	object	<code>id</code> , <code>title</code> , <code>username</code> , <code>type</code> (<code>channel</code> / <code>group</code> / <code>user</code> / <code>unknown</code>). Any may be <code>null</code> when Telegram did not return the entity.
<code>results[].text</code>	string or null	Message body.
<code>results[].date</code>	datetime or null	UTC.
<code>results[].sender</code>	object or null	<code>id</code> , <code>username</code> , <code>display_name</code> . Often absent for broadcast posts.
<code>results[].media</code>	object or null	<code>media_type</code> , <code>filename</code> , <code>mime_type</code> , <code>file_size</code> .
<code>results[].url</code>	string or null	Permalink; <code>null</code> when the channel has no username.
<code>results[].source_type</code>	string	Always <code>"telegram"</code> .
<code>results[].visibility</code>	string	<code>"public"</code> (searchPosts) or <code>"account_scoped"</code> (searchGlobal). The key honesty field.
<code>results[].collection_method</code>	string	Always <code>"global_search"</code> here.
<code>results[].raw</code>	object or null	Full Telegram payload when <code>include_raw=true</code> .
<code>pagination.limit</code>	integer	Echo of the requested limit.
<code>pagination.next_cursor</code>	string or null	Pass back as <code>cursor</code> .
<code>pagination.has_more</code>	boolean	True when Telegram returned a full page.
<code>method_used</code>	string	<code>channels.searchPosts</code> or <code>messages.searchGlobal</code> .
<code>quota</code>	object or null	<code>remaining</code> , <code>total_daily</code> , <code>stars_required</code> — present only when the flood check succeeded.
<code>warnings</code>	string[]	Human-readable caveats, e.g. quota exhaustion or the account-scoped disclaimer.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Search executed (possibly with warnings and zero results).
401	Unauthorized	<code>NOT_AUTHENTICATED</code> / <code>TELEGRAM_SESSION_INVALID</code> .
402	Payment Required	<code>PREMIUM_OR_QUOTA_REQUIRED</code> if Telegram raises it outside the handled fallback.
422	Unprocessable	<code>q</code> empty or over 256 chars, <code>limit</code> out of range, <code>sort</code> not matching the pattern, bad cursor.
429	Too Many Requests	<code>FLOOD_WAIT</code> with <code>retry_after_seconds</code> .
502 / 503	Bad Gateway / Unavailable	<code>TELEGRAM_RPC_ERROR</code> / <code>NETWORK_ERROR</code> .

Error Responses

```
{ "error": { "code": "FLOOD_WAIT", "message": "Telegram is rate-limiting this account - wait 42s before retrying.", "re
```

```
{ "error": { "code": "NOT_AUTHENTICATED", "message": "The Telegram authorization session is no longer valid. Re-authent
```

This is not "search all of Telegram." When `method_used` is `messages.searchGlobal` , results come from Telegram's index *for this account* — in practice mostly channels it has already joined — and a warning saying exactly that is returned in `warnings` . Only `channels.searchPosts` reaches channels the account has not joined, and Telegram caps that with a small free daily quota (then Telegram Stars, which this service will never spend automatically).

Filter caveat: `channel_username` , `channel_id` , and (on the `searchPosts` path) `media_type` / `from_date` / `to_date` are applied *after* Telegram returns a page. A request with `limit=20` plus a narrow filter can legitimately return far fewer than 20 results while `has_more` is still `true` . Page through with `cursor` rather than assuming an empty page means "no more matches".

Example Usage

```
curl -G "http://localhost:8000/api/telegram/search/global" \
  --data-urlencode "q=example" \
  --data-urlencode "limit=20" \
  --data-urlencode "sort=date_desc"
```

```
import httpx

def paged_global_search(q, pages=3):
    cursor, out = None, []
    with httpx.Client(base_url="http://localhost:8000", timeout=60) as c:
        for _ in range(pages):
            params = {"q": q, "limit": 50}
            if cursor:
                params["cursor"] = cursor
            r = c.get("/api/telegram/search/global", params=params)
            if r.status_code == 429:
                print("flood wait:", r.json()["error"]["retry_after_seconds"], "s")
                break
            r.raise_for_status()
            body = r.json()
            out += body["results"]
            for w in body["warnings"]:
                print("WARNING:", w)
            cursor = body["pagination"]["next_cursor"]
            if not body["pagination"]["has_more"] or not cursor:
                break
    return out
```

5.22 GET /api/sources/{source_id}/search

Purpose. **Live** Telegram search inside one already-registered source, trusting its stored access status instead of re-probing on every call.

When to use. Targeted keyword queries against a channel you already track and have verified access to.

Authentication. None for the caller. Requires an authorized Telegram session **and** a source whose stored `access_status` is one of `PUBLIC_ACCESSIBLE`, `ACCESSIBLE`, `JOINED`, `MONITORING`.

Request

```
GET /api/sources/1/search?q=invoice&limit=20 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
<code>source_id</code>	integer (path)	Yes	Registered source id.	<code>1</code>
<code>q</code>	string (query)	Yes	1–256 characters.	<code>invoice</code>
<code>limit</code>	integer (query)	No (default 20)	1–100.	<code>50</code>
<code>cursor</code>	string (query)	No	Opaque cursor (<code>offset_id</code>) from a previous response.	<code>eyJvZmZzZXRfawQiOjcyNjk3fQ==</code>
<code>sender_username</code>	string (query)	No	Applied natively by Telegram (<code>from_user</code>).	<code>poster</code>
<code>from_date</code>	datetime (query)	No	Walk stops when older messages are reached (local break).	<code>2026-09-01T00:00:00Z</code>
<code>to_date</code>	datetime (query)	No	Passed natively as <code>offset_date</code> .	<code>2026-09-06T00:00:00Z</code>
<code>media_type</code>	string (query)	No	Native Telegram filter (<code>photo</code> / <code>video</code> / <code>document</code> / <code>voice</code> / <code>audio</code>).	<code>document</code>
<code>include_raw</code>	boolean (query)	No	Include the raw payload per result.	<code>false</code>

Processing Flow

```

Client Request
  |
  v
SearchService.channel_search_by_source()
  |
  v
Load source (404 if missing) -> reject if source_type == "bot" (422 NOT_A_CHANNEL)
  |
  v
stored access_status in SEARCHABLE_STATUSES? --no--> 403/404 WITHOUT contacting Telegram
  |
  v
_require_session() -> Telegram: get_entity(normalized identifier)
  |
  v
Telegram: messages.search via iter_messages(entity, search=q, from_user, filter, offset_id, offset_date)
  |
  v
Normalize -> build next_cursor (offset_id of the last message)
  |
  v
audit SEARCH_CHANNEL + structured log -> API Response 200

```

Response (200)

```
{
  "results": [
    {
      "message_id": "72697",
      "channel": { "id": "1234567890", "title": "Example Channel", "username": "examplechannel", "type": "channel" },
      "text": "Example matching message",
      "date": "2026-09-06T08:59:11Z",
      "sender": { "id": "9876543210", "username": null, "display_name": null },
      "media": null,
      "url": "https://t.me/examplechannel/72697",
      "source_type": "telegram",
      "visibility": "public",
      "collection_method": "channel_search",
      "raw": null
    }
  ],
  "pagination": { "limit": 20, "next_cursor": null, "has_more": false },
  "warnings": []
}
```

Response Fields. Same result shape as 5.21, with two differences: `collection_method` is `"channel_search"` , and `visibility` is `"public"` when the channel has a username, otherwise `"member"` . `sender.username` and `sender.display_name` are always `null` on this path — the per-channel search does not resolve senders (only `sender.id` is returned).

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Search executed.
401	Unauthorized	Telegram session unusable.
403	Forbidden	Stored status blocks searching: <code>JOIN_REQUIRED</code> , <code>JOIN_PENDING</code> , <code>ACCESS_REJECTED</code> , <code>ACCESS_DENIED</code> .
404	Not Found	Source id unknown, or its status is <code>NOT_FOUND</code> .
422	Unprocessable	<code>q</code> / <code>limit</code> invalid, bad cursor, or the source is a bot (<code>NOT_A_CHANNEL</code>).
429	Too Many Requests	<code>FLOOD_WAIT</code> .

Error Responses

```
{ "error": { "code": "JOIN_PENDING", "message": "A join request for this source is still pending Telegram's approval.",
  "code": "NOT_A_CHANNEL", "message": "This Telegram entity is a bot, not a channel/group - bots have no mes
```

An inaccessible source is **refused with an explicit error, never answered with an empty result list**. That distinction is deliberate: "no matches" and "you cannot read this" must not look alike.

Example

```
curl -G "http://localhost:8000/api/sources/1/search" \
  --data-urlencode "q=invoice" --data-urlencode "media_type=document"
```

5.23 GET /api/telegram/channels/{channel_id}/search

Purpose. Live search inside an arbitrary channel that is **not** registered as a source. Performs a fresh access probe on every call, because there is no stored state to trust.

When to use. One-off exploration of a channel found via discovery, before deciding to register it.

Authentication. None for the caller. Requires an authorized Telegram session.

Request

```
GET /api/telegram/channels/examplechannel/search?q=invoice HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
channel_id	string (path)	Yes	Any identifier <code>normalize_identifier</code> accepts: <code>@username</code> , bare username, or numeric id.	examplechannel
q, limit, cursor, sender_username, from_date, to_date, media_type, include_raw			Identical to 5.22.	

Processing Flow

```
Client Request
|
v
SearchService.channel_search_by_identifier() -> _require_session()
|
v
normalize_identifier -> access_manager.check_access()      LIVE PROBE ON EVERY CALL
|      (Telegram: get_entity + messages.getHistory(1), or checkChatInvite)
|
+-- session_invalid -> latch + 401
+-- entity is a bot -> 422 NOT_A_CHANNEL
+-- status not in SEARCHABLE_STATUSES -> 403/404 with the probe's own reason
v
Telegram: messages.search on the resolved entity
|
v
audit SEARCH_CHANNEL (source_id = null) -> API Response 200
```

Response. Identical shape to 5.22.

HTTP Status Codes. Same as 5.22, except 404 additionally covers "no Telegram entity found for this identifier".

Cost: this endpoint spends *two* Telegram round trips before the search itself (resolve + read probe) and repeats them on every call. For a channel you query more than once, register it (`POST /api/sources`) and use `GET /api/sources/{id}/search` instead — that path skips the re-probe and is materially safer for the account's rate budget.

Example

```
curl -G "http://localhost:8000/api/telegram/channels/examplechannel/search" \
--data-urlencode "q=invoice" --data-urlencode "limit=10"
```

5.24 POST /api/telegram/search/sources

Purpose. Runs the same per-source live search across a chosen set of registered sources, reporting which ones were skipped and why instead of failing the whole call.

When to use. A keyword sweep across your tracked estate.

Authentication. None for the caller. Requires an authorized Telegram session.

Request Body

```
{
  "source_ids": [1, 3, 4],
  "q": "invoice",
  "limit": 20,
  "cursor": null,
  "sender_username": null,
  "from_date": null,
  "to_date": null,
  "media_type": null,
  "include_raw": false
}
```

Parameter	Type	Required	Description	Example
source_ids	integer[] or null	No	Sources to search. Omit or null to search every source that is currently MONITORING with the flag enabled.	[1, 3]
q	string	Yes	Search text. Length is validated in the discovery layer (1–256), not by the schema.	invoice
limit	integer	No (default 20)	Per-source page size; clamped to 100 downstream. No ge / le on the schema itself.	50
cursor	string or null	No	Passed to every per-source search. See the caveat below.	null
sender_username	string or null	No	Native Telegram filter.	poster
from_date / to_date	datetime or null	No	Date bounds.	null
media_type	string or null	No	Native Telegram filter.	photo
include_raw	boolean	No (default false)	Include raw payloads.	false

Processing Flow



Response (200)

```
{
  "results": [ { "message_id": "72697", "channel": { "...": "..." }, "collection_method": "channel_search", "...": "...",
    "searched_source_ids": [1, 3],
    "skipped": [ { "source_id": 4, "status": "JOIN_REQUEST_PENDING", "reason": "source is JOIN_REQUEST_PENDING, not curre",
    "warnings": []
  }
}
```

Field	Type	Description
results	array	Merged results from every searched source (same item shape as 5.21). Not re-sorted across sources — they appear grouped in source order.
searched_source_ids	integer[]	Sources actually queried.
skipped[].source_id	integer	Source not queried.
skipped[].status	string or null	Its stored access status, or <code>null</code> when the id did not exist.
skipped[].reason	string	Why it was skipped, including translated Telegram errors such as <code>"FLOOD_WAIT: ..."</code> .
warnings	string[]	Always empty on this path today.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Even when every source was skipped.
401	Unauthorized	Telegram session unusable (the only whole-call failure).
422	Validation error	Malformed body.

Pagination is not exposed here (Partially Implemented). The request accepts a `cursor` , but `MultiSourceSearchResponse` has no `pagination / next_cursor` field — so there is no way to page a multi-source sweep. Also note a single `cursor` is applied to *every* source, though cursors are per-channel `offset_id` values; passing one from a different channel's response will produce meaningless offsets. Leave `cursor` null and use per-source search (5.22) when you need to page.

Cost scales linearly. Omitting `source_ids` issues at least two Telegram calls (resolve + search) for *every* monitoring-enabled source, sequentially, in one HTTP request. With many sources this is a long request and a large burst against one account's rate budget. Prefer an explicit `source_ids` list.

Example

```
curl -X POST http://localhost:8000/api/telegram/search/sources \
-H "Content-Type: application/json" \
-d '{"q": "invoice", "source_ids": [1, 3], "limit": 25}'
```

5.25 POST /api/telegram/search/results/save

Purpose. Promotes one search result into stored data: registers the source if needed, **re-fetches the exact message live from Telegram**, and persists it through the same evidence-preserving path monitoring and backfill use.

When to use. This is the **only** way a search result is ever written to the database. Search itself is read-only and ephemeral.

Authentication. None for the caller. Requires an authorized Telegram session.

Request Body

```
{
  "identifier": "@examplechannel",
  "telegram_message_id": 72697,
  "collection_method": "manual_collection",
  "monitoring_enabled": false
}
```

Parameter	Type	Required	Description	Example
identifier	string	Yes	@username , t.me link, or numeric id of the source channel the result came from.	"@examplechannel"
telegram_message_id	integer	Yes	Telegram's message id from the search result (results[].message_id).	72697
collection_method	string	No (default manual_collection)	Recorded on the message row. Free-form string; not validated against the CollectionMethod enum.	"global_search"
monitoring_enabled	boolean	No (default false)	Passed to registration if the source is new. Does not start monitoring.	false

Processing Flow



Response (201)

```
{
  "source": { "id": 1, "identifier": "@examplechannel", "access_status": "PUBLIC_ACCESSIBLE", "...": "..." },
  "message": { "id": 12805, "source_id": 1, "telegram_message_id": 72697, "text": "Example matching message", "...": "." },
  "inserted": true
}
```

Field	Type	Description
source	object	The <code>SourceOut</code> for the (possibly newly registered) source.
message	object or null	The persisted <code>MessageOut</code> row.
inserted	boolean	<code>false</code> when the message was already stored (saved twice, or already collected by monitoring/backfill) — the call still succeeds with 201.

HTTP Status Codes

Status	Meaning	When It Occurs
201	Created	Saved, or already present (<code>inserted: false</code>).
401	Unauthorized	Telegram session unusable.
403	Forbidden	The source is not currently searchable/readable.
404	Not Found	Message id not found on that source.
409	Conflict	<code>{"error": { "code": "NOT_FOUND", ... } }</code> — the source exists but could not be located (rare edge case).
422	Unprocessable	<code>identifier</code> is not a valid Telegram identifier.

The message is **re-fetched from Telegram** rather than trusting the client's copy, so stored evidence always matches what Telegram actually served at save time. The client only resubmits the identifying pair (source + message id).

Side effect: if `identifier` is not yet registered, this endpoint **creates a source row** and performs a live access probe. It is not a pure "save" operation.

Example

```
curl -X POST http://localhost:8000/api/telegram/search/results/save \  
-H "Content-Type: application/json" \  
-d '{"identifier": "@examplechannel", "telegram_message_id": 72697, "collection_method": "global_search"}'
```

5.26 GET /api/telegram/channels/search

Purpose. Channel/group discovery — finds candidate channels matching a keyword via `contacts.search`. Returns **channels, not messages**.

When to use. Finding sources worth registering. Follow up with `POST /api/telegram/channels/{telegram_id}/register` or `POST /api/sources`.

Authentication. None for the caller. Requires an authorized Telegram session.

Request

```
GET /api/telegram/channels/search?q=example&limit=20 HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
q	string (query)	Yes	1–256 characters.	example
limit	integer (query)	No (default 20)	1–100.	20

Processing Flow

```
Client Request -> SearchService.discover_channels -> _require_session()
-> rate_policy.run_bounded_search
-> Telegram: contacts.search(q, limit)
-> keep Channel/Chat results only, de-duplicate by id
-> derive a COARSE access hint from entity flags (no extra RPC per result)
-> audit CHANNEL_DISCOVERY -> API Response 200
```

Response (200)

```
{
  "results": [
    {
      "telegram_id": "1234567890",
      "access_hash": "-7654321098765432109",
      "title": "Example Channel",
      "username": "examplechannel",
      "type": "channel",
      "public": true,
      "verified": false,
      "member_count": 15234,
      "access_status": "PUBLIC_LIKELY",
      "source_url": "https://t.me/examplechannel"
    }
  ],
  "pagination": { "limit": 20, "next_cursor": null, "has_more": false },
  "warnings": ["contacts.search has no native pagination - this is Telegram's complete response for this query."]
}
```

Field	Type	Description
<code>results[].telegram_id</code>	string	Telegram's channel id. Use it with the register endpoint.
<code>results[].access_hash</code>	string or null	Telegram access hash, as a string.
<code>results[].title</code> / <code>username</code>	string or null	As returned.
<code>results[].type</code>	string	<code>channel</code> (broadcast) or <code>group</code> (megagroup/legacy chat).
<code>results[].public</code>	boolean	True when a username exists.
<code>results[].verified</code>	boolean or null	Telegram's verified flag.
<code>results[].member_count</code>	integer or null	<code>participants_count</code> , when exposed.
<code>results[].access_status</code>	string	Coarse hint only: <code>ALREADY_MEMBER</code> , <code>PUBLIC_LIKELY</code> , or <code>ACCESS_UNKNOWN</code> . These are not <code>SourceAccessStatus</code> values.
<code>results[].source_url</code>	string or null	<code>https://t.me/<username></code> when public.
<code>pagination.next_cursor</code> / <code>has_more</code>	null / boolean	Always <code>null</code> / <code>false</code> — <code>contacts.search</code> has no pagination.
<code>warnings</code>	string[]	Always includes the no-pagination note.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Discovery executed.
401	Unauthorized	Telegram session unusable.
422	Unprocessable	<code>q</code> / <code>limit</code> invalid.
429	Too Many Requests	<code>FLOOD_WAIT</code> .

`access_status` here is a **guess derived from flags already present in the search response**, deliberately avoiding an extra RPC per result. It is not the verified state machine value. Always confirm with `POST /api/sources` + `POST /api/sources/{id}/check-access` before treating a discovered channel as accessible.

Example

```
curl -G "http://localhost:8000/api/telegram/channels/search" --data-urlencode "q=example"
```

5.27 POST /api/telegram/channels/{telegram_id}/register

Purpose. Registers a channel discovered through 5.26 as a source. It is a thin wrapper over the same registration path as `POST /api/sources`.

When to use. Immediately after discovery, when you have a numeric `telegram_id` rather than a username.

Authentication. None for the caller. Telegram session needed for the immediate access probe.

Request

```
POST /api/telegram/channels/1234567890/register?monitoring_enabled=true HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
telegram_id	string (path)	Yes	The telegram_id from a discovery result (or any identifier normalize_identifier accepts).	1234567890
monitoring_enabled	boolean (query)	No (default true)	Query parameter, not a body field. Sets the flag only; does not start monitoring.	false

Request Body. None.

Processing Flow

```
Client Request -> SourceService.register_source(telegram_id, monitoring_enabled)
-> identical path to POST /api/sources: normalize, dedup by identifier,
    insert DISCOVERED, check_access, dedup by telegram_entity_id
-> API Response 201 (SourceOut)
```

Response (201) — SourceOut (see 5.7).

HTTP Status Codes

Status	Meaning	When It Occurs
201	Created	Registered.
409	Conflict	Already registered under this or another identifier (e.g. previously added by @username).
422	Unprocessable	Identifier not recognizable.

A channel already registered by @username and then re-registered by numeric id is correctly rejected with **409** — the second de-duplication pass compares the resolved telegram_entity_id . That 409 is the endpoint's contract, not a failure.

Example

```
curl -X POST "http://localhost:8000/api/telegram/channels/1234567890/register?monitoring_enabled=false"
```

5.28 POST /api/telegram/discovery/extract-links

Purpose. Scans free text for Telegram links, classifies each one, and reports its **live access state** — a read-only preview. Nothing is registered, joined or monitored.

When to use. Triaging links found inside a collected message before deciding what to track.

Authentication. None for the caller. A Telegram session is needed to resolve each candidate; without one, candidates are returned with access_status: null and the session error in reason .

Request Body

```
{ "text": "join us at https://t.me/examplechannel and https://example.com/ignored" }
```

Parameter	Type	Required	Description	Example
text	string	Yes	Free text to scan. Only http(s):// URLs are matched.	see above

Processing Flow



Response (200)

```
{
  "candidates": [
    {
      "raw_url": "https://t.me/examplechannel",
      "identifier": "@examplechannel",
      "access_status": "PUBLIC_ACCESSIBLE",
      "title": "Example Channel",
      "username": "examplechannel",
      "already_registered_source_id": 1,
      "reason": null
    }
  ]
}
```

Field	Type	Description
<code>candidates[].raw_url</code>	string	The URL exactly as found in the text.
<code>candidates[].identifier</code>	string or null	Normalized identifier; <code>null</code> when the Telegram URL could not be parsed into one.
<code>candidates[].access_status</code>	string or null	A real <code>SourceAccessStatus</code> from a live probe; <code>null</code> when no probe ran.
<code>candidates[].title</code> / <code>username</code>	string or null	From resolution.
<code>candidates[].already_registered_source_id</code>	integer or null	Existing source id, if this is already tracked.
<code>candidates[].reason</code>	string or null	Probe explanation, parse failure, or session error.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Always, including an empty <code>candidates</code> array.
422	Validation error	<code>text</code> missing.

Hard rule enforced in `url_classifier.py` : non-Telegram URLs are classified from the string alone and **never fetched**. Only `t.me` and `telegram.me` are recognized as Telegram hosts — shorteners and unknown domains are discarded, never followed to "find out".

Cost: up to 20 live Telegram probes in a single HTTP request (each up to two RPCs). This endpoint is not covered by the search concurrency semaphore. Use it on individual messages, not in a loop over a corpus.

Example

```
curl -X POST http://localhost:8000/api/telegram/discovery/extract-links \
-H "Content-Type: application/json" \
-d '{"text": "see https://t.me/examplechannel and https://example.com/ignored"}'
```

5.29 POST /api/telegram/bots/{source_id}/start

Purpose. The API equivalent of pressing "Start Bot" in the Telegram app: sends exactly one `/start` message to a registered bot, waits (bounded) for its reply, and registers any Telegram links found in that reply as **dormant** sources.

When to use. When a bot is the gateway to a set of channels. Operator-initiated only — nothing schedules this.

Authentication. None for the caller. Requires an authorized Telegram session **and** a source whose `source_type` is exactly `bot`.

Request

```
POST /api/telegram/bots/5/start HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
source_id	integer (path)	Yes	Id of a registered source whose source_type == "bot" .	5

Request Body. None.

Processing Flow

```

Client Request
|
v
BotService.start_bot()
|
v
Load source -> 404 if missing -> 422 if source_type != "bot"
|
v
verify_authorized() -> Telegram: get_entity
|
v
audit TELEGRAM_BOT_START_REQUESTED -> COMMIT
|
v
bot_interaction.start_bot()
|   Telegram: messages.sendMessage("/start")    <-- the ONLY message this service ever sends
|   poll Telegram: iter_messages(limit=10) every BOT_START_POLL_INTERVAL_SECONDS
|   until a newer inbound message appears or BOT_START_TIMEOUT_SECONDS elapses
v
Scan reply text + inline button URLs for t.me/telegram.me links ONLY
|   non-Telegram URLs classified and discarded, NEVER fetched
|   callback buttons surfaced (text only) but NEVER executed
|   cap at MAX_LINKS_PER_BOT_RESPONSE = 10
v
FOR EACH link: register as DORMANT source (monitoring_enabled = false,
      discovery_type = "BOT_RESPONSE", discovered_from_source_id = bot)
      or refresh access if already registered
|
v
audit TELEGRAM_LINK_DISCOVERED / TELEGRAM_INVITE_RESOLVED -> API Response 200

```

Response (200)

```

{
  "source": { "id": 5, "identifier": "@examplebot", "source_type": "bot", "...": "..." },
  "status": "started",
  "response_text": "Welcome! Pick a channel: https://t.me/examplechannel",
  "response_message_id": 900,
  "response_date": "2026-09-06T09:45:03Z",
  "buttons": [ { "text": "Open channel", "url": "https://t.me/examplechannel" } ],
  "discovered": [
    {
      "source_id": 8,
      "identifier": "@examplechannel",
      "access_status": "PUBLIC_ACCESSIBLE",
      "source_type": "channel",
      "already_existed": false,
      "raw_url": "https://t.me/examplechannel"
    }
  ],
  "error": null
}

```

Field	Type	Description
<code>source</code>	object	The bot source (<code>SourceOut</code>).
<code>status</code>	string	<code>"started"</code> , <code>"timed_out"</code> , or <code>"failed"</code> .
<code>response_text</code>	string or null	The bot's reply body.
<code>response_message_id</code>	integer or null	Telegram id of the reply.
<code>response_date</code>	datetime or null	UTC timestamp of the reply.
<code>buttons[].text</code>	string	Inline button label.
<code>buttons[].url</code>	string or null	Button URL; <code>null</code> for callback buttons (surfaced but never executed).
<code>discovered[]</code>	array	Sources registered or refreshed from the reply's links.
<code>discovered[].already_existed</code>	boolean	True when the link pointed at a source already tracked.
<code>error</code>	string or null	Failure or timeout reason.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Including <code>status: "timed_out"</code> and <code>status: "failed"</code> — these are body states, not HTTP errors.
404	Not Found	No such source.
422	Unprocessable	The source exists but is not a bot.

Error Responses

```
{ "detail": "source 1 is a 'channel', not a bot - the bot-start workflow only applies to SourceType.BOT sources" }
```

```
{ "source": { "...": "..." }, "status": "timed_out", "error": "no response from bot within timeout", "buttons": [], "di
```

This is the only endpoint in the service that sends a message to Telegram. It is unauthenticated. The message is always the literal string `/start` and nothing else — no arbitrary text, no button clicks, no callback execution — but repeated calls do send repeated `/start` messages, with no cooldown or de-duplication. That is visible outbound activity on the account.

Discovered channels are registered **dormant**: monitoring disabled, never auto-joined. Acting on them stays an explicit operator decision via `POST /api/sources/{id}/request-access` and `.../monitoring/start` .

Example

```
curl -X POST http://localhost:8000/api/telegram/bots/5/start
```

5.30 POST /api/telegram/invites/join

Purpose. Joins — or submits an approval request for — a Telegram source using a raw invite link.

When to use. You hold a `t.me/+hash` or `t.me/joinchat/hash` link. For plain `@usernames`, use `POST /api/sources` followed by `.../request-access` instead.

Authentication. None for the caller. Requires an authorized Telegram session.

Request Body

```
{ "identifier": "https://t.me/+AbCdEfGhIjKlM" }
```

Parameter	Type	Required	Description	Example
identifier	string	Yes	Invite link (<code>t.me/+hash</code> , <code>t.me/joinchat/hash</code>) or the bare hash. A username is rejected with 422.	<code>"https://t.me/+AbCdEfGhIjKlM"</code>

Processing Flow

```
Client Request
  |
  v
SourceService.join_by_invite()
  |
  v
normalize_identifier -> must be INVITE_HASH, else 422
  |
  v
register_source(monitored_enabled = false) [Telegram: messages.checkChatInvite via check_access]
  |   already exists -> load it and re-check access
  v
Branch on resulting access_status:
  ACCESSIBLE/JOINED/PUBLIC_ACCESSIBLE/MONITORING -> "already_member" (no join RPC sent)
  JOIN_REQUEST_PENDING                          -> "already_pending" (no duplicate request)
  not JOIN_REQUEST_REQUIRED and not ERROR        -> "failed"
  |
  v
request_access() -> Telegram: messages.importChatInvite
  |
  v
final status -> "joined" | "pending_approval" | "failed"
  |
  v
audit TELEGRAM_JOIN_REQUESTED / _COMPLETED / _REQUEST_PENDING / _ALREADY_MEMBER / _FAILED
  |
  v
API Response 200
```

Response (200)

```
{
  "source": { "id": 9, "identifier": "https://t.me/+AbCdEfGhIjKlM", "access_status": "JOIN_REQUEST_PENDING", "...": ".." },
  "status": "pending_approval",
  "reason": "join request sent - awaiting admin approval"
}
```

Field	Type	Description
source	object	The registered source (<code>SourceOut</code>).
status	string	"joined" , "already_member" , "pending_approval" , "already_pending" , or "failed" .
reason	string or null	The source's <code>status_reason</code> , or the failure explanation.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	All join outcomes, including "failed" .
422	Unprocessable	Not an invite link, or an unparseable identifier.

Error Responses

```
{ "detail": "'@examplechannel' is not a Telegram invite link (t.me/+hash or t.me/joinchat/hash) - use POST /api/sources
```

Idempotent by design: an already-accessible source is reported as `already_member` with *no* RPC sent, and a pending request returns `already_pending` — a duplicate join is never submitted, however many times this is called.

Unauthenticated, and it changes the Telegram account's real-world membership. A caller who reaches this port can make the account join channels. Rapid repeated joins are a well-known trigger for Telegram account restrictions (see section 10).

Example

```
curl -X POST http://localhost:8000/api/telegram/invites/join \  
-H "Content-Type: application/json" \  
-d '{"identifier": "https://t.me/+AbCdEfGhIjKlM"}'
```

5.31 POST /api/sources/{source_id}/backfill

Purpose. Queues a **background** job that walks *backward* into a source's history, collecting older messages. Returns immediately with a job id.

When to use. After monitoring is established and you need history older than the point collection began.

Authentication. None for the caller. Requires a source with verified access; the Telegram session is validated inside the background job.

Request

```
POST /api/sources/1/backfill HTTP/1.1
Host: localhost:8000
Content-Type: application/json
```

Request Body (optional — omit entirely for defaults)

```
{ "from_date": "2026-01-01T00:00:00Z", "to_date": "2026-06-01T00:00:00Z", "limit": 1000 }
```

Parameter	Type	Required	Description	Example
<code>source_id</code>	integer (path)	Yes	Source id.	1
<code>from_date</code>	datetime or null	No	Stop boundary — the walk breaks when messages older than this are reached.	2026-01-01T00:00:00Z
<code>to_date</code>	datetime or null	No	Start boundary — used as <code>offset_date</code> on a fresh job. Ignored when resuming (the job's own checkpoint wins).	2026-06-01T00:00:00Z
<code>limit</code>	integer	No (default 1000)	Maximum messages, 1–10000 (schema-validated and re-clamped in the service).	2000

Processing Flow

```

Client Request
  |
  v
BackfillService.start_backfill()
  |
  v
Source exists? --no--> 404
access_status in {PUBLIC_ACCESSIBLE, ACCESSIBLE, JOINED, MONITORING}? --no--> 409
  |
  v
INSERT backfill_jobs (status = PENDING) + audit BACKFILL_STARTED + COMMIT
  |
  v
asyncio.create_task(_run_backfill_job(...))    <-- fire and forget, same process
  |
  v
API Response 202 (job id)                      [the background task then runs independently:]
                                              status = RUNNING, own DB session
                                              verify_authorized -> get_entity
                                              Telegram: iter_messages backward
                                              (offset_id = job checkpoint, else offset_date = to_date)
                                              persist per message; advance job.checkpoint_message_id
                                              (the OLDEST id seen - never the monitoring checkpoint)
                                              status = COMPLETED | PARTIAL | FAILED
                                              audit BACKFILL_COMPLETED / BACKFILL_FAILED

```

Response (202)

```

{
  "id": 1,
  "source_id": 1,
  "status": "PENDING",
  "from_date": "2026-01-01T00:00:00Z",
  "to_date": "2026-06-01T00:00:00Z",
  "requested_limit": 1000,
  "messages_found": 0,
  "messages_inserted": 0,
  "messages_skipped": 0,
  "checkpoint_message_id": null,
  "error_count": 0,
  "error_message": null,
  "started_at": null,
  "completed_at": null,
  "created_at": "2026-09-06T10:00:00Z"
}

```

Field	Type	Description
<code>id</code>	integer	Job id — use it with <code>GET .../backfill/{job_id}</code> .
<code>status</code>	string	<code>PENDING</code> , <code>RUNNING</code> , <code>COMPLETED</code> , <code>PARTIAL</code> , <code>FAILED</code> .
<code>requested_limit</code>	integer	The clamped limit actually applied.
<code>messages_found</code> / <code>messages_inserted</code> / <code>messages_skipped</code>	integer	Progress counters; <code>skipped</code> means already stored (dedup).
<code>checkpoint_message_id</code>	integer or null	Oldest message id persisted so far — the resume point.
<code>error_count</code> / <code>error_message</code>	integer / string	Persistence or walk failures.
<code>started_at</code> / <code>completed_at</code>	datetime or null	Job lifecycle timestamps.

HTTP Status Codes

Status	Meaning	When It Occurs
202	Accepted	Job queued.
404	Not Found	No such source.
409	Conflict	Source access not positively verified.

Error Responses

```
{ "detail": "source 2 is in state JOIN_REQUEST_PENDING; backfill requires verified access first" }
```

Durability gap (ISS-02). The job runs as an in-process `asyncio` task. If the service restarts while it is running, the row stays `RUNNING` forever and nothing resumes it — there is no startup sweep and no cancel endpoint. Recovery is manual: start a new backfill, which resumes safely from `checkpoint_message_id`.

Backfill maintains its **own** checkpoint and never advances `collection_checkpoints`, so it cannot corrupt the forward monitoring pipeline. There is also no concurrency guard: several backfills can be queued for one source at once, all competing for the same Telegram budget.

Example

```
curl -X POST http://localhost:8000/api/sources/1/backfill \
-H "Content-Type: application/json" \
-d '{"limit": 500, "from_date": "2026-01-01T00:00:00Z"}'
```

```
import httpx, time

job = httpx.post("http://localhost:8000/api/sources/1/backfill", json={"limit": 500}).json()
while True:
    j = httpx.get(f"http://localhost:8000/api/sources/1/backfill/{job['id']}").json()
    print(j["status"], j["messages_inserted"], "/", j["requested_limit"])
    if j["status"] in ("COMPLETED", "PARTIAL", "FAILED"):
        break
    time.sleep(5)
```

5.32 GET /api/sources/{source_id}/backfill

Purpose. Lists backfill jobs for one source, newest first (most recent 20).

Authentication. None. Database only.

Request Parameters

Parameter	Type	Required	Description	Example
source_id	integer (path)	Yes	Source id.	1

Processing Flow

```
Client Request -> BackfillService.list_backfill_jobs -> SELECT ... ORDER BY id DESC LIMIT 20
```

Response (200) — array of `BackfillJobResponse` (fields as in 5.31).

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Including an empty array for an unknown source (no 404).

Example

```
curl http://localhost:8000/api/sources/1/backfill
```

5.33 GET /api/sources/{source_id}/backfill/{job_id}

Purpose. Polls one backfill job's progress.

Authentication. None. Database only.

Request Parameters

Parameter	Type	Required	Description	Example
source_id	integer (path)	Yes	Source id — must match the job's owner.	1
job_id	integer (path)	Yes	Job id from the 202 response.	1

Processing Flow

```
Client Request -> BackfillService.get_backfill_job -> SELECT WHERE id = ? AND source_id = ? -> 404 if None
```

Response (200) — one `BackfillJobResponse` .

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Job found for that source.
404	Not Found	<code>{"detail": "backfill job 7 not found for source 1"} .</code>

Example

```
curl http://localhost:8000/api/sources/1/backfill/1
```

5.34 GET /api/notifications

Purpose. Reads persisted application notifications — significant access/collection state changes.

When to use. Polling for `TELEGRAM_ACCESS_GRANTED` after a join request, or `TELEGRAM_COLLECTION_ERROR` alerts.

Authentication. None. Database only.

Request

```
GET /api/notifications?limit=50&offset=0&unread_only=true HTTP/1.1
Host: localhost:8000
```

Parameter	Type	Required	Description	Example
<code>limit</code>	integer (query)	No (default 50)	Page size, unbounded in code.	<code>50</code>
<code>offset</code>	integer (query)	No (default 0)	Rows to skip.	<code>0</code>
<code>unread_only</code>	boolean (query)	No (default <code>false</code>)	Only rows with <code>is_read = false</code> .	<code>true</code>

Processing Flow

```
Client Request -> NotificationService.list_notifications -> SELECT ... ORDER BY id DESC LIMIT/OFFSET
```

Response (200)

```
[
  {
    "id": 1,
    "event_type": "TELEGRAM_ACCESS_GRANTED",
    "source_id": 2,
    "telegram_entity_id": "1234567890",
    "source_name": "Example Private Channel",
    "previous_status": "JOIN_REQUEST_PENDING",
    "new_status": "JOINED",
    "payload": {
      "event_type": "TELEGRAM_ACCESS_GRANTED",
      "source_id": 2,
      "telegram_entity_id": "1234567890",
      "source_name": "Example Private Channel",
      "previous_status": "JOIN_REQUEST_PENDING",
      "new_status": "JOINED",
      "timestamp": "2026-09-06T21:20:00Z"
    },
    "is_read": false,
    "created_at": "2026-09-06T21:20:00Z"
  }
]
```

Field	Type	Description
id	integer	Notification id.
event_type	string	TELEGRAM_ACCESS_GRANTED , TELEGRAM_ACCESS_REJECTED , or TELEGRAM_COLLECTION_ERROR . (TELEGRAM_MONITORING_STARTED is declared but never emitted.)
source_id	integer or null	Related source.
telegram_entity_id	string or null	Telegram id of that source.
source_name	string or null	Title, username, or identifier.
previous_status / new_status	string or null	The transition that raised it.
payload	object	Full JSON payload, also what any registered dispatcher receives.
is_read	boolean	Set by POST /api/notifications/{id}/read .
created_at	datetime	Creation timestamp.

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Including an empty array.

The `processed` column (used by the dispatch job) is **not** exposed in the response — `is_read` is an operator-facing flag, `processed` an internal dispatch flag. With no dispatcher registered (the default), notifications simply accumulate here.

Example

```
curl "http://localhost:8000/api/notifications?unread_only=true&limit=20"
```

5.35 POST /api/notifications/{notification_id}/read

Purpose. Marks one notification as read.

Authentication. None. Database only.

Request Parameters

Parameter	Type	Required	Description	Example
notification_id	integer (path)	Yes	Notification id.	1

Processing Flow

```
Client Request -> NotificationService.mark_read -> is_read = true -> flush (NO COMMIT)
-> response serialized from the in-memory object
-> get_db closes the session -> TRANSACTION ROLLED BACK -> flag lost
```

Confirmed defect ISS-01 (same root cause as PATCH /api/sources/{id}). `NotificationRepository.mark_read` flushes but never commits, and no caller commits. The endpoint returns **200 with `is_read: true`**, but the row is unchanged on the next read — so `?unread_only=true` keeps returning the same notification forever. Verified against SQLAlchemy 2.0.52 + aioslite. One-line fix: commit in `NotificationService.mark_read`.

Response (200) — the updated `NotificationOut` with `is_read: true` (in the response body only, see above).

HTTP Status Codes

Status	Meaning	When It Occurs
200	OK	Marked read.
404	Not Found	<code>{"detail": "notification not found"}</code> .

Example

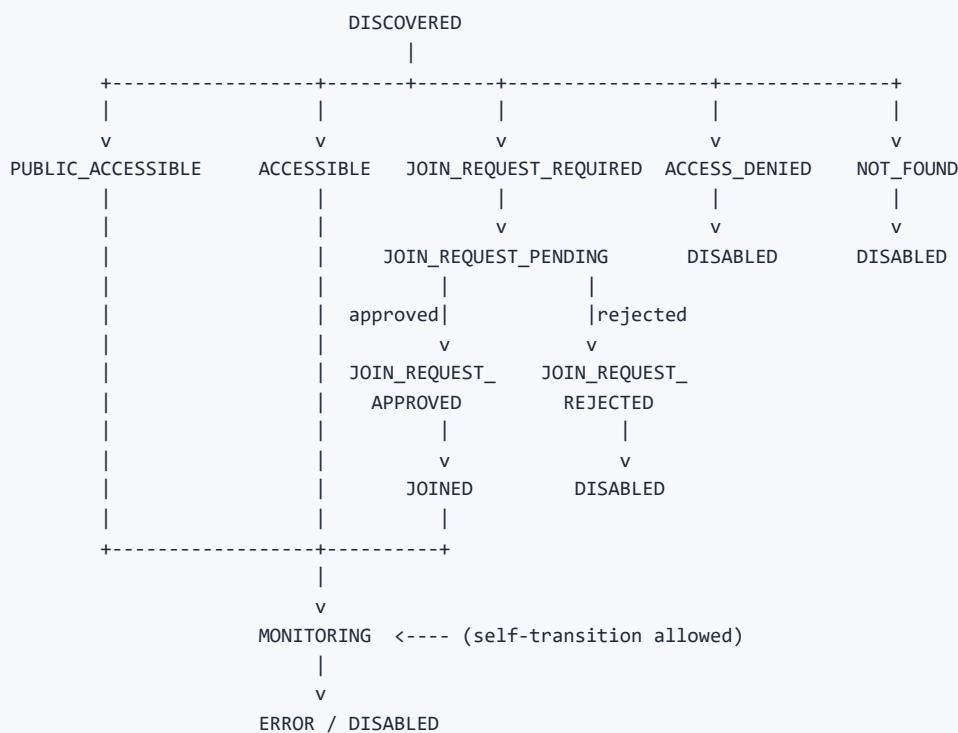
```
curl -X POST http://localhost:8000/api/notifications/1/read
```

6. End-to-End Workflows

Each workflow below is labelled **Implemented**, **Partially Implemented**, or **Not Implemented** based on what the code actually does.

6.1 The access state machine (the backbone of every workflow)

Thirteen states, with every transition validated against an explicit allow-list in `access_manager.ALLOWED_TRANSITIONS`. An invalid transition is refused and logged; it does **not** raise through to the caller — `_apply_status` leaves `access_status` untouched and records the attempt in `status_reason`.



ERROR is recoverable: an explicit POST /check-access can move it back to anything DISCOVERED can reach. DISABLED can only return to DISCOVERED.

Hard invariant, enforced centrally in `SourceService._apply_status`: `monitoring_enabled` is forced to `false` whenever `access_status` is set to `ERROR`, `ACCESS_DENIED`, `NOT_FOUND`, `JOIN_REQUEST_REQUIRED`, `JOIN_REQUEST_PENDING`, `JOIN_REQUEST_REJECTED` or `DISABLED`.

6.2 Telegram Account Authentication Flow — Implemented

```
Configure .env (TELEGRAM_API_ID / TELEGRAM_API_HASH)
|
v
Start service -> lifespan: init_db -> connect (best effort) -> scheduler.start()
|
v
GET /api/telegram/status          authenticated == false
|
v
POST /api/telegram/auth/send-code -> Telegram: auth.sendCode
|                               phone_code_hash held IN MEMORY only
v
POST /api/telegram/auth/verify-code -> Telegram: auth.signIn
|
+-- requires_password == true --> POST /api/telegram/auth/verify-password
|                               -> Telegram: auth.checkPassword
v
Session file authorized at TELEGRAM_SESSION_PATH (clear_session_invalid())
|
v
GET /api/telegram/status          authenticated == true, masked account returned
|
v
Every restart from here on reconnects and reuses the session - no repeat login
```

Restart caveat: the pending `phone_code_hash` lives only in process memory. If the service restarts between `send-code` and `verify-code`, `verify-code` returns `"no pending code request for this phone - call send-code first"` and the flow must restart. The multi-step login is also **not safe across multiple service replicas** — a second replica has neither the pending hash nor the session file.

6.3 Source Discovery and Registration Flow — Implemented



Discovery is never monitoring. No discovery path auto-joins or auto-monitors. Every candidate requires an explicit operator call to be tracked, and bot-discovered sources are registered dormant.

6.4 Private Source Access Flow (join request) — Implemented, with a Telegram-imposed blind spot

```

POST /api/sources                                -> access_status = JOIN_REQUEST_REQUIRED
|
v
POST /api/sources/{id}/request-access
|
+-- a PENDING request already exists? -> return it, send NOTHING to Telegram
|
v
Telegram: channels.joinChannel (username) OR messages.importChatInvite (invite hash)
|
+-- joined immediately                -> JOINED          + notification TELEGRAM_ACCESS_GRANTED
+-- InviteRequestSentError            -> JOIN_REQUEST_PENDING (access-request row, next_check_at +12h)
+-- error / FloodWait                 -> ERROR
|
v
Background job reconcile_pending_access, every ACCESS_RECONCILIATION_INTERVAL_HOURS (12)
|
+-- invite hash -> Telegram: messages.checkChatInvite
|   ChatInviteAlready          -> APPROVED
|   InviteHashExpired/Invalid  -> REJECTED
|   ChatInvite                  -> still PENDING
|
+-- username -> Telegram: get_entity, inspect the `left` flag
|   left == False               -> APPROVED
|   otherwise                   -> still PENDING (see blind spot below)
|
v
APPROVED -> JOIN_REQUEST_APPROVED -> JOINED -> notification -> start_monitoring() AUTOMATICALLY
REJECTED -> JOIN_REQUEST_REJECTED -> monitoring_enabled = false -> notification
      No alternative access mechanism is ever attempted afterwards.

```

Telegram platform limitation: for a *username-based* join request that an admin silently declines, Telegram exposes no decline signal to the requesting account. The request accurately stays `JOIN_REQUEST_PENDING` indefinitely. This is what the API exposes, not a defect — but it means a pending request is not proof that a decision is still outstanding.

6.5 Message Monitoring / Collection Flow — Implemented

```

POST /api/sources/{id}/monitoring/start
|
v
verify_authorized -> status check -> transition to MONITORING -> immediate first cycle
|
v
Scheduler job run_collection_cycle, every COLLECTION_INTERVAL_MINUTES (5), max_instances=1
|
v
FOR EACH source WHERE monitoring_enabled = true AND access_status = 'MONITORING':
|
v
verify_authorized()
|   transient network failure -> skip this cycle, monitoring stays ON
|   session invalid           -> source -> ERROR, monitoring OFF, stop
v
Telegram: get_entity
|
v
checkpoint = collection_checkpoints.get_or_create(source)
|
v
Telegram: iter_messages(min_id = checkpoint.last_message_id, reverse=True,
                        limit = COLLECTION_BATCH_LIMIT)    forward, oldest-first
|
+-- FloodWaitError      -> stop, report wait, keep what was already fetched
+-- ChannelPrivateError -> "access was revoked" -> source -> ERROR + notification
+-- session invalid     -> latch + source -> ERROR
|
v
FOR EACH message: normalize -> write raw evidence JSON -> INSERT ... ON CONFLICT DO NOTHING
                    -> advance checkpoint -> COMMIT      (one transaction per message)
|
v
Optional media download when MEDIA_DOWNLOAD_ENABLED=true, within size/type policy
|
v
Update source.last_message_id / last_collected_at -> audit -> structured JSON log line

```

Restart safety: per-message commits mean the checkpoint only ever advances past durably persisted messages. An interrupted cycle resumes at exactly the right point, and the DB unique constraint makes re-collection a no-op rather than a duplicate.

6.6 Live Search Flow — Implemented

```

Search request
  |
  v
_require_session()      -> 401 immediately if the session is latched invalid (no RPC)
  |
  v
rate_policy.run_bounded_search(key)
  |   semaphore: max 3 concurrent search RPCs across all callers
  |   in-flight de-duplication: identical concurrent calls share ONE execution
  v
Global search           Channel search
  |                       |
Telegram: checkSearchPostsFlood   registered source: trust stored status
  |                               arbitrary channel: fresh live probe
+-- quota -> searchPosts          |
+-- else  -> searchGlobal          v
  |                               Telegram: messages.search
  v                               |
Normalize -> local filters -> sort -> cursor -+
  |
  v
audit SEARCH_GLOBAL / SEARCH_CHANNEL + structured log
  |
  v
API response      NOTHING IS PERSISTED - search is read-only and ephemeral
  |
  v
(optional) POST /api/telegram/search/results/save  <- the ONLY path search data is stored

```

6.7 Historical Backfill Flow — Partially Implemented

Implemented: queueing, access gating, the backward walk, an independent per-job checkpoint, resume-from-checkpoint on a *new* job, progress polling, **PARTIAL** / **FAILED** outcomes. Not implemented: crash recovery, cancellation, concurrency limits per source, and progress push.

```

POST /api/sources/{id}/backfill  -> 202 with job id (status PENDING)
  |
  v
asyncio.create_task(_run_backfill_job)      IN-PROCESS, fire and forget
  |
  v
status = RUNNING; open an independent DB session
  |
  v
verify_authorized -> get_entity      (failure -> FAILED with the reason)
  |
  v
resume_before_id = job.checkpoint_message_id ; remaining = limit - messages_found
  |
  v
Telegram: iter_messages backward (offset_id = resume point, else offset_date = to_date)
  stop at from_date or when the limit is reached
  |
  v
persist per message -> advance job.checkpoint_message_id to the OLDEST id seen
  (collection_checkpoints is deliberately NEVER touched)
  |
  v
COMPLETED | PARTIAL (error but some inserted) | FAILED -> audit -> structured log

```

6.8 Bot Interaction Flow — Implemented

```

POST /api/sources with a bot username -> source_type resolved to "bot"
|
| audit TELEGRAM_BOT_DETECTED
v
POST /api/telegram/bots/{id}/start
|
v
Telegram: sendMessage("/start") one message, nothing else, ever
|
v
Bounded poll (BOT_START_TIMEOUT_SECONDS / BOT_START_POLL_INTERVAL_SECONDS)
| returns as soon as a reply arrives; never waits out the full timeout needlessly
v
Reply text + inline button URLs scanned for t.me/telegram.me links only
| non-Telegram URLs discarded, never fetched
| callback buttons surfaced but NEVER executed
| capped at 10 links
v
Each link -> register as DORMANT source (monitoring off, discovery_type = BOT_RESPONSE)
|
v
Operator decides next steps via request-access / monitoring-start

```

The bot's reply is **never written to** `telegram_messages`, and no wider bot chat history is fetched. Only the one reply is inspected.

6.9 Notification Flow — Partially Implemented

```

State change (access granted / rejected / collection error)
|
v
NotificationService.create() -> INSERT notifications (processed = false, is_read = false)
|
v
Scheduler job process_notifications, every NOTIFICATION_PROCESSING_INTERVAL_MINUTES (1)
|
v
notification_manager.dispatch(payload) -> every registered dispatcher
|
| *** NO DISPATCHER IS REGISTERED BY DEFAULT ***
| The loop runs, dispatches to nobody, and marks the row processed.
v
mark_processed -> COMMIT
|
v
GET /api/notifications <- the only way anyone learns about it today

```

Not Implemented: WebSocket, webhook, email or dashboard delivery. `NotificationManager.register()` exists and works; no provider is wired in.

6.10 Not Implemented workflows

Workflow	Status	Note
Event-driven access-change fast path	Not Implemented (dead code)	<code>event_handler.register_access_change_handlers</code> is fully written and unit-tested but never called by the application. Contradicts README §11.
Sending arbitrary messages / posting / replying	Not Implemented	Only the literal <code>/start</code> to a bot.
Media re-download, retention or cleanup	Not Implemented	Downloads happen only during collection when enabled; nothing prunes.
Message export (CSV/JSON bulk)	Not Implemented	Only paged JSON via the API.
Multi-account rotation, proxies	Not Implemented	Single account, single connection.
NLP / OCR / entity extraction / correlation / risk scoring	Not Implemented	Explicit downstream scope.
Deleting or editing content on Telegram	Not Implemented	The service is read-only against Telegram apart from joins and <code>/start</code> .

7. Telegram Accessibility and Privacy Behavior

Everything the service can see is exactly **what the one authenticated Telegram user account can see** — no more. This section states that boundary precisely.

7.1 Access by source type

Source type	What the service can do	Status
Public channel (has a username)	Resolve, read history, search within it, monitor, backfill, discover via <code>contacts.search</code> . Probe result: <code>PUBLIC_ACCESSIBLE</code> .	Supported
Public group / supergroup (has a username)	Same as public channels. Sender identity is usually richer than in broadcast channels.	Supported
Private channel/group — account is already a member	Full read, search, monitor, backfill. Probe result: <code>ACCESSIBLE</code> (<code>left == False</code>).	Supported
Private channel/group with a username, not a member	Detected as <code>JOIN_REQUEST_REQUIRED</code> . No content until a join is approved.	Partially Supported (access must be granted first)
Private, invite link, joins immediately	<code>messages.importChatInvite</code> succeeds -> <code>JOINED</code> -> readable.	Supported
Private, invite link, approval required	Request submitted once -> <code>JOIN_REQUEST_PENDING</code> . Content only after Telegram approves.	Partially Supported
Private, no username and no invite	<code>ACCESS_DENIED</code> . Nothing is attempted.	Not Supported (by design)
Invite link expired / invalid	<code>ACCESS_DENIED</code> / <code>NOT_FOUND</code> . No retry with a mutated hash, ever.	Not Supported (by design)
Bot	Detected as <code>source_type = "bot"</code> . Message search is explicitly refused (<code>NOT_A_CHANNEL</code> , 422). Only the <code>/start</code> workflow applies.	Partially Supported (start only)
User account (a person)	<code>check_access</code> returns <code>ACCESSIBLE</code> for any resolvable user without a read probe . See the privacy note below.	Partially Supported
Restricted content (geo/age/copyright restricted by Telegram)	Not handled specially. Telegram's own restriction applies to the account; results simply do not appear.	Not Supported
Deleted or unavailable content	Not recoverable. <code>save_search_result</code> returns 404 when the message is gone. Already-collected copies remain in local storage.	Not Supported

Privacy note — user sources. `access_manager._check_username_or_id_access` returns `ACCESSIBLE` for a `User` entity immediately, skipping the read probe applied to channels. Because `ACCESSIBLE` is a monitorable state, a *person* or a *bot* can be registered as a source and put into `MONITORING`, at which point `collect_new_messages` walks `iter_messages(user_entity)` — which is the **one-to-one conversation history between the service's account and that user**. Only *channel search* rejects bots; registration, monitoring and backfill do not. The live database contains one such source today (a bot registered as `source_type = "user"`, in `MONITORING`, with 0 messages collected). This is a capability of the generic source path rather than a documented feature, and it deserves an explicit policy decision before the service is used more widely.

7.2 Capability matrix

Capability	Status	Evidence in code
Search public Telegram content beyond joined channels	Partially Supported	<code>channels.searchPosts</code> — only while free daily quota remains; the service never spends Stars.
Search Telegram-wide as a fallback	Partially Supported	<code>messages.searchGlobal</code> is account-scoped; a warning says so and <code>visibility</code> is <code>account_scoped</code> .
Read messages from public channels/groups	Supported	<code>iter_messages</code> after a verified probe.
Search messages within one channel	Supported	<code>messages.search</code> , native <code>from_user</code> /media/date filters.
Discover channels by keyword	Supported	<code>contacts.search</code> ; no pagination available from Telegram.
Join public groups/channels	Supported	<code>channels.joinChannel</code> via <code>request-access</code> .
Join private groups via invite link	Supported	<code>messages.importChatInvite</code> via <code>invites/join</code> .
Access invite-only sources requiring approval	Partially Supported	Request submitted once; content only after approval; approval detected within 12h.
Detect that a join request was declined	Partially Supported	Invite-hash requests: yes. Username-based: no signal exists — stays PENDING.
Access content without appropriate Telegram authorization	Not Supported — deliberately	No invite guessing, no membership spoofing, no <code>ChannelPrivateError</code> workaround anywhere.
Send messages	Partially Supported	Exactly one literal <code>/start</code> , only to a registered bot source. Nothing else.
Retrieve historical messages	Supported	Backfill, max 10,000 per job, subject to Telegram history visibility.
Monitor sources continuously	Supported	5-minute collection cycle, checkpointed.
Scrape/search messages	Supported within the account's access	Both live search and local search over collected data.
Download media	Supported, disabled by default	<code>MEDIA_DOWNLOAD_ENABLED=false</code> ; metadata is always recorded regardless.
Read message reactions, poll results, comment threads, stories	Not Implemented	Not requested from Telegram and not modelled. Only <code>reply_count</code> is captured.
Enumerate channel members / participant lists	Not Implemented	No participants RPC is called anywhere.
Read the account's own dialog list	Not Implemented	<code>get_dialogs</code> is never called.
Access another Telegram account's data	Not Supported	Single account; no impersonation.

7.3 What "public" means in this service

Three distinct meanings appear in responses, and conflating them is the most common misreading of the data:

Value	Where it appears	Meaning
<code>visibility:</code> <code>"public"</code>	Search results	The result came from <code>channels.searchPosts</code> (genuinely cross-membership), or from a channel that has a username.
<code>visibility:</code> <code>"account_scoped"</code>	Global search results	From <code>messages.searchGlobal</code> — Telegram's index for this account , in practice mostly channels it has joined. Not a search of all public Telegram.
<code>visibility:</code> <code>"member"</code>	Channel search results	The channel has no username; the service can read it only because the account is a member.

8. Telegram API Limitations

Limitations are separated by who imposes them. This distinction matters for review: platform limitations cannot be engineered away; application limitations can.

8.1 Telegram Platform Limitations

#	Limitation	Effect on this service
TP-1	Access equals the account's permissions.	Nothing can be collected that the authenticated account cannot see. No configuration changes this.
TP-2	<code>channels.searchPosts</code> is quota-gated . A small free daily allowance (<code>channels.checkSearchPostsFlood</code>), then Telegram Stars or Premium.	Cross-membership public search degrades to the account-scoped fallback once the quota is spent. The service never pays automatically.
TP-3	<code>messages.searchGlobal</code> is account-scoped . No Telegram method full-text searches every public channel for a standard account.	"Global search" results are not a complete view of Telegram. Surfaced via <code>visibility</code> and a <code>warnings</code> entry.
TP-4	<code>contacts.search</code> has no pagination . No offset/cursor parameter exists.	Channel discovery is a single bounded call; <code>has_more</code> is always <code>false</code> . Documented rather than faked.
TP-5	No decline signal for username-based join requests.	A silently declined request is indistinguishable from a pending one and stays <code>JOIN_REQUEST_PENDING</code> .
TP-6	<code>FloodWaitError</code> — Telegram dictates an exact wait period after too many requests.	Every affected operation stops and reports <code>retry_after_seconds</code> . Never bypassed.
TP-7	Session invalidation (revoked, logged out elsewhere, account deactivated/banned).	Latched; all Telegram operations return 401 until re-authentication.
TP-8	Metadata availability varies . Sender identity, view counts and reply counts are often absent, especially in broadcast channels.	Stored as <code>NULL</code> , never fabricated.
TP-9	Deleted content is gone.	Not recoverable. Already-collected copies persist locally.
TP-10	Private sources require membership or approval.	Enforced by Telegram; the state machine simply reflects it.
TP-11	Invite links expire and can be revoked.	Mapped to <code>ACCESS_DENIED</code> / rejection. No retry with a modified hash.
TP-12	Anti-abuse enforcement . Rapid joins, bulk history reads and repeated searches can trigger restrictions up to a permanent ban.	Mitigated only partially — see section 10.
TP-13	History visibility rules . A group may hide history from members who joined later.	Backfill silently returns less than requested; not distinguishable from "no more messages".
TP-14	Telegram may change API behaviour at any time.	The pinned floor is <code>teletthon>=1.36.0</code> with no upper bound — a breaking Telethon or Telegram change can surface without warning.

8.2 Application Limitations

#	Limitation	Impact	Fixable?
AP-1	No caller authentication or authorization.	Anyone who can reach the port controls the Telegram account.	Yes — SEC-01.
AP-2	<code>PATCH /api/sources/{id}</code> and <code>POST /api/notifications/{id}/read</code> never commit .	Both silently discard the change while returning 200.	Yes — ISS-01, one line each.
AP-3	Backfill jobs do not survive a restart.	Rows stuck in <code>RUNNING</code> ; no resume, no cancel.	Yes — ISS-02.
AP-4	Single Telegram account, single process.	No horizontal scaling; API traffic and collection share one rate budget.	Architectural.
AP-5	No application-level rate limiting.	Only a 3-way concurrency cap on <i>search</i> . Collection, access checks, joins and link extraction are uncapped.	Yes.
AP-6	No FloodWait backoff or scheduling.	The wait is reported, never honoured automatically; a caller can immediately retry and worsen the situation.	Yes.
AP-7	Multi-source search returns no cursor.	Sweeps cannot be paged.	Yes.
AP-8	Local search is substring-only.	No boolean/nested queries, no full-text index, no relevance, no result totals.	Yes.
AP-9	Unbounded <code>limit</code> on stored-message and notification endpoints.	A single request can attempt to materialize the whole table.	Yes — ISS-05.
AP-10	SQLite-specific insert. <code>MessageRepository</code> uses <code>sqlalchemy.dialects.sqlite.insert</code> .	The "swap <code>DATABASE_URL</code> to Postgres" claim does not hold for this one method without a change.	Yes.
AP-11	No migration tool. <code>create_all</code> plus a SQLite-only additive-column patch list.	Schema changes on other backends, and any column type change, are unmanaged.	Yes.
AP-12	<code>next_status_check_at</code> is never written.	Always <code>null</code> in responses; no scheduled re-probing of source access exists.	Yes.
AP-13	Event-driven access detection is dead code.	Approval detection latency is up to 12 hours, not near-real-time.	Yes — DOC-01.
AP-14	<code>telegram_accounts</code> identity columns are never populated.	<code>telegram_user_id</code> , <code>username</code> , <code>phone_masked</code> stay <code>NULL</code> ; the table records little more than "a session exists".	Yes.
AP-15	<code>request-access</code> returns 500 on an unusable session.	<code>RuntimeError</code> is unmapped by the route.	Yes — ISS-03.
AP-16	Deleting a source orphans its raw-evidence files.	<code>data/raw_messages/<id>/</code> is left on disk; audit rows keep a dangling <code>source_id</code> .	Yes.
AP-17	<code>source_type</code> in <code>SourceCreate</code> is accepted and ignored.	Misleading API contract.	Yes.

#	Limitation	Impact	Fixable?
AP-18	Stuck <code>scheduler_jobs</code> rows . An unclear shutdown leaves <code>status = "RUNNING"</code> forever. One such row exists in the live database.	Cosmetic/observability.	Yes.
AP-19	<code>processing_status</code> is always <code>"raw"</code> .	The <code>normalized</code> / <code>enriched</code> states are declared but never set.	By design (downstream scope).

9. Security

Findings are rated **Critical / High / Medium / Low / Informational**. Only issues supported by the code are listed; nothing here is speculative.

9.1 Security architecture as implemented

Area	Implementation
API authentication	None. No API key, token, cookie, CORS policy or IP restriction anywhere in <code>app/</code> .
Authorization	None. No roles, no per-source ownership, no scoping. Every caller can do everything.
Telegram credentials	<code>TELEGRAM_API_ID</code> / <code>TELEGRAM_API_HASH</code> are read from the environment/ <code>.env</code> only. No hard-coded defaults. Passed straight to Telethon; never logged, never returned by any endpoint.
Session material	The Telethon session file is a bearer credential for the Telegram account. Stored at <code>TELEGRAM_SESSION_PATH</code> , gitignored, never exposed through any endpoint.
Secrets in transit	Verification codes and the 2FA password are accepted as plain JSON over plain HTTP.
Secrets management	<code>.env</code> on local disk. No vault, no KMS, no encryption at rest.
Database security	Local SQLite file, no encryption, no DB authentication, filesystem permissions only.
Input validation	Pydantic v2 on every body; <code>Query</code> constraints on the search endpoints (<code>min_length</code> , <code>max_length</code> , <code>ge</code> , <code>le</code> , <code>pattern</code>); <code>normalize_identifier</code> rejects anything that is not a valid Telegram identifier; <code>clamp_limit</code> bounds page sizes in the discovery layer; <code>decode_cursor</code> rejects malformed cursors.
SQL injection	Not reachable — all queries are SQLAlchemy expression constructs. The only raw SQL is the startup <code>PRAGMA table_info</code> / <code>ALTER TABLE</code> patcher, built from a hard-coded internal list.
SSRF	Actively prevented: <code>url_classifier</code> parses URLs with <code>urllib</code> and never performs an HTTP request ; only <code>t.me</code> / <code>telegram.me</code> are recognized, everything else is discarded rather than fetched.
Logging	Structured JSON via <code>log_structured</code> ; search logs record <code>query_length</code> , not the query text. <code>mask_phone</code> masks phone numbers. Credential material is never logged.
Sensitive data at rest	Collected third-party message content and raw evidence JSON are stored unencrypted under <code>data/</code> .
Error handling	Stable error codes; messages carry only exception class names and Telegram's own public text. No stack traces are returned.

9.2 Findings

SEC-01 — No authentication or authorization on any endpoint · High

(Critical in any deployment reachable beyond loopback — which the default `API_HOST=0.0.0.0` produces.)

All 35 endpoints are open. An unauthenticated caller can start a Telegram login, submit the 2FA password, make the account **join channels** (`/api/telegram/invites/join` , `/api/sources/{id}/request-access`), **send /start messages** (`/api/telegram/bots/{id}/start`), drive unbounded Telegram search (risking FloodWait or an account restriction), read all collected third-party message content, and **delete sources with all their data**.

Evidence: no `middleware` , `Depends(security)` , `HTTPBearer` , `OAuth2` , `api_key` or `jwt` reference exists anywhere under `app/` . `app/main.py` registers only routers and one exception handler.

Recommendation: add an API-key or bearer dependency at router level; bind to `127.0.0.1` by default; place the service behind an authenticating reverse proxy. Treat the write/Telegram-affecting endpoints (auth, joins, bot-start, delete) as privileged even after that.

SEC-02 — Telegram session file is an unencrypted bearer credential on disk · Medium

`data/telegram_service.session` grants full API access to the Telegram account without any password or 2FA challenge. It is correctly gitignored and never exposed through the API, but it is unencrypted and protected only by filesystem permissions. A copy of that file elsewhere is a full account takeover.

Recommendation: restrict directory permissions, exclude `data/` from backups that leave the host, and document a revocation procedure ("terminate other sessions" in Telegram) for suspected compromise.

SEC-03 — Credentials accepted over plain HTTP · Medium

`auth/send-code` , `auth/verify-code` and `auth/verify-password` accept a phone number, a login code and the 2FA cloud password in plain JSON. The service speaks HTTP; no TLS is configured or referenced.

Recommendation: terminate TLS at a reverse proxy and never expose these endpoints beyond a trusted network segment.

SEC-04 — `.env` with live credentials present in the working tree · Medium

A populated `.env` exists alongside the code with real `TELEGRAM_API_ID` / `TELEGRAM_API_HASH` values. It is correctly gitignored (as are `*.session` and `data/`), so this is a host-security and handling concern, not a repository leak. *(No credential values are reproduced anywhere in this document.)*

Recommendation: move to a secret manager or environment injection for anything beyond a developer machine; rotate the `api_hash` if the host is shared.

SEC-05 — Collected third-party content stored unencrypted with no retention policy · Medium

The live database holds **12,805 messages and 8,134 media metadata rows**, plus one raw-evidence JSON file per message under `data/raw_messages/` . This is real third-party personal data, unencrypted, with no expiry, no access control and no deletion mechanism beyond `DELETE /api/sources/{id}` (which does not remove the evidence files).

Recommendation: define retention and access rules before this data set grows or is shared; consider full-disk or per-directory encryption; add an evidence-cleanup path to source deletion.

SEC-06 — No rate limiting on the application's own API · Medium

The only throttle is `SEARCH_CONCURRENCY_LIMIT = 3` on search RPCs. Nothing bounds request *rate*, and collection, access checks, link extraction (up to 20 live probes per call) and joins are not covered at all. Combined with SEC-01 this makes it straightforward for an unauthenticated caller to drive the account into a FloodWait or a restriction.

Recommendation: add per-client rate limiting at the proxy, and a global outbound Telegram request budget in `rate_policy` .

SEC-07 — Unbounded `limit` on stored-data endpoints · Low

`GET /api/messages`, `GET /api/sources/{id}/messages`, `GET /api/sources` and `GET /api/notifications` declare `limit: int` with no `le` constraint. `limit=1000000` is accepted and attempted — a memory/CPU denial-of-service against the service itself (not against Telegram).

Recommendation: apply `Query(default=50, ge=1, le=200)` consistently.

SEC-08 — Destructive endpoints without confirmation or soft-delete · Low

`DELETE /api/sources/{id}` cascades to messages, media, access requests and the checkpoint with no confirmation, no soft-delete and no undo. Combined with SEC-01, an accidental or hostile call is unrecoverable from the database alone.

Recommendation: soft-delete, or require an explicit confirmation parameter.

SEC-09 — SQLite foreign keys not enforced · Low

The app never sets `PRAGMA foreign_keys=ON`, so `ON DELETE SET NULL` on `audit_logs.source_id` and `notifications.source_id` never fires. The code compensates in exactly one place (`AuditService.detach_source` during duplicate rejection); every other deletion path leaves dangling references.

Recommendation: enable the pragma on connect, or accept and document dangling audit references.

SEC-10 — Dependency versions are floors with no upper bound · Low

Every entry in `requirements.txt` is `>=`, with no lock file. Builds are not reproducible and a breaking Telethon or FastAPI release can be pulled in silently.

Recommendation: pin exact versions or add a lock file.

SEC-11 — Good practices confirmed · Informational

Verified positives, worth stating explicitly for review:

- `api_hash`, session data, verification codes and 2FA passwords are **never** logged or returned. `GET /api/telegram/status` masks the phone number.
- Search audit logs record `query_length`, never the query text.
- Non-Telegram URLs are never fetched (no SSRF surface).
- No raw SQL string interpolation on any user-controlled path.
- Error responses expose no stack traces and no internal file paths.
- `.gitignore` correctly excludes `.env`, `*.session*`, `data/` and PoC output.
- Callback buttons on bot replies are surfaced but never executed.

10. Account Safety and Operational Considerations

The Telegram account is the single most valuable and most fragile asset in this system. If it is restricted or banned, the service stops entirely.

10.1 Protections that ARE implemented

Protection	Implementation	Where
Invalid-session latch	Once Telegram rejects the auth key, every operation short-circuits with no further RPC until re-authentication.	<code>client.py</code>
Mandatory pre-flight	<code>verify_authorized()</code> before every Telegram operation, so a known-bad session never issues the caller's actual RPC.	<code>client.py</code> and all services
No retry-on-FloodWait loop	<code>FloodWaitError</code> stops the operation and reports <code>retry_after_seconds</code> . Nothing sleeps and retries.	collector, access_manager, join_request_manager, errors
Search concurrency cap	At most 3 concurrent search RPCs across all callers.	<code>rate_policy.SEARCH_CONCURRENCY_LIMIT</code>
In-flight de-duplication	Identical concurrent searches share one Telegram call instead of issuing N.	<code>rate_policy.InFlightDeduplicator</code>
No overlapping jobs	Every APScheduler job is <code>max_instances=1</code> , <code>coalesce=True</code> .	<code>scheduler/jobs.py</code>
Single join request per source	A PENDING request short-circuits the endpoint; no duplicate is ever sent.	<code>source_service.request_access</code>
No alternative access after rejection	A rejected request only moves toward <code>DISABLED</code> ; nothing retries by another route.	state machine
No automatic payment	Exhausted <code>searchPosts</code> quota falls back rather than spending Telegram Stars.	<code>global_search.py</code>
Bounded page sizes	<code>clamp_limit</code> caps every search at 100; collection at <code>COLLECTION_BATCH_LIMIT</code> (200); backfill at 10,000.	pagination, config
Bounded link resolution	20 probes per <code>extract-links</code> call, 10 per bot reply.	<code>source_service</code> , <code>bot_service</code>
Single <code>/start</code> per call	One message, no button clicks, no callback execution, bounded wait.	<code>bot_interaction.py</code>
Reconciliation stops early on session invalidation	The 12-hour job breaks out mid-cycle rather than hammering the remaining requests.	<code>scheduler/jobs.py</code>
Session-invalid detection disables monitoring	A source moves to <code>ERROR</code> and monitoring is switched off rather than retrying forever.	<code>monitoring_service</code>
Full audit trail	Every access check, join, search, collection and bot interaction writes an <code>audit_logs</code> row.	<code>audit_service</code>

10.2 Protections that are NOT implemented (recommended)

Everything in this table is a **recommendation**. None of it exists in the code today.

Gap	Risk	Recommendation
No FloodWait backoff or scheduling	The wait is reported to the caller, but nothing enforces it. A retrying client can immediately re-trigger it and escalate toward a restriction.	Record the flood deadline centrally and refuse Telegram operations until it passes.
No global outbound request budget	Only search concurrency is bounded. Collection, access checks, joins and link extraction are uncapped.	Add a token-bucket in <code>rate_policy</code> covering every outbound RPC.
No jitter or spacing between collection cycles	Every monitored source is collected back-to-back on a fixed 5-minute boundary — a recognizably mechanical pattern.	Add jitter and inter-source delays.
No cap on monitored sources	The cycle iterates every monitoring-enabled source with no ceiling; adding sources linearly increases per-cycle request volume.	Cap sources per cycle, or stagger them across cycles.
No join-rate limiting	<code>invites/join</code> and <code>request-access</code> are unauthenticated and uncapped. Rapid joining is one of the most reliable triggers of Telegram restrictions.	Enforce a minimum interval and a daily join budget.
No repeated-search guard	<code>SEARCH_GLOBAL</code> audit rows are written but never consulted. Identical searches, spaced out in time, are re-issued freely.	Add short-lived result caching and a per-query cooldown.
No circuit breaker on repeated FloodWait	Successive floods produce no escalating restraint.	Trip a breaker after N floods in a window and pause outbound activity.
No account-health monitoring or alerting	Nothing watches for a rising FloodWait rate or a spike in <code>ERROR</code> sources.	Export metrics; alert on <code>session_invalid</code> , FloodWait frequency and collection failure rate.
No multi-account support	Single account, no rotation, no isolation of risky operations.	If throughput must grow, isolate high-risk operations onto a separate account rather than raising the rate on one.
No <code>/start</code> cooldown	Repeated calls send repeated <code>/start</code> messages to the same bot.	De-duplicate or rate-limit per bot source.

10.3 Operational risk by activity

Activity	Frequency in code	Account risk	Notes
Stored-message reads (<code>/api/messages</code> , <code>/api/sources/{id}/messages</code>)	Unlimited	None	No Telegram traffic at all. Prefer these.
Scheduled collection	Every 5 min per monitoring source	Low–Medium	Grows linearly with source count; incremental (<code>min_id</code>) so batches stay small once caught up.
Access check / registration	On demand	Low	1–2 RPCs each.
Per-channel live search	On demand	Medium	2 RPCs for a registered source, 3+ for an arbitrary channel (fresh probe every call).
Global search	On demand	Medium	Quota-tracked by Telegram; <code>checkSearchPostsFlood</code> is itself an RPC.
Multi-source search	On demand	Medium–High	Sequential search across many sources in one request.
<code>extract-links</code>	On demand	Medium–High	Up to 20 live probes per call, outside the concurrency cap.
Historical backfill	On demand	High	Up to 10,000 messages of bulk history reading — the classic FloodWait trigger.
Joining channels	On demand	High	The most common cause of Telegram account restrictions.
Sending <code>/start</code>	On demand	Medium	Visible outbound messaging activity.

10.4 Evidence from the running deployment

Observed in the live SQLite database at the time of analysis:

- **12,805** messages collected across **4** sources; **12,804** by monitoring and **1** by a saved search result.
- **71** completed collection cycles, plus **1** row still marked `RUNNING` — evidence of an unclean shutdown (AP-18).
- **288** audit rows. Only **1** failure recorded (a `SEARCH_CHANNEL`); **no** FloodWait-driven collection failures.
- **0** access requests, **0** backfill jobs, **0** notifications — those paths are implemented and unit-tested but have **never been exercised in this deployment**.
- **5** collection checkpoints, all with `error_count = 0` .

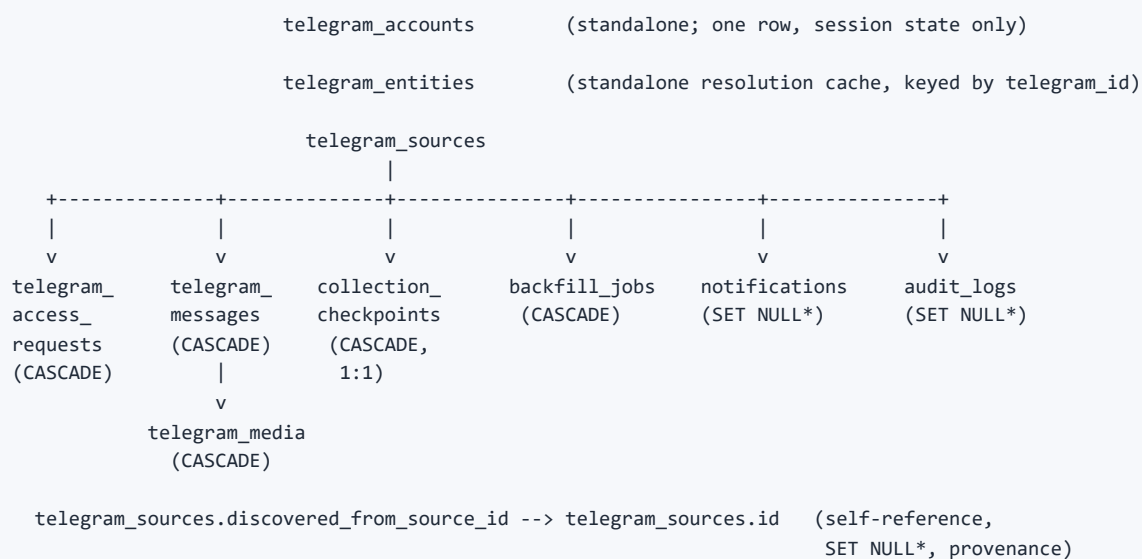
The account has therefore been operating well inside Telegram's limits so far. That reflects modest usage, not the presence of a rate-limiting safety net.

11. Database Interaction

11.1 Schema overview — 11 tables

Table	Purpose	Key columns
telegram_accounts	One row describing the service's Telegram session.	session_path , is_authenticated , connected_at , last_seen_at . telegram_user_id / username / phone_masked are never written (AP-14).
telegram_sources	The central entity: every registered channel/group/bot/user.	identifier (normalized), telegram_entity_id , source_type , access_status , monitoring_enabled , last_message_id , last_probe_* , discovered_from_source_id , discovery_type
telegram_access_requests	One row per submitted join request.	source_id , status , requested_at , next_check_at , attempt_count , approved_at , rejected_at
telegram_messages	Collected messages. UNIQUE(source_id, telegram_message_id) is the real dedup guarantee.	telegram_message_id , sender fields, message_date , text , views / forwards / reply_count , media_type , raw_data_hash , raw_data_ref , collection_method
telegram_media	Media metadata (always) and download state (optional).	message_id , media_type , filename , mime_type , file_size , local_path , sha256 , downloaded
telegram_entities	Resolution cache. Never proof of access.	telegram_id (unique), entity_type , username , title
collection_checkpoints	Forward monitoring anchor, one per source.	source_id (unique), last_message_id , last_success_at , last_error_at , error_count
backfill_jobs	Historical collection jobs with their own backward checkpoint.	source_id , status , from_date / to_date , requested_limit , messages_* , checkpoint_message_id
notifications	Persisted state-change events.	event_type , source_id , previous_status , new_status , payload (JSON), is_read , processed
audit_logs	Append-only trail of every significant operation.	event_type , source_id , actor , details (JSON), success , timestamp
scheduler_jobs	Background job run history (observability, not a lock).	job_name , started_at , finished_at , status , details (JSON)

11.2 Entity relationships



* ON DELETE SET NULL is declared but does NOT fire: SQLite foreign keys are not enabled (SEC-09). Only AuditService.detach_source clears references, and only during duplicate-source rejection.

11.3 Write paths

Operation	Tables written	Transaction behaviour
Register source	telegram_sources, audit_logs, telegram_entities	Commit after insert, then again after the access check.
Check access	telegram_sources, telegram_entities, audit_logs	Single commit at the end.
Request access	telegram_access_requests, telegram_sources, audit_logs, notifications	Single commit at the end.
Monitoring cycle	telegram_messages, telegram_media, collection_checkpoints, telegram_sources, audit_logs, notifications	One commit per message , then a final commit for source/audit updates.
Backfill batch	telegram_messages, telegram_media, backfill_jobs, audit_logs	One commit per message, advancing the job's own checkpoint.
Save search result	telegram_sources, telegram_messages, telegram_media, audit_logs	Commit per persistence step.
Live search	audit_logs only	One commit. No message rows are ever written by a search.
Bot start	telegram_sources (discovered), audit_logs	Commits between phases.
Scheduler jobs	scheduler_jobs, plus each job's own tables	Commit at start and finish of each run.
PATCH source / mark notification read	nothing	Flush without commit — the change is rolled back (ISS-01).

11.4 Data integrity mechanisms

- **De-duplication:** `UNIQUE(source_id, telegram_message_id)` with `INSERT ... ON CONFLICT DO NOTHING`. The insert's `rowcount` is what decides inserted-vs-skipped — never an application pre-check, so it stays correct under concurrency.
- **Source de-duplication:** two passes — the normalized identifier, then the resolved `telegram_entity_id` (which catches `@name` versus numeric id for the same channel).
- **Checkpoint correctness:** advanced only after a message is durably committed, and only ever forward.
- **Checkpoint separation:** backfill has its own `checkpoint_message_id` and never touches `collection_checkpoints`.
- **Evidence integrity:** every persisted message stores a SHA-256 of the raw Telegram payload plus a path to the preserved JSON; downloaded media is SHA-256 hashed too.
- **State-machine gating:** every `access_status` write funnels through `_apply_status`, which validates the transition and enforces the monitoring invariant.

11.5 Migrations

`init_db()` runs `Base.metadata.create_all` (creates missing tables only) and then `_apply_additive_column_patches`, a hard-coded list of `ALTER TABLE ... ADD COLUMN` statements applied **only on SQLite** for columns added after a table may already exist. There is no Alembic setup, no version table, and no support for column changes, drops or renames.

The additive patcher covers exactly six columns: `telegram_messages.collection_method` and `telegram_sources.last_probe_status / last_probe_reason / last_probe_at / discovered_from_source_id / discovery_type`. Any future column must be added to that list by hand, and on PostgreSQL the patcher is a no-op — an existing Postgres database would silently miss every one of them.

12. Configuration

All configuration is environment-driven through `pydantic-settings`, loaded from `telegram_service/.env`. No secret has a hard-coded default. Values are cached process-wide with `lru_cache`, so **changing `.env` requires a restart**.

12.1 Environment variables

Variable	Required	Purpose	Example / Default
TELEGRAM_API_ID	Yes , for any Telegram operation	MTPROTO application id from <code>my.telegram.org/apps</code> .	<TELEGRAM_API_ID>
TELEGRAM_API_HASH	Yes , for any Telegram operation	MTPROTO application hash. Secret .	<TELEGRAM_API_HASH>
TELEGRAM_SESSION_PATH	No	Where Telethon persists the authorized session. Treat as a credential .	<code>./data/telegram_service.session</code>
TELEGRAM_PHONE	No	Convenience only; the login flow takes the phone in the request body. Never logged (a <code>mask_phone()</code> helper exists).	<TELEGRAM_PHONE>
DATABASE_URL	No	SQLAlchemy async URL.	<code>sqlite+aiosqlite:///./data/telegram_service.db</code>
MEDIA_DOWNLOAD_ENABLED	No	Master switch for downloading media bytes. Metadata is stored regardless.	<code>false</code>
MEDIA_STORAGE_PATH	No	Directory for downloaded media.	<code>./data/media</code>
MAXIMUM_MEDIA_SIZE_BYTES	No	Per-file download ceiling; larger files are skipped, metadata kept.	<code>20971520</code> (20 MB)
ALLOWED_MEDIA_TYPES	No	Comma-separated download allow-list.	<code>photo,document,video,audio,voice</code>
RAW_EVIDENCE_STORAGE_PATH	No	Root for per-message raw JSON evidence.	<code>./data/raw_messages</code>
COLLECTION_BATCH_LIMIT	No	Max messages fetched per source per collection cycle.	<code>200</code>
BOT_START_TIMEOUT_SECONDS	No	How long to wait for a bot's <code>/start</code> reply.	<code>15</code>
BOT_START_POLL_INTERVAL_SECONDS	No	Poll interval while waiting for that reply.	<code>1</code>
ACCESS_RECONCILIATION_INTERVAL_HOURS	No	Pending join-request reconciliation cadence.	<code>12</code>
COLLECTION_INTERVAL_MINUTES	No	Monitoring collection cadence.	<code>5</code>
CONNECTION_HEALTH_CHECK_INTERVAL_MINUTES	No	Telegram connection health-check cadence.	<code>5</code>
NOTIFICATION_PROCESSING_INTERVAL_MINUTES	No	Notification dispatch cadence.	<code>1</code>

Variable	Required	Purpose	Example / Default
LOG_LEVEL	No	Root log level.	INFO
API_HOST	No	Uvicorn bind address. <code>0.0.0.0</code> exposes an unauthenticated API on every interface — see SEC-01.	<code>0.0.0.0</code>
API_PORT	No	Uvicorn port.	8000

There is **no** configuration for API authentication, TLS, CORS, rate limiting, proxies, or a second Telegram account — those features do not exist in the code, so no setting can enable them.

12.2 Example `.env` (placeholders only)

```
# --- Telegram MTPROTO credentials (https://my.telegram.org/apps) ---
TELEGRAM_API_ID=<TELEGRAM_API_ID>
TELEGRAM_API_HASH=<TELEGRAM_API_HASH>
TELEGRAM_SESSION_PATH=./data/telegram_service.session
TELEGRAM_PHONE=<TELEGRAM_PHONE>

# --- Database ---
DATABASE_URL=sqlite+aiosqlite:///./data/telegram_service.db

# --- Collection / media policy ---
MEDIA_DOWNLOAD_ENABLED=false
MEDIA_STORAGE_PATH=./data/media
MAXIMUM_MEDIA_SIZE_BYTES=20971520
ALLOWED_MEDIA_TYPES=photo,document,video,audio,voice
RAW_EVIDENCE_STORAGE_PATH=./data/raw_messages
COLLECTION_BATCH_LIMIT=200

# --- Bot-start workflow ---
BOT_START_TIMEOUT_SECONDS=15
BOT_START_POLL_INTERVAL_SECONDS=1

# --- Scheduler intervals ---
ACCESS_RECONCILIATION_INTERVAL_HOURS=12
COLLECTION_INTERVAL_MINUTES=5
CONNECTION_HEALTH_INTERVAL_MINUTES=5
NOTIFICATION_PROCESSING_INTERVAL_MINUTES=1

# --- App ---
LOG_LEVEL=INFO
API_HOST=127.0.0.1      # recommended: do NOT expose an unauthenticated API on 0.0.0.0
API_PORT=8000
```

Never commit a real `.env`, session file, or database. `.gitignore` already excludes `.env`, `*.session*`, `data/` and `poc/output/*.json` — keep it that way. This document deliberately contains **no** real credential values.

13. How to Run the API

13.1 Prerequisites

Requirement	Detail
Python	3.11 or newer. The codebase uses PEP 604 unions (<code>str None</code>) at runtime in Pydantic models and <code>datetime.UTC</code> -style patterns; 3.11+ is the safe floor.
Telegram developer credentials	An <code>api_id</code> and <code>api_hash</code> from <code>https://my.telegram.org/apps</code> , created with the Telegram account the service will operate as.
A Telegram account	A real account with a phone that can receive the login code. The service inherits exactly this account's permissions.
Database	None to install — SQLite is created automatically at <code>data/telegram_service.db</code> .
Network	Outbound access to Telegram's MTProto endpoints.
Dependencies	<code>fastapi</code> , <code>uvicorn[standard]</code> , <code>telethon>=1.36.0</code> , <code>sqlalchemy>=2.0.30</code> , <code>aiosqlite</code> , <code>pydantic>=2.7</code> , <code>pydantic-settings</code> , <code>apscheduler</code> , <code>python-dotenv</code> , <code>python-multipart</code> , plus <code>pytest</code> , <code>pytest-asyncio</code> , <code>httpx</code> for tests.

13.2 Installation

```
cd telegram_service
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

13.3 Configuration

```
cp .env.example .env      # Windows: copy .env.example .env
```

Then edit `.env` and set at minimum:

```
TELEGRAM_API_ID=<TELEGRAM_API_ID>
TELEGRAM_API_HASH=<TELEGRAM_API_HASH>
API_HOST=127.0.0.1      # strongly recommended - the API has no authentication
```

13.4 Starting the service

```
uvicorn app.main:app --reload          # development
uvicorn app.main:app --host 127.0.0.1 --port 8000 # explicit bind
```

Startup performs, in order: create missing tables and apply additive column patches, attempt a Telegram connection (a failure is a warning, not fatal), start all four scheduler jobs. **There is no separate scheduler process to launch.**

Expected startup log lines:

```
[2026-09-06 10:00:00,000] INFO telegram_service.main: Started server process
[2026-09-06 10:00:00,120] WARNING telegram_service.main: startup: could not connect to Telegram yet (...) - will retry
INFO:      Uvicorn running on http://127.0.0.1:8000
```

13.5 Verifying it is up

```
curl http://localhost:8000/health # {"status":"ok"}
curl http://localhost:8000/ready  # {"status":"ready","telegram_configured":true}
curl http://localhost:8000/api/telegram/status
```

13.6 API documentation

FastAPI's built-in docs are enabled (they are never disabled in `app/main.py`):

```
http://localhost:8000/docs      Swagger UI - interactive, executes real calls
http://localhost:8000/redoc     ReDoc      - reference view
http://localhost:8000/openapi.json OpenAPI 3.x - the machine-readable contract
```

Swagger UI issues **real** requests against the live Telegram account. "Try it out" on `/api/telegram/invites/join` genuinely joins a channel; on `/api/telegram/bots/{id}/start` it genuinely sends a message; on `DELETE /api/sources/{id}` it genuinely deletes collected data. Since nothing is authenticated, treat `/docs` as a privileged console.

13.7 Running the tests

```
pytest -v          # service suite (tests/), no network and no on-disk DB required
cd poc && python -m pytest tests # the archived proof-of-concept's own suite, run separately
```

The suite uses an in-memory SQLite database and a fake Telethon-shaped client. `tests/test_endpoint_coverage.py` asserts that the set of endpoints exercised equals the set the app publishes, so a new route cannot land untested.

Documentation drift (DOC-02): the README states "68 tests". The suite currently defines **189** test functions across 26 files. The count in the README is stale; the tests themselves are not.

14. How to Use the API

A complete walkthrough, in the order a new developer should follow it. Every request below matches the real schemas.

```
Step 1 -> Start the API and confirm health
Step 2 -> Authenticate the Telegram account (once)
Step 3 -> Discover or register a source
Step 4 -> Verify the source's real access status
Step 5 -> (private sources only) Request access and wait for approval
Step 6 -> Start monitoring
Step 7 -> Search live, or read what has been collected
Step 8 -> (optional) Backfill history
Step 9 -> Monitor status and notifications
```

Step 1 — Start the API and confirm health

Endpoint: GET /health , GET /ready · **Input:** none

```
curl http://localhost:8000/health
curl http://localhost:8000/ready
```

```
{ "status": "ok" }
{ "status": "ready", "telegram_configured": true }
```

Step 2 — Authenticate the Telegram account

Endpoints: GET /api/telegram/status , then the three auth/* steps · **Input:** phone, code, and 2FA password if enabled

```
curl http://localhost:8000/api/telegram/status
# -> {"connected":true,"authenticated":false,...}

curl -X POST http://localhost:8000/api/telegram/auth/send-code \
-H "Content-Type: application/json" -d '{"phone":"+15551234567"}'
# -> {"ok":true,"requires_password":false,"error":null}

curl -X POST http://localhost:8000/api/telegram/auth/verify-code \
-H "Content-Type: application/json" -d '{"phone":"+15551234567","code":"12345"}'
# -> {"ok":true,...} or {"ok":false,"requires_password":true,"error":"2FA password required"}

curl -X POST http://localhost:8000/api/telegram/auth/verify-password \
-H "Content-Type: application/json" -d '{"password":"<SECRET>"}'
```

Confirm:

```
curl http://localhost:8000/api/telegram/status
# -> {"connected":true,"authenticated":true,"account":{"id":"...", "username":"...", "phone":"+1***67"},...}
```

Do this once. The session file is reused on every restart.

Step 3 — Discover or register a source

Option A — you already know the channel. POST /api/sources

```
curl -X POST http://localhost:8000/api/sources \
-H "Content-Type: application/json" \
-d '{"identifier": "@examplechannel", "monitoring_enabled": true}'
```

```
{ "id": 1, "identifier": "@examplechannel", "source_type": "channel",
  "access_status": "PUBLIC_ACCESSIBLE", "monitoring_enabled": true, "...": "..." }
```

Option B — find one first. GET /api/telegram/channels/search , then register by id.

```
curl -G "http://localhost:8000/api/telegram/channels/search" --data-urlencode "q=example"
curl -X POST "http://localhost:8000/api/telegram/channels/1234567890/register?monitoring_enabled=false"
```

Step 4 — Verify the real access status

Endpoint: GET /api/sources/1/access-status (or POST .../check-access to re-probe)

```
curl http://localhost:8000/api/sources/1/access-status
```

```
{ "source_id": 1, "access_status": "PUBLIC_ACCESSIBLE", "status_reason": null,
  "last_probe_status": "PUBLIC_ACCESSIBLE", "last_probe_at": "2026-09-06T09:15:02Z", "...": "..." }
```

Decision point:

access_status	Next step
PUBLIC_ACCESSIBLE , ACCESSIBLE , JOINED	Go to Step 6 (start monitoring).
JOIN_REQUEST_REQUIRED	Go to Step 5.
ACCESS_DENIED , NOT_FOUND	Stop — this account cannot reach it.
ERROR	Re-run POST .../check-access ; if it persists, check GET /api/telegram/status .

Step 5 — Request access (private sources only)

Endpoint: POST /api/sources/2/request-access · **Input:** none

```
curl -X POST http://localhost:8000/api/sources/2/request-access
```

```
{ "id": 1, "source_id": 2, "status": "PENDING",
  "requested_at": "2026-09-06T09:20:00Z", "next_check_at": "2026-09-06T21:20:00Z", "attempt_count": 1 }
```

Then wait. The 12-hour reconciliation job detects approval, moves the source to JOINED , writes a TELEGRAM_ACCESS_GRANTED notification, and **starts monitoring automatically**. Calling this endpoint again while PENDING is safe — it returns the existing request and sends nothing to Telegram.

Step 6 — Start monitoring

Endpoint: POST /api/sources/1/monitoring/start · **Input:** none

```
curl -X POST http://localhost:8000/api/sources/1/monitoring/start
```

```
{ "source_id": 1, "monitoring_enabled": true, "access_status": "MONITORING",  
  "monitoring_started_at": "2026-09-06T09:30:00Z",  
  "last_collected_at": "2026-09-06T09:30:04Z", "last_message_id": 72697 }
```

Remember: registering with `monitoring_enabled: true` is **not** enough — this call is what actually starts collection. It also runs the first cycle synchronously, so allow a generous timeout.

Step 7 — Search live, or read collected data

Live, against Telegram:

```
curl -G "http://localhost:8000/api/sources/1/search" --data-urlencode "q=invoice"  
curl -G "http://localhost:8000/api/telegram/search/global" --data-urlencode "q=invoice" --data-urlencode "limit=20"
```

Local, against collected data (free, no Telegram traffic):

```
curl "http://localhost:8000/api/messages?keyword=invoice&source_id=1&limit=20"  
curl "http://localhost:8000/api/sources/1/messages?media_type=photo&limit=20"
```

Promote one live result into stored data:

```
curl -X POST http://localhost:8000/api/telegram/search/results/save \  
  -H "Content-Type: application/json" \  
  -d '{"identifier":"@examplechannel","telegram_message_id":72697,"collection_method":"global_search"}'
```

Step 8 — Backfill history (optional)

```
curl -X POST http://localhost:8000/api/sources/1/backfill \  
  -H "Content-Type: application/json" -d '{"limit":500}'  
# -> 202 {"id":1,"status":"PENDING",...}  
  
curl http://localhost:8000/api/sources/1/backfill/1  
# -> {"status":"RUNNING","messages_inserted":120,...}
```

Step 9 — Monitor status and notifications

```
curl http://localhost:8000/api/sources/1/monitoring-status  
curl "http://localhost:8000/api/notifications?unread_only=true"  
curl http://localhost:8000/api/telegram/status
```

Complete Python client

```

"""End-to-end walkthrough against a running telegram_service."""
import time
import httpx

BASE = "http://localhost:8000"

with httpx.Client(base_url=BASE, timeout=120) as c:
    # 1 - health
    assert c.get("/health").json()["status"] == "ok"

    # 2 - the Telegram account must already be authorized
    status = c.get("/api/telegram/status").json()
    if not status["authenticated"]:
        raise SystemExit(f"authenticate first: {status['error']}")

    # 3 - register (409 simply means it already exists)
    r = c.post("/api/sources", json={"identifier": "@examplechannel", "monitoring_enabled": True})
    if r.status_code == 409:
        source = next(s for s in c.get("/api/sources").json()
                      if s["identifier"] == "@examplechannel")
    else:
        r.raise_for_status()
        source = r.json()
    sid = source["id"]

    # 4 - verify real access
    access = c.get(f"/api/sources/{sid}/access-status").json()
    print("access:", access["access_status"], access["status_reason"])

    # 5 - private source: request access once, then wait for reconciliation
    if access["access_status"] == "JOIN_REQUEST_REQUIRED":
        print("join request:", c.post(f"/api/sources/{sid}/request-access").json()["status"])
        raise SystemExit("approval pending - reconciliation runs every 12h")

    # 6 - start monitoring (runs the first collection synchronously)
    if access["access_status"] in ("PUBLIC_ACCESSIBLE", "ACCESSIBLE", "JOINED", "MONITORING"):
        m = c.post(f"/api/sources/{sid}/monitoring/start")
        m.raise_for_status()
        print("monitoring:", m.json())

    # 7 - live search, then the local copy
    live = c.get(f"/api/sources/{sid}/search", params={"q": "invoice", "limit": 10})
    if live.status_code == 200:
        print("live hits:", len(live.json()["results"]))
    else:
        print("live search refused:", live.json())

    stored = c.get("/api/messages", params={"keyword": "invoice", "source_id": sid, "limit": 10})
    print("stored hits:", len(stored.json()))

    # 8 - backfill and poll
    job = c.post(f"/api/sources/{sid}/backfill", json={"limit": 200}).json()
    while True:
        j = c.get(f"/api/sources/{sid}/backfill/{job['id']}").json()
        print("backfill:", j["status"], j["messages_inserted"])
        if j["status"] in ("COMPLETED", "PARTIAL", "FAILED"):
            break
        time.sleep(5)

```

15. Postman / cURL Examples

15.1 Postman setup

1. **Create an environment** with one variable: `base_url` = `http://localhost:8000`.
2. **Authentication:** set Auth type to **No Auth** on every request — the API has none.
3. **Headers:** add `Content-Type: application/json` only on requests that send a body (`POST` / `PATCH`).
4. **Import the contract:** `File > Import > Link` -> `http://localhost:8000/openapi.json` generates the whole collection automatically, which is preferable to hand-building it.

Postman's Collection Runner against these endpoints drives real Telegram traffic. Do not loop search or join requests — the account, not Postman, absorbs the consequences.

15.2 Request reference

Register a source

Method	<code>POST</code>
URL	<code>{{base_url}}/api/sources</code>
Headers	<code>Content-Type: application/json</code>
Auth	None
Body (raw JSON)	<code>{"identifier": "@examplechannel", "monitoring_enabled": true}</code>
Expected	<code>201</code> with a <code>SourceOut</code> ; <code>409</code> if already registered; <code>422</code> for a bad identifier

```
curl -X POST http://localhost:8000/api/sources \
-H "Content-Type: application/json" \
-d '{"identifier": "@examplechannel", "monitoring_enabled": true}'
```

Check access

Method	<code>POST</code>
URL	<code>{{base_url}}/api/sources/1/check-access</code>
Body	none
Expected	<code>200</code> with the refreshed <code>SourceOut</code> ; <code>404</code> if the source does not exist

```
curl -X POST http://localhost:8000/api/sources/1/check-access
```

Start monitoring

Method	POST
URL	{{base_url}}/api/sources/1/monitoring/start
Body	none
Expected	200 ; 409 if access is not verified — allow a long timeout, the first cycle runs inline

```
curl -X POST http://localhost:8000/api/sources/1/monitoring/start
```

Live global search

Method	GET
URL	{{base_url}}/api/telegram/search/global
Params	q=invoice , limit=20 , sort=date_desc
Expected	200 ; 429 with retry_after_seconds on FloodWait; 401 if the session is invalid

```
curl -G "http://localhost:8000/api/telegram/search/global" \
--data-urlencode "q=invoice" --data-urlencode "limit=20"
```

Multi-source search

Method	POST
URL	{{base_url}}/api/telegram/search/sources
Headers	Content-Type: application/json
Body	{"q": "invoice", "source_ids": [1, 3], "limit": 25}
Expected	200 ; inaccessible sources appear in skipped , never as an error

```
curl -X POST http://localhost:8000/api/telegram/search/sources \
-H "Content-Type: application/json" \
-d '{"q": "invoice", "source_ids": [1, 3], "limit": 25}'
```

Save a search result

Method	POST
URL	{{base_url}}/api/telegram/search/results/save
Body	{"identifier": "@examplechannel", "telegram_message_id": 72697}
Expected	201 with inserted: true (or false when already stored)

```
curl -X POST http://localhost:8000/api/telegram/search/results/save \
-H "Content-Type: application/json" \
-d '{"identifier": "@examplechannel", "telegram_message_id": 72697}'
```

Search stored messages (no Telegram traffic)

Method	GET
URL	{{base_url}}/api/messages
Params	keyword=invoice , source_id=1 , media_type=photo , limit=20
Expected	200 with a MessageOut array

```
curl "http://localhost:8000/api/messages?keyword=invoice&source_id=1&limit=20"
```

Join by invite link

Method	POST
URL	{{base_url}}/api/telegram/invites/join
Body	{"identifier": "https://t.me/+AbCdEfGhIjKlM"}
Expected	200 with status ∈ joined / already_member / pending_approval / already_pending / failed

```
curl -X POST http://localhost:8000/api/telegram/invites/join \
-H "Content-Type: application/json" \
-d '{"identifier": "https://t.me/+AbCdEfGhIjKlM"}'
```

Start a bot

Method	POST
URL	{{base_url}}/api/telegram/bots/5/start
Body	none
Expected	200 with status ∈ started / timed_out / failed ; 422 if the source is not a bot

```
curl -X POST http://localhost:8000/api/telegram/bots/5/start
```

Queue and poll a backfill

```
curl -X POST http://localhost:8000/api/sources/1/backfill \
-H "Content-Type: application/json" -d '{"limit": 500}'
curl http://localhost:8000/api/sources/1/backfill/1
```

15.3 Handling errors in a client

```
# Show the status code alongside the body
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/api/sources/9999
# 404

curl -s -G "http://localhost:8000/api/sources/2/search" --data-urlencode "q=x" | python -m json.tool
# {"error": {"code": "JOIN_PENDING", "message": "...", "retryable": false}}
```

```
import httpx

def call(client, method, url, **kw):
    r = client.request(method, url, **kw)
    if r.status_code == 429:
        # FloodWait
        body = r.json()["error"]
        raise RuntimeError(f"rate limited, wait {body['retry_after_seconds']}s")
    if r.status_code == 401:
        # session invalid
        raise RuntimeError("re-authenticate the Telegram account")
    if r.status_code in (403, 404, 409, 422):
        payload = r.json()
        detail = payload.get("detail") or payload.get("error", {}).get("message")
        raise RuntimeError(f"{r.status_code}: {detail}")
    r.raise_for_status()
    return r.json()
```

16. Error Handling

16.1 The two error envelopes

The service returns **two different error shapes**. Clients must handle both.

(a) FastAPI `HTTPException` — used by routes for domain errors:

```
{ "detail": "source already registered as id=1" }
```

(b) `TelegramSearchError` — Telegram-facing failures on the search/discovery/save paths, rendered by the global handler in `app/main.py`:

```
{ "error": { "code": "FLOOD_WAIT", "message": "Telegram is rate-limiting this account - wait 42s before retrying.", "re
```

(c) Pydantic validation — FastAPI's standard 422 body:

```
{ "detail": [ { "type": "missing", "loc": ["body", "identifier"], "msg": "Field required", "input": {} } ] }
```

`retry_after_seconds` appears **only** on `FLOOD_WAIT`.

16.2 Centralized error reference

Error	HTTP Status	Cause	Recommended Action
Pydantic validation failure	422	Missing/invalid body or query field.	Fix the request; inspect <code>detail[] .loc</code> .
<code>q</code> empty or > 256 chars	422	Route or discovery-layer guard.	Shorten or supply the query.
<code>limit</code> out of range	422	<code>Query(ge=1, le=100)</code> on search endpoints.	Use 1–100.
<code>sort</code> not <code>date_desc</code> / <code>relevanc</code> <code>e</code>	422	Route pattern constraint.	Use a supported value.
<code>INVALID_REQUEST</code> / invalid identifier	422	<code>normalize_identifier</code> rejected the string.	Use <code>@username</code> , a <code>t.me</code> link, or a numeric id.
"...is not a Telegram invite link"	422	A username was sent to <code>invites/join</code> .	Use <code>POST /api/sources</code> + <code>request-access</code> .
<code>USERNAME_INVALID</code>	422	Telegram: <code>UsernameInvalidError</code> .	Correct the username.
<code>NOT_A_CHANNEL</code>	422	The entity is a bot; bots have no searchable message source.	Use the bot-start workflow.
<code>INVITE_HASH_INVALID</code>	422	Telegram: <code>InviteHashInvalidError</code> .	Obtain a valid invite.
Source is not a bot	422	<code>SourceIsNotABotError</code> .	Only call bot-start on <code>source_type == "bot"</code> .
"source not found" / "message not found"	404	No such row.	Check the id.
<code>NOT_FOUND</code> (Telegram)	404	<code>UsernameNotOccupiedError</code> / <code>ChannelInvalidError</code> , or a saved message that no longer exists.	The entity or message is gone.
Backfill job not found	404	Wrong <code>job_id</code> / <code>source_id</code> pair.	Use the ids from the 202 response.
Duplicate source	409	Same identifier, or same resolved <code>telegram_entity_id</code> .	Use the existing source id from the message.
Invalid access-request state	409	Source is not <code>JOIN_REQUEST_REQUIRED</code> / <code>ERROR</code> .	No join is needed, or resolve the current state first.
<code>MonitoringStateError</code>	409	Access not verified, or the Telegram session is unusable.	Run <code>check-access</code> ; re-authenticate if needed.
<code>BackfillNotAllowedError</code>	409	Source access not positively verified.	Verify access first.
<code>NOT_FOUND</code> on save	409	Source exists but could not be located after a duplicate error.	Re-register or locate the source manually.
<code>NOT_AUTHENTICATED</code>	401	<code>verify_authorized()</code> failed before any RPC.	Complete the auth flow.
<code>TELEGRAM_SESSION_INVALID</code>	401	Auth key unregistered/revoked, session expired, account deactivated or banned.	Re-authenticate. Nothing else will work until then.

Error	HTTP Status	Cause	Recommended Action
AUTHENTICATION_ERROR	401	<code>AuthKeyError</code> / <code>SessionPasswordNeededError</code> outside the login flow.	Re-authenticate.
PREMIUM_OR_QUOTA_REQUIRED	402	<code>PremiumAccountRequiredError</code> — Premium or remaining quota needed.	Rely on the automatic fallback, or accept reduced coverage.
CHANNEL_PRIVATE	403	<code>ChannelPrivateError</code> — private or inaccessible.	Register and request access.
CHAT_ADMIN_REQUIRED	403	Admin privileges required.	Not achievable with this account.
USER_NOT_PARTICIPANT	403	Not a member of the chat.	Join first.
ACCESS_DENIED	403	Stored status is <code>ACCESS_DENIED</code> .	Unreachable for this account.
JOIN_REQUIRED	403	Stored status is <code>JOIN_REQUEST_REQUIRED</code> .	Call <code>request-access</code> .
JOIN_PENDING	403	A join request is awaiting approval.	Wait for reconciliation.
ACCESS_REJECTED	403	The join request was rejected.	No alternative is attempted, by policy.
INVITE_HASH_EXPIRED	410	The invite link expired.	Obtain a fresh invite.
FLOOD_WAIT	429	Telegram is rate-limiting the account.	Wait <code>retry_after_seconds</code> before any further Telegram call. Nothing enforces this for you.
TELEGRAM_RPC_ERROR	502	Any other Telethon <code>RPCError</code> .	Inspect logs; usually transient.
NETWORK_ERROR	503	<code>ConnectionError</code> / <code>OSError</code> / <code>asyncio.TimeoutError</code> .	Retry with backoff; the health-check job reconnects on its own cadence.
UNKNOWN_ERROR	500	Unmapped exception.	Report with the logged class name.
Unmapped <code>RuntimeError</code> from <code>request-access</code>	500	Telegram session unusable during a join (ISS-03).	Check <code>GET /api/telegram/status</code> ; treat as 401.

16.3 Errors that never surface as HTTP errors

Several failure modes are recorded as **state**, not raised:

Situation	What the caller sees
<code>FloodWaitError</code> during a collection cycle	Cycle stops, partial batch persisted, <code>checkpoint.error_count</code> incremented. The API is unaffected — the failure is visible only in audit logs and the checkpoint.
<code>ChannelPrivateError</code> during collection ("access revoked")	Source moves to <code>ERROR</code> , monitoring disabled, a <code>TELEGRAM_COLLECTION_ERROR</code> notification is written.
Access-check failure	Recorded in <code>access_status</code> / <code>last_probe_status</code> ; <code>POST /check-access</code> still returns 200 .
Bot <code>/start</code> timeout	<code>200</code> with <code>status: "timed_out"</code> .
Backfill failure	Job row becomes <code>FAILED</code> / <code>PARTIAL</code> with <code>error_message</code> ; the original call already returned 202.
Blocked state transition	Logged as a warning, <code>status_reason</code> records it, <code>access_status</code> is left unchanged, HTTP 200.
Notification dispatch failure	Logged; other dispatchers still run; the row is still marked processed.

Because of this design, **a 200 response does not mean the Telegram operation succeeded**. Always inspect `access_status`, `status`, or `error` in the body.

17. Current Implementation Status

Legend:  Implemented ·  Partially Implemented ·  Not Implemented ·  Restricted by Telegram

Feature	Status	Endpoint	Notes
Telegram authentication (code + 2FA)	✓	POST /api/telegram/auth/*	Non-interactive, three steps; pending hash is in-memory only.
Session reuse across restarts	✓	—	Telethon session file at <code>TELEGRAM_SESSION_PATH</code> .
Connection/auth status reporting	✓	GET /api/telegram/status	Masked phone; never leaks session data.
Invalid-session detection and latch	✓	all Telegram endpoints	Stops repeated RPCs with a known-bad key.
Source registration + normalization	✓	POST /api/sources	@username, t.me URL, invite hash, numeric id.
Duplicate prevention (identifier + entity)	✓	POST /api/sources	Two-pass dedup; 409 on either.
Live access probing	✓	POST /api/sources/{id}/check-access	Real read probe, never inference.
Access state machine (13 states)	✓	—	Validated allow-list; monitoring invariant enforced centrally.
Public source access	✓	POST /api/sources	<code>PUBLIC_ACCESSIBLE</code> .
Private source access via join request	⚠ 🔒	POST /api/sources/{id}/request-access	Submitted once; content only after Telegram approves.
Invite-link join	✓	POST /api/telegram/invites/join	Idempotent; never duplicates a request.
Join-request rejection detection	⚠ 🔒	(background job)	Invite-hash: yes. Username-based: no decline signal exists.
12-hour reconciliation	✓	(background job)	<code>max_instances=1</code> ; stops early on session invalidation.
Event-driven access fast path	✗	—	<code>event_handler.py</code> is complete but never wired in (DOC-01).
Automatic monitoring after approval	✓	(background job)	Starts only after membership is verified.
Start/stop monitoring	✓	POST /api/sources/{id}/monitoring/start and /stop	Start runs the first cycle synchronously.
Incremental collection with checkpoints	✓	(background job, every 5 min)	Per-message commits; restart-safe.
DB-level message de-duplication	✓	—	<code>UNIQUE(source_id, telegram_message_id) + ON CONFLICT DO NOTHING</code> .
Raw evidence preservation + SHA-256	✓	—	One JSON file per message under <code>data/raw_messages/</code> .
Media metadata capture	✓	—	Always recorded.

Feature	Status	Endpoint	Notes
Media download	⚠️	—	Implemented but off by default ; size/type policy applies.
Historical backfill	⚠️	POST <code>/api/sources/{id}/backfill</code>	Works; no crash recovery, no cancel, no per-source concurrency guard.
Live global search	⚠️ 🔒	GET <code>/api/telegram/search/global</code>	<code>searchPosts</code> is quota-gated; the fallback is account-scoped.
Live per-channel search	✅	GET <code>/api/sources/{id}/search</code>	Native sender/media/date filters.
Live arbitrary-channel search	✅	GET <code>/api/telegram/channels/{channel_id}/search</code>	Fresh probe on every call.
Multi-source search	⚠️	POST <code>/api/telegram/search/sources</code>	Works, but returns no pagination cursor .
Channel discovery	⚠️ 🔒	GET <code>/api/telegram/channels/search</code>	<code>contacts.search</code> offers no pagination; access status is a coarse hint.
Register a discovered channel	✅	POST <code>/api/telegram/channels/{telegram_id}/register</code>	Same path as <code>POST /api/sources</code> .
Telegram link extraction/classification	✅	POST <code>/api/telegram/discovery/extract-links</code>	Never fetches a URL; capped at 20 probes.
Save a search result	✅	POST <code>/api/telegram/search/results/save</code>	The only path search data reaches the DB.
Bot <code>/start</code> workflow	✅	POST <code>/api/telegram/bots/{source_id}/start</code>	One message; no callback execution; capped at 10 links.
Stored-message retrieval	✅	GET <code>/api/sources/{id}/messages</code> , GET <code>/api/messages/{id}</code>	No Telegram traffic.
Local message search	⚠️	GET <code>/api/messages</code>	Flat filters only — no boolean/nested queries, no ranking, no totals.
Notifications persisted	✅	GET <code>/api/notifications</code>	Written on access granted/rejected and collection errors.
Mark notification read	❌	POST <code>/api/notifications/{id}/read</code>	Returns 200 but does not persist (ISS-01).
Update a source (PATCH)	❌	PATCH <code>/api/sources/{id}</code>	Returns 200 but does not persist (ISS-01).
Notification dispatch (websocket/webhook/email)	❌	—	Registry exists; no provider registered .
Audit logging	✅	—	~30 event types; 5 declared but never emitted.

Feature	Status	Endpoint	Notes
Structured logging	✓	—	Single-line JSON; query text never logged.
Health/readiness probes	✓	GET /health , GET /ready	/ready reflects configuration only.
Swagger / OpenAPI docs	✓	/docs , /redoc , /openapi.json	Enabled and unauthenticated.
API authentication / authorization	✗	—	None anywhere (SEC-01).
API rate limiting	✗	—	Only a 3-way search concurrency cap.
FloodWait backoff / retry scheduling	✗	—	Reported, never enforced.
Multi-account support / rotation	✗	—	Single account by design.
Proxy support	✗	—	Not present.
Database migrations (Alembic)	✗	—	create_all + a SQLite-only additive patcher.
Message export (bulk CSV/JSON)	✗	—	Paged JSON API only.
NLP / OCR / entity extraction / correlation	✗	—	Explicit downstream scope.
Deployment tooling (Docker/CI)	✗	—	Not present.
Sending arbitrary messages	✗	—	Only the literal /start to a bot.
Reading reactions / polls / comment threads / stories	✗	—	Not requested from Telegram.
Participant/member enumeration	✗	—	No participants RPC anywhere.
Test suite	✓	—	189 test functions in 26 files; a coverage test forbids untested endpoints.

18. Known Issues and Recommendations

Three clearly separated categories: **defects in current behaviour**, **technical debt**, and **future improvements**. Nothing here is speculative — every item cites the code.

18.1 Current Issues (defects in behaviour today)

ISS-01 — Two endpoints return 200 but discard the change · High

`SourceService.update_source` and `NotificationService.mark_read` both call a repository method that `flush()` es without `commit()`, and neither caller commits. The FastAPI `get_db` dependency closes the session when the request ends, rolling the transaction back. Both endpoints serialize the in-memory object, so the response *looks* successful.

- Affected: `PATCH /api/sources/{source_id}`, `POST /api/notifications/{notification_id}/read`.
- Symptom: `?unread_only=true` keeps returning the same notification forever; a renamed source reverts on the next read.
- **Verified** by reproducing the exact dependency + repository pattern against SQLAlchemy 2.0.52 / aiomysql: the value read after the request was the original.
- Not caught by tests: the smoke test asserts only the status code, and no test re-reads after a PATCH.
- **Fix:** `await self.session.commit()` in each service method (one line each), plus a regression test that re-reads after mutating.

ISS-02 — Backfill jobs do not survive a restart · Medium

`start_backfill` fires `asyncio.create_task(...)` in the API process. A restart kills the task; the row stays `RUNNING` forever. There is no startup sweep, no cancel endpoint, and no per-source concurrency guard (several jobs can run against one source at once).

- **Fix:** on startup, mark orphaned `RUNNING` jobs as `FAILED` (or resume them from `checkpoint_message_id`); add `DELETE /api/sources/{id}/backfill/{job_id}` to cancel; refuse a second `RUNNING` job for the same source.

ISS-03 — `request-access` returns 500 when the session is unusable · Low

`SourceService.request_access` raises a bare `RuntimeError` when `verify_authorized()` fails, and `routes_sources` does not catch it — FastAPI turns it into a 500. Every comparable path returns 401 or 409.

- **Fix:** catch it in the route and return 409 (or a `TelegramSearchError` with 401) to match the rest of the API.

ISS-04 — `PATCH` bypasses the access-status invariant · Low

`update_source` writes `monitoring_enabled` straight through the repository, skipping `_apply_status`, the choke point that guarantees monitoring is never enabled from a blocked state. Currently masked by ISS-01 (nothing persists), so fixing ISS-01 without fixing this would *introduce* the inconsistency.

- **Fix:** route `monitoring_enabled` changes through the monitoring service, or re-validate the invariant inside `update_source`.

ISS-05 — Unbounded `limit` on stored-data endpoints · Low

`GET /api/messages`, `GET /api/sources/{id}/messages`, `GET /api/sources` and `GET /api/notifications` accept any integer `limit`. The live database already holds 12,805 messages, so `limit=1000000` is a self-inflicted denial of service.

- **Fix:** `Query(default=50, ge=1, le=200)` on all four.

ISS-06 — Stale `RUNNING` scheduler rows · Informational

An unclean shutdown leaves `scheduler_jobs.status = "RUNNING"` permanently — one such row exists in the live database. Purely an observability wart; the no-overlap guarantee comes from APScheduler's `max_instances=1`, not this table.

- **Fix:** reconcile stale rows on startup.

ISS-07 — Deleting a source orphans its evidence files · Low

`DELETE /api/sources/{id}` cascades in the database but leaves `data/raw_messages/<source_id>/` on disk, and (because SQLite foreign keys are off) leaves `audit_logs.source_id` / `notifications.source_id` pointing at a row that no longer exists.

- **Fix:** delete the evidence directory, and either enable `PRAGMA foreign_keys=ON` or call `detach_source` on every deletion path.

18.2 Documentation vs implementation discrepancies

The code is the source of truth; these are places where existing documents disagree with it.

DOC-01 — README claims an event-driven access fast path that is not wired in · Medium

README §11 describes "Two complementary mechanisms", the first being event-driven detection via `app/telegram/event_handler.py`. That module is complete and unit-tested, but `register_access_change_handlers` is **never called** anywhere in the application — a grep across `app/` finds only docstring mentions. In production, approval detection latency is therefore **up to 12 hours**, not near-real-time.

- **Fix:** either wire it into the lifespan (register the handler after a successful connect, with a callback that triggers `check_access` for the affected source) or correct the README.

DOC-02 — Test count is stale · Informational

README §17 and the closing summary both state "68 tests". The suite currently defines **189** test functions across 26 files.

DOC-03 — "Swap `DATABASE_URL` to Postgres" is not quite true · Low

The README states that switching to PostgreSQL is a one-line change because nothing above the repository layer touches a specific dialect. `MessageRepository.upsert_ignore_duplicate` imports `sqlalchemy.dialects.sqlite.insert` and uses `on_conflict_do_nothing` — this is the single dedup guarantee in the system, and it would need a dialect-aware variant. The additive-column patcher is also SQLite-only and silently no-ops elsewhere.

DOC-04 — The `poc/` technical report describes different software · Informational

`docs/technical-report.md` reviews the archived single-file proof-of-concept (`poc/main.py`), not this service. Its limits (100 results per call, 10 channels per run, no pagination, `--public-search` requiring Premium) **do not apply** to

`telegram_service`, which paginates, has no channel-count cap, and falls back rather than requiring Premium. Keep the two clearly separated during review.

18.3 Technical debt

#	Debt	Why it matters
TD-1	No migration tool. <code>create_all</code> plus a hand-maintained six-entry additive patch list, SQLite-only.	Any column rename/type change, or any non-SQLite deployment, is unmanaged. Adopt Alembic.
TD-2	Dependency floors with no lock file. Every requirement is <code>>=</code> .	Non-reproducible builds; a breaking Telethon release can arrive silently.
TD-3	Dead / unreachable code. <code>event_handler.py</code> (unused), the audit constants <code>TELEGRAM_AUTHENTICATED</code> , <code>MEDIA_DOWNLOADED</code> , <code>ERROR_OCCURRED</code> , <code>CHANNEL_REGISTERED_FROM_DISCOVERY</code> , the notification event <code>TELEGRAM_MONITORING_STARTED</code> , and the schemas <code>MessageSearchQuery</code> , <code>TelegramSearchErrorResponse</code> , <code>TelegramErrorDetail</code> (never referenced by a <code>response_model</code> , so absent from the OpenAPI schema).	Suggests capability that does not exist. Wire it up or remove it.
TD-4	telegram_accounts is nearly empty. <code>telegram_user_id</code> , <code>username</code> and <code>phone_masked</code> are never written by any code path — confirmed <code>NULL</code> in the live database.	The table implies account tracking that does not happen. Populate it in the health-check job or drop the columns.
TD-5	next_status_check_at is never written.	Always <code>null</code> in <code>SourceOut</code> / <code>AccessStatusOut</code> ; implies a re-probe schedule that does not exist.
TD-6	processing_status never leaves "raw".	The <code>normalized</code> / <code>enriched</code> states anticipate a downstream pipeline that is out of scope. Acceptable, but should be labelled.
TD-7	SourceCreate.source_type accepted and ignored.	Misleading contract; remove it or honour it as a hint.
TD-8	Duplicated media-filter maps. <code>_MEDIA_FILTERS</code> is defined identically in <code>global_search.py</code> and <code>channel_search.py</code> .	Minor drift risk; extract to one module.
TD-9	collection_method is unvalidated free text on <code>SaveSearchResultRequest</code> .	Any string reaches the database column; validate against the <code>CollectionMethod</code> enum.
TD-10	Multi-source search is sequential by necessity (one shared <code>AsyncSession</code>).	Correct as written, but caps throughput. Would need per-source sessions to parallelize — and a rate budget before that would be safe.
TD-11	Two error envelope shapes (<code>{"detail": ...}</code> vs <code>{"error": {...}}</code>).	Every client must handle both. Consider one envelope.
TD-12	No metrics endpoint. Logs and <code>audit_logs</code> are the only observability.	No Prometheus/OpenTelemetry surface for FloodWait rate, collection lag, or error rate.

18.4 Recommended improvements (future work — none of this exists today)

Priority 1 — before any non-local deployment

1. **Add API authentication and authorization** (SEC-01). An API-key dependency at router level is the minimum; treat auth/join/bot-start/delete as privileged.
2. **Bind to** `127.0.0.1` **by default** and change `.env.example` accordingly.
3. **Fix ISS-01** (two missing commits) and add regression tests that re-read after mutating.
4. **Terminate TLS** at a reverse proxy so login codes and the 2FA password are not sent in clear text.
5. **Bound every** `limit` (ISS-05).

Priority 2 — account safety

6. **Central FloodWait enforcement**: record the deadline and refuse outbound Telegram calls until it passes, instead of only reporting it.
7. **Global outbound request budget** in `rate_policy`, covering collection, access checks, joins and link extraction — not just search.
8. **Join-rate limiting** with a daily budget.
9. **Jitter and inter-source spacing** in the collection cycle.
10. **Account-health metrics and alerting** on `session_invalid`, FloodWait frequency and `ERROR` source count.

Priority 3 — correctness and operations

11. **Backfill lifecycle**: startup reconciliation of orphaned jobs, a cancel endpoint, and a per-source concurrency guard (ISS-02).
12. **Alembic migrations** (TD-1) and a dialect-agnostic upsert (DOC-03) before any Postgres move.
13. **Wire in or remove the event handler** (DOC-01); wiring it reduces approval latency from 12 hours to near-real-time.
14. **Enable SQLite foreign keys** or make every deletion path detach references (SEC-09, ISS-07).
15. **Pin dependencies / add a lock file** (TD-2).

Priority 4 — functionality

16. **Pagination for multi-source search** (AP-7).
17. **Richer local search**: boolean queries, a full-text index, result totals (AP-8).
18. **A notification dispatcher** (webhook is the smallest useful one) so state changes reach operators without polling.
19. **Retention and deletion policy** for collected content and raw evidence (SEC-05), including evidence cleanup on source deletion.
20. **An explicit policy decision on user/DM sources** (section 7.1) — either block `SourceType.USER / BOT` from monitoring, or document the behaviour deliberately.

19. API Sequence Diagrams

Only workflows that exist in the code are diagrammed.

19.1 Source registration with access verification



19.2 Private source: join request and 12-hour reconciliation

Client	API	SourceService	join_request_mgr	Telegram	Scheduler(12h)	Database
	POST /request-access					
----->						
		----->	active PENDING request?			
			----->			
			<-- none (else return it, NO RPC) -----			
			state == JOIN_REQUEST_REQUIRED? (else 409)			
			submit_join_request()			
			----->			
			joinChannel / importChatInvite			
			----->			
			<-- InviteRequestSentError --			
			<-- PENDING ----			
			INSERT access_request (next_check_at +12h) + COMMIT			
			----->			
<-- 200 (PENDING) -----						
				reconcile_pending_access		
				<-----		
			check_pending_status()			
			<-----			
			checkChatInvite / get_entity(left flag)			
			<----->			
			<-- ChatInviteAlready -----			
			APPROVED			
			<----->			
			JOIN_REQUEST_APPROVED -> JOINED, notification, COMMIT			
					----->	
			start_monitoring() AUTOMATICALLY (verified access)			
					----->	
GET /api/notifications						
----->						
<-- TELEGRAM_ACCESS_GRANTED -----						

19.3 Scheduled collection cycle

```

Scheduler      MonitoringService  ClientManager  collector  Telegram  MessageService  Database
|
| every COLLECTION_INTERVAL_MINUTES (max_instances=1)
|----->|
|
|   list sources WHERE monitoring_enabled AND status = MONITORING
|----->|
|   FOR EACH source:
|   verify_authorized()
|----->|
|   <-- ok / invalid --|
|   invalid -> source = ERROR, monitoring off, next source
|   get_entity
|----->|----->|
|   checkpoint.get_or_create()
|----->|
|   collect_new_messages(min_id, reverse=True, limit)
|----->|----->|
|                                     <-- messages / FloodWait / Private --|
|   <-- CollectionOutcome (partial batch kept on error) ---|
|   persist_collected_messages()
|----->|
|       FOR EACH message: raw evidence JSON -> INSERT ON CONFLICT
|                               DO NOTHING -> advance checkpoint -> COMMIT
|                               |----->|
|   update source.last_message_id / last_collected_at + audit + COMMIT
|----->|
|   <-- structured JSON log: collection_completed --|
|

```

19.4 Live global search with quota fallback

```

Client      API      SearchService  rate_policy  global_search  Telegram  Database
|
| GET /api/telegram/search/global?q=...
|----->|
|
|   ----->|
|   |
|   |----->|
|   |
|   | <-- ok (else 401, NO RPC issued) -----|
|   | run_bounded_search(key)
|   |----->|
|   |
|   | semaphore(3) + in-flight dedup
|   |----->|
|   |
|   | checkSearchPostsFlood
|   |----->|
|   | <-- remains / stars -----|
|   |
|   | free quota? --yes--> channels.searchPosts
|   | |----->| visibility=public
|   | --no--> warning, NO payment
|   | |
|   | | messages.searchGlobal
|   | |----->| account_scoped
|   |
|   | <-- results --|
|   |
|   | <-- outcome --|
|   |
|   | normalize -> local filters -> sort -> cursor
|   | audit SEARCH_GLOBAL (query_length only) + COMMIT
|   |----->|
|   |
|   | <-- response
|   |
|   | <-- 200 --| results + pagination + method_used + quota + warnings
|   |
|   |
|   | NOTHING IS PERSISTED. Only POST /api/telegram/search/results/save writes data.
|

```

19.5 Bot `/start` and link harvesting

Client	API	BotService	bot_interaction	Telegram	url_classifier	SourceService	DB
	POST /api/telegram/bots/{id}/start						
			load source; 404 / 422 if not a bot				
			verify_authorized -> get_entity				
			audit BOT_START_REQUESTED + COMMIT				
			start_bot()				
			sendMessage("/start")				
			poll iter_messages until reply or timeout				
			<-- reply ----				
			<-- outcome --				
			classify reply text + button URLs				
			<-- Telegram links only; others discarded, NEVER fetched -				
			cap at 10; FOR EACH link: register DORMANT (monitoring off)				
			audit LINK_DISCOVERED / INVITE_RESOLVED + COMMIT				
	<-- 200 --		status, response_text, buttons, discovered[]				

19.6 Backfill job (asynchronous)

Client	API	BackfillService	asyncio task	Telegram	MessageService	Database
	POST /api/sources/{id}/backfill					
			source exists? access verified? (404 / 409)			
			INSERT backfill_jobs (PENDING) + audit + COMMIT			
			create_task(_run_backfill_job)			
			<-- 202 (job id) -----			
			status = RUNNING; own DB session			
			verify_authorized -> get_entity			
			iter_messages BACKWARD			
			offset_id = job checkpoint, else offset_date			
			<-- older messages -----			
			persist_backfill_batch()			
			per message: evidence -> INSERT ON CONFLICT			
			advance job.checkpoint_message_id (OLDEST)			
			collection_checkpoints NEVER touched			
			COMPLETED PARTIAL FAILED + audit			
	GET /api/sources/{id}/backfill/{job_id}					
			<-- 200 (status, messages_inserted, checkpoint_message_id) -----			

20. Final Technical Summary

For a Team Leader, Software Architect or Engineering Manager. This section stands alone.

1. What does our Telegram API currently do?

It exposes **35 REST endpoints** that turn **one authenticated Telegram user account** into a controlled data-collection service. It registers Telegram channels/groups/bots as sources, determines each one's **real** access status by probing Telegram rather than assuming it, runs a legitimate join-request workflow for private sources with 12-hourly reconciliation, monitors accessible sources on a 5-minute checkpointed cycle, backfills history on demand, searches Telegram live, and stores everything in SQLite with per-message raw evidence and SHA-256 hashes. Its scope is deliberately **data collection and access management** — analysis is downstream.

Real usage to date: **12,805 messages collected across 4 sources over 71 completed collection cycles, with one recorded failure and no rate-limit incidents.**

2. What endpoints are available?

Group	Count	Highlights
Health / status	2	<code>/health</code> , <code>/ready</code>
Telegram session / auth	4	3-step non-interactive login, status
Source management	8	register, CRUD, request-access, check-access
Monitoring	3	start (runs the first cycle inline), stop, status
Stored messages	3	per-source, by id, flat-filter search
Live Telegram search	5	global, per-source, arbitrary channel, multi-source, save-result
Discovery	3	channel search, register discovered, extract links
Bots / invites	2	bot <code>/start</code> , join by invite
Backfill	3	start (202), list, poll
Notifications	2	list, mark read

3. How does the data flow?

Client -> FastAPI router -> Service -> (Telegram layer via Telethon | Repository layer via SQLAlchemy) -> Telegram MTPROTO | SQLite . Routes never touch Telethon or SQL; `app/telegram/*` is the only Telethon importer; repositories are the only query builders. Live search is **read-only and ephemeral** — the sole path that writes search data is `POST /api/telegram/search/results/save` . Collection persists one message per transaction, advancing a checkpoint only past durably committed rows.

4. What Telegram capabilities are available?

Read public channels and groups; read private sources the account has joined; search within any accessible channel; discover channels by keyword; join public channels and invite links; submit approval-gated join requests; monitor continuously; backfill history; capture media metadata (and optionally download bytes); send exactly one `/start` to a registered bot.

5. What Telegram capabilities are restricted?

- Private content before Telegram approves access — **hard stop, by design**.
- Cross-membership public search — quota-gated by Telegram; the fallback is **account-scoped**, not a search of all Telegram.
- Detecting a silently declined username-based join request — **no signal exists**.
- Channel discovery pagination — Telegram provides none.
- Deleted or restricted content — not recoverable.
- Sender identity, view and reply counts — frequently absent; stored as `NULL`, never fabricated.
- Reactions, polls, comment threads, stories, participant lists — **not implemented**.

6. What are the major technical limitations?

Platform: access equals one account's permissions; FloodWait; search quotas; no decline signal; history-visibility rules.

Application: no caller authentication; **two endpoints** (`PATCH /api/sources/{id}` , `POST /api/notifications/{id}/read`) **return 200 without persisting**; backfill does not survive a restart; no application rate limiting or FloodWait enforcement; single account and single process; no migrations; the README's event-driven fast path is not wired in, so approval latency is up to 12 hours.

7. What security considerations exist?

One High finding: the API has no authentication of any kind, and the default bind is `0.0.0.0` . Anyone who can reach the port can make the Telegram account join channels, send `/start` messages, drive search until Telegram rate-limits it, read all collected third-party content, and delete sources with their data. Swagger UI at `/docs` is an unauthenticated console over all of it.

Medium findings: the session file is an unencrypted bearer credential; login codes and the 2FA password cross the wire in clear text; a populated `.env` sits in the working tree; 12,805 collected messages are stored unencrypted with no retention policy; there is no API rate limiting.

Confirmed positives: credentials are never logged or returned; phone numbers are masked; search queries are logged only by length; no SSRF surface (non-Telegram URLs are never fetched); no SQL injection surface; secrets are correctly gitignored.

8. What should be improved next?

Before any non-local deployment 1. Add API authentication; bind to `127.0.0.1` ; put TLS in front. 2. Fix the two missing commits (**ISS-01**) — one line each, plus regression tests. 3. Bound every `limit` parameter.

Then, for account safety 4. Enforce FloodWait centrally instead of only reporting it. 5. Add a global outbound Telegram request budget covering collection, joins and link extraction — not just search. 6. Rate-limit joins; add jitter to collection cycles; alert on account health.

Then, for correctness and operations 7. Backfill lifecycle: startup reconciliation, cancellation, per-source concurrency guard. 8. Alembic migrations and a dialect-agnostic upsert before any PostgreSQL move. 9. Wire in or delete the event handler; correct the README where it disagrees with the code. 10. Decide policy on user/DM sources — the generic source path can monitor a one-to-one conversation, which is very likely not intended.

Overall assessment

The architecture is **sound and unusually disciplined for a proof of concept**: strict layering that holds under inspection, a validated state machine that refuses to guess access, restart-safe checkpointing backed by a database-level uniqueness guarantee, evidence preservation, comprehensive audit logging, and an explicit, well-documented refusal to bypass Telegram's controls. The test suite (189 tests, with a coverage guard that forbids untested endpoints) is a genuine strength.

The gaps are **operational, not architectural**: no authentication, two silent persistence bugs, and no enforcement of the rate-limit discipline the design otherwise respects. All are small, well-isolated changes. **Recommendation: keep the service on a trusted network only, fix ISS-01 and add authentication before any wider deployment, then address the account-safety backlog before increasing collection volume.**